Appendix: Analysis Scripts

R scripts for the fsQCA analysis (Chapter 3) and the regression analysis (Chapter 4)

Examining Multilevel Interconnectedness in Entrepreneurial Ecosystems: Within-Ecosystem and Between-Ecosystem Dynamics

Pallavpreet Kaur

2 November 2026

Contents

About These Scripts 3

Part I: fsQCA Analysis (Chapter 3) 4

I.1 Main Analysis 5

I.2 Robustness: Calibration Anchors and Consistency Threshold 8

I.3 Robustness: Unicorn Firms 11

I.4 Robustness: Frequency Threshold of Two Cases 13

Part II: Regression Analysis (Chapter 4) 15

II.A Primary (Lagged) Specification 16

II.A.1 Primary Specification 16

II.A.2 Cross-Effect Checks 27

II.A.3 Predicted Margins 30

II.A.4 Pairwise Comparisons of Average Marginal Effects 33

II.A.5 Robustness and Diagnostic Checks 35

II.B Robustness Specification 39

II.B.1 Robustness (Overlapping-Window) Specification 39

II.C Supplementary Checks and Candidate Controls 49

II.C.1 Alternative Moderator: KIBS Employment Share 49

II.C.2 Candidate Control: Business Churn 50

II.C.3 Candidate Controls: Net Migration Rate and Household Net Worth 51

II.C.4 Candidate Controls: Extended Set 54

II.C.5 Candidate Controls in the Robustness Specification 56

II.C.6 Candidate Control: Homelessness 59

II.C.7 Candidate Controls: Voluntary Work and Homelessness (Screening) 61

II.C.8 Omitted-Variable Sensitivity Check (N = 79) 63

About These Scripts

This appendix reproduces the R code used for the fuzzy-set qualitative comparative analysis in Chapter 3 and the regression analysis in Chapter 4. Each script begins with a header that states its purpose, the tables or figures it produces and, where relevant, the script that must be run before it.

Software. R with the packages `QCA` (Chapter 3) and `tidyverse`, `readxl`, `lubridate`, `janitor`, `igraph`, `lmtest`, `sandwich`, `car`, `MASS`, `marginaleffects` and `ggplot2` (Chapter 4). Heteroskedasticity-robust (HC1) standard errors are used throughout Chapter 4.

Data. The Chapter 3 scripts read `FSQCA_data.csv` from the working directory. The Chapter 4 scripts read their source files from a `data` folder in the working directory; file names are given at the top of each script.

Run order (Chapter 4). Script 1 is run first in a new R session, followed by Scripts 2–5 and 7–10, 12–14 in the same session (Script 2 must precede Scripts 3 and 4). Script 6 is self-contained and is run in a separate, new session; Script 11 is run after Script 6.

Script	Thesis tables and figures
`Paper2_Script1_Main_fsQCA.R`	Tables 3.2–3.6, B3, B4, B8–B10
`Paper2_Script2_Robustness_Calibration_Threshold.R`	Tables B5, B6; Section B.4
`Paper2_Script3_Robustness_Unicorn_TableB7.R`	Table B7
`Paper2_Script4_Robustness_Frequency2.R`	Tables B1, B2
`Paper3_Script1_Primary_Specification.R`	Tables 4.4–4.6
`Paper3_Script2_Cross_Effects_TablesC1_C2.R`	Tables C1, C2
`Paper3_Script3_Predicted_Margins_Figures4.2_4.3.R`	Figures 4.2, 4.3
`Paper3_Script4_Pairwise_Comparisons_Tables4.7_C9.R`	Tables 4.7, C9
`Paper3_Script5_Robustness_Diagnostics_TableC7.R`	Table C7; Section 4.4.5
`Paper3_Script6_Robustness_Specification_TablesC3_C6.R`	Tables C3–C6
`Paper3_Script7 to Script14`	Table C8; Section 4.4.5

Part I: fsQCA Analysis (Chapter 3)

I.1 Main Analysis

Paper2_Script1_Main_fsQCA.R

Tables 3.2–3.6, B3, B4 and B8–B10: descriptive statistics, correlations, necessity analysis (presence and absence), truth tables, and complex, parsimonious and intermediate solutions for both outcomes.

```
################################################################################
# PAPER 2 (Chapter 3) - SCRIPT 1 OF 4: MAIN fsQCA ANALYSIS
# Ecosystem Homeostasis and High-Growth Entrepreneurship, 88 Australian SA4 regions
#
# Produces: Table 3.2 (descriptives), Table 3.3 (correlations), Table 3.4
#           (necessity, presence and absence), Tables 3.5-3.6 (via the
#           solutions below), Tables B3-B4 (intermediate solutions), Tables
#           B8-B9 (truth tables) and Table B10 (calibration anchors).
# Input:    FSQCA_data.csv (raw, pre-transformation values).
#
# Parameters (as reported in Section 3.4):
#   Homeostasis:                 incl.cut = 0.80, pri.cut = 0.65, n.cut = 3
#   Highgrowth.entrepreneurship: incl.cut = 0.85, pri.cut = 0.65, n.cut = 3
#   Calibration: 25th / 50th / 75th percentile of the TRANSFORMED variable.
################################################################################

## ---- Packages ----------------------------------------------------------------
if (!requireNamespace("QCA", quietly = TRUE)) install.packages("QCA")
library(QCA)

## ---- Load raw data and clean column names ------------------------------------
data <- read.csv("FSQCA_data.csv", stringsAsFactors = FALSE, check.names = TRUE)

names(data) <- trimws(names(data))
names(data) <- gsub("\\.$", "", names(data))
data$SA4.region <- trimws(data$SA4.region)
rownames(data) <- data$SA4.region

outcome_A <- "Homeostasis"
outcome_B <- "Highgrowth.entrepreneurship"
condition_vars <- c(
  "Intermediaries", "Finance", "Digital.infrastructure",
  "Physical.infrastructure", "Talent", "Demand", "Culture",
  "New.knowledge", "Networks"
)
all_vars <- c(outcome_A, outcome_B, condition_vars)

missing_vars <- all_vars[!all_vars %in% names(data)]
if (length(missing_vars) > 0) {
  stop(sprintf("Missing expected column(s): %s. Check the raw file's column names.",
               paste(missing_vars, collapse = ", ")))
}
cat(sprintf("Loaded %d cases, %d causal conditions, 2 outcomes.\n",
            nrow(data), length(condition_vars)))

## ---- Transformation (applied BEFORE fuzzy calibration) -----------------------
## Conditions: z-score -> exponentiate -> normalise to mean = 1 -> cap at 5.
## Outcomes:   ratio to sample mean -> cap at 5.
transform_condition <- function(x) {
  z <- as.numeric(scale(x))
  e <- exp(z)
  pmin(e / mean(e), 5)
}
transform_outcome <- function(x) pmin(x / mean(x), 5)

transformed <- data.frame(row.names = rownames(data))
for (v in condition_vars) transformed[[v]] <- transform_condition(as.numeric(data[[v]]))
transformed[[outcome_A]] <- transform_outcome(as.numeric(data[[outcome_A]]))
transformed[[outcome_B]] <- transform_outcome(as.numeric(data[[outcome_B]]))

## ---- Direct-method fuzzy calibration (Ragin, 2008) ---------------------------
## Anchors are percentiles of the variable's own distribution:
##   full non-membership = p_low, crossover = p_mid, full membership = p_high.
## A constant of 0.001 is then added to every membership score below 1 (capped
## at 0.999) so that no case sits at exactly 0.50 and is dropped from the truth
## table (cf. Fiss, 2011). This is the adjustment used for ALL reported results.
calibrate_pct <- function(x, p_low, p_mid, p_high) {
```

```
  th <- quantile(x, probs = c(p_low, p_mid, p_high), na.rm = TRUE)
  if (length(unique(th)) < 3) stop("Percentile thresholds are not distinct.")
  fs <- calibrate(x, type = "fuzzy",
                  thresholds = c(e = th[[1]], cr = th[[2]], i = th[[3]]))
  below_one <- which(fs < 1)
  fs[below_one] <- pmin(fs[below_one] + 0.001, 0.999)
  fs
}

dir.exp <- setNames(rep(1, length(condition_vars)), condition_vars)  # presence expected

## ---- Helper: truth table + complex / parsimonious / intermediate solutions ---
run_solutions <- function(fs_df, outcome, incl.cut, pri.cut = 0.65, n.cut = 3, label) {
  cat("\n\n========================================\n")
  cat(label, "| incl.cut =", incl.cut, "| pri.cut =", pri.cut, "| n.cut =", n.cut, "\n")
  cat("========================================\n\n")

  tt <- truthTable(fs_df, outcome = outcome, conditions = condition_vars,
                   incl.cut = incl.cut, pri.cut = pri.cut, n.cut = n.cut,
                   show.cases = TRUE, sort.by = "incl")
  print(tt)

  n_retained <- sum(tt$tt$n >= n.cut)
  cat(sprintf("\nConfigurations meeting n.cut: %d; treated as remainders: %d of %d (%.0f%%)\n",
              n_retained, 2^length(condition_vars) - n_retained, 2^length(condition_vars),
              100 * (2^length(condition_vars) - n_retained) / 2^length(condition_vars)))

  sol_complex <- minimize(tt, details = TRUE, show.cases = TRUE)
  cat("\n--- COMPLEX SOLUTION ---\n"); print(sol_complex)

  sol_pars <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?", method = "CCubes"),
    error = function(e) { cat("PARSIMONIOUS SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_pars)) { cat("\n--- PARSIMONIOUS SOLUTION ---\n"); print(sol_pars) }

  sol_int <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?",
             dir.exp = dir.exp, method = "CCubes"),
    error = function(e) { cat("INTERMEDIATE SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_int)) { cat("\n--- INTERMEDIATE SOLUTION ---\n"); print(sol_int) }

  list(tt = tt, complex = sol_complex, parsimonious = sol_pars, intermediate = sol_int)
}

## ---- Calibration for the main analysis (25/50/75, transformed variables) ----
fs_df <- as.data.frame(lapply(transformed, function(v) calibrate_pct(v, 0.25, 0.50, 0.75)))
rownames(fs_df) <- rownames(transformed)

cat("\nDescriptive statistics of the transformed variables (Table 3.2):\n")
desc <- data.frame(
  N      = sapply(transformed, function(v) sum(!is.na(v))),
  Mean   = sapply(transformed, mean),
  SD     = sapply(transformed, sd),
  Min    = sapply(transformed, min),
  P25    = sapply(transformed, quantile, 0.25),
  Median = sapply(transformed, median),
  P75    = sapply(transformed, quantile, 0.75),
  Max    = sapply(transformed, max))
print(round(desc, 3))   # the P25 / Median / P75 columns are also the Table B10 anchors

cat("\nCorrelation matrix of the transformed variables (Table 3.3):\n")
print(round(cor(transformed[, c(outcome_A, outcome_B, condition_vars)]), 2))

## ---- Necessity analysis: presence AND absence (Table 3.4) -------------------
necessity_both <- function(fs_df, outcome) {
  pres <- pof(fs_df[, condition_vars], fs_df[[outcome]], relation = "necessity")
  neg_df <- 1 - fs_df[, condition_vars]
  names(neg_df) <- paste0("~", condition_vars)
  neg <- pof(neg_df, fs_df[[outcome]], relation = "necessity")
  list(presence = pres, absence = neg)
}

## ---- Sufficiency analysis ---------------------------------------------------
res_A <- run_solutions(fs_df, outcome_A, incl.cut = 0.80, pri.cut = 0.65, n.cut = 3,
                       label = "Homeostasis (main analysis)")
```

```
res_B <- run_solutions(fs_df, outcome_B, incl.cut = 0.85, pri.cut = 0.65, n.cut = 3,
                       label = "High-growth entrepreneurship (main analysis)")

nec_A <- necessity_both(fs_df, outcome_A)
nec_B <- necessity_both(fs_df, outcome_B)
cat("\n--- NECESSITY: Homeostasis (presence, then absence) ---\n")
print(nec_A$presence); print(nec_A$absence)
cat("\n--- NECESSITY: High-growth entrepreneurship (presence, then absence) ---\n")
print(nec_B$presence); print(nec_B$absence)

## ---- Export ----------------------------------------------------------------
sink("fsQCA_MAIN_Homeostasis.txt")
print(res_A$tt); print(res_A$complex); print(res_A$parsimonious); print(res_A$intermediate)
print(nec_A$presence); print(nec_A$absence)
sink()

sink("fsQCA_MAIN_HighGrowth.txt")
print(res_B$tt); print(res_B$complex); print(res_B$parsimonious); print(res_B$intermediate)
print(nec_B$presence); print(nec_B$absence)
sink()

cat("\nDone. Files written to:", getwd(), "\n")
```

I.2 Robustness: Calibration Anchors and Consistency Threshold

`Paper2_Script2_Robustness_Calibration_Threshold.R`

Tables B5 and B6 and the 0.90-threshold results in Section B.4. Only the causal-condition anchors move to the 20th/50th/80th percentiles; the outcome anchors stay at 25/50/75.

```
################################################################################
# PAPER 2 (Chapter 3) - SCRIPT 2 OF 4: ROBUSTNESS - CALIBRATION ANCHORS AND
#                                      CONSISTENCY THRESHOLD
#
# Produces: Tables B5 and B6 (alternative calibration) and the stricter-
#           threshold (0.90) results summarised in Section B.4 (not reported
#           as separate tables). The truth tables underlying Tables B5 and B6
#           are printed to the console but are not reproduced in the thesis.
# Input:    FSQCA_data.csv (raw, pre-transformation values).
#
# Design (Section 3.5.6) - each factor is varied independently of the other:
#   Alternative calibration: CAUSAL CONDITIONS recalibrated at the 20th / 50th /
#     80th percentile of the transformed variable. The OUTCOME anchors stay at
#     25th / 50th / 75th. Consistency thresholds stay at 0.80 / 0.85.
#   Stricter threshold: original 25/50/75 calibration, incl.cut raised to 0.90.
# All runs use the same transformation as the main analysis (Script 1) and
# n.cut = 3, pri.cut = 0.65.
################################################################################

## ---- Packages ----------------------------------------------------------------
if (!requireNamespace("QCA", quietly = TRUE)) install.packages("QCA")
library(QCA)

## ---- Load raw data and clean column names ------------------------------------
data <- read.csv("FSQCA_data.csv", stringsAsFactors = FALSE, check.names = TRUE)

names(data) <- trimws(names(data))
names(data) <- gsub("\\.$", "", names(data))
data$SA4.region <- trimws(data$SA4.region)
rownames(data) <- data$SA4.region

outcome_A <- "Homeostasis"
outcome_B <- "Highgrowth.entrepreneurship"
condition_vars <- c(
  "Intermediaries", "Finance", "Digital.infrastructure",
  "Physical.infrastructure", "Talent", "Demand", "Culture",
  "New.knowledge", "Networks"
)
all_vars <- c(outcome_A, outcome_B, condition_vars)

missing_vars <- all_vars[!all_vars %in% names(data)]
if (length(missing_vars) > 0) {
  stop(sprintf("Missing expected column(s): %s. Check the raw file's column names.",
               paste(missing_vars, collapse = ", ")))
}
cat(sprintf("Loaded %d cases, %d causal conditions, 2 outcomes.\n",
            nrow(data), length(condition_vars)))

## ---- Transformation (applied BEFORE fuzzy calibration) -----------------------
## Conditions: z-score -> exponentiate -> normalise to mean = 1 -> cap at 5.
## Outcomes:   ratio to sample mean -> cap at 5.
transform_condition <- function(x) {
  z <- as.numeric(scale(x))
  e <- exp(z)
  pmin(e / mean(e), 5)
}
transform_outcome <- function(x) pmin(x / mean(x), 5)

transformed <- data.frame(row.names = rownames(data))
for (v in condition_vars) transformed[[v]] <- transform_condition(as.numeric(data[[v]]))
transformed[[outcome_A]] <- transform_outcome(as.numeric(data[[outcome_A]]))
transformed[[outcome_B]] <- transform_outcome(as.numeric(data[[outcome_B]]))

## ---- Direct-method fuzzy calibration (Ragin, 2008) ---------------------------
## Anchors are percentiles of the variable's own distribution:
##   full non-membership = p_low, crossover = p_mid, full membership = p_high.
## A constant of 0.001 is then added to every membership score below 1 (capped
```

```
## at 0.999) so that no case sits at exactly 0.50 and is dropped from the truth
## table (cf. Fiss, 2011). This is the adjustment used for ALL reported results.
calibrate_pct <- function(x, p_low, p_mid, p_high) {
  th <- quantile(x, probs = c(p_low, p_mid, p_high), na.rm = TRUE)
  if (length(unique(th)) < 3) stop("Percentile thresholds are not distinct.")
  fs <- calibrate(x, type = "fuzzy",
                  thresholds = c(e = th[[1]], cr = th[[2]], i = th[[3]]))
  below_one <- which(fs < 1)
  fs[below_one] <- pmin(fs[below_one] + 0.001, 0.999)
  fs
}

dir.exp <- setNames(rep(1, length(condition_vars)), condition_vars)  # presence expected

## ---- Helper: truth table + complex / parsimonious / intermediate solutions ---
run_solutions <- function(fs_df, outcome, incl.cut, pri.cut = 0.65, n.cut = 3, label) {
  cat("\n\n========================================\n")
  cat(label, "| incl.cut =", incl.cut, "| pri.cut =", pri.cut, "| n.cut =", n.cut, "\n")
  cat("========================================\n\n")

  tt <- truthTable(fs_df, outcome = outcome, conditions = condition_vars,
                   incl.cut = incl.cut, pri.cut = pri.cut, n.cut = n.cut,
                   show.cases = TRUE, sort.by = "incl")
  print(tt)

  n_retained <- sum(tt$tt$n >= n.cut)
  cat(sprintf("\nConfigurations meeting n.cut: %d; treated as remainders: %d of %d (%.0f%%)\n",
              n_retained, 2^length(condition_vars) - n_retained, 2^length(condition_vars),
              100 * (2^length(condition_vars) - n_retained) / 2^length(condition_vars)))

  sol_complex <- minimize(tt, details = TRUE, show.cases = TRUE)
  cat("\n--- COMPLEX SOLUTION ---\n"); print(sol_complex)

  sol_pars <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?", method = "CCubes"),
    error = function(e) { cat("PARSIMONIOUS SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_pars)) { cat("\n--- PARSIMONIOUS SOLUTION ---\n"); print(sol_pars) }

  sol_int <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?",
             dir.exp = dir.exp, method = "CCubes"),
    error = function(e) { cat("INTERMEDIATE SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_int)) { cat("\n--- INTERMEDIATE SOLUTION ---\n"); print(sol_int) }

  list(tt = tt, complex = sol_complex, parsimonious = sol_pars, intermediate = sol_int)
}

## ---- Calibration sets ------------------------------------------------------
## Original (25/50/75) for every variable - used for the 0.90 threshold runs.
fs_2575 <- as.data.frame(lapply(transformed, function(v) calibrate_pct(v, 0.25, 0.50, 0.75)))
rownames(fs_2575) <- rownames(transformed)

## Alternative: conditions at 20/50/80, outcomes left at 25/50/75.
fs_alt <- fs_2575
for (v in condition_vars) fs_alt[[v]] <- calibrate_pct(transformed[[v]], 0.20, 0.50, 0.80)

cat("\n20th/50th/80th percentile anchors used for the causal conditions:\n")
print(round(data.frame(
  p20 = sapply(transformed[, condition_vars], quantile, 0.20),
  p50 = sapply(transformed[, condition_vars], quantile, 0.50),
  p80 = sapply(transformed[, condition_vars], quantile, 0.80)), 3))

## ---- Alternative calibration (Tables B5 and B6) ----------------------------
tableB5 <- run_solutions(fs_alt, outcome_A, incl.cut = 0.80, n.cut = 3,
  label = "TABLE B5: Homeostasis - alternative calibration (conditions 20/50/80)")
tableB6 <- run_solutions(fs_alt, outcome_B, incl.cut = 0.85, n.cut = 3,
  label = "TABLE B6: High-growth - alternative calibration (conditions 20/50/80)")

## ---- Stricter consistency threshold (Section B.4) --------------------------
strict_A <- run_solutions(fs_2575, outcome_A, incl.cut = 0.90, n.cut = 3,
  label = "Homeostasis - threshold-only (incl.cut = 0.90, calibration 25/50/75)")
strict_B <- run_solutions(fs_2575, outcome_B, incl.cut = 0.90, n.cut = 3,
  label = "High-growth - threshold-only (incl.cut = 0.90, calibration 25/50/75)")

## ---- Export ----------------------------------------------------------------
```

```
sink("fsQCA_robustness_calibration_and_threshold.txt")
for (r in list(tableB5, tableB6, strict_A, strict_B)) {
  print(r$tt); print(r$complex); print(r$parsimonious); print(r$intermediate)
}
sink()
cat("\nDone. File written to:", getwd(), "\n")
```

I.3 Robustness: Unicorn Firms

Paper2_Script3_Robustness_Unicorn_TableB7.R

Table B7: necessity analysis for unicorn presence, with the outcome calibrated crisply (count > 0 = 1) and the conditions calibrated on their untransformed values.

```
################################################################################
# PAPER 2 (Chapter 3) - SCRIPT 3 OF 4: ROBUSTNESS - UNICORN FIRMS (Table B7)
#
# Necessity analysis for unicorn presence (>USD 1bn valuation). Only 3 of 88
# regions host a unicorn (counts 1, 3 and 7), so percentile anchors would all
# be zero. The outcome is therefore calibrated CRISPLY: count > 0 = 1, else 0.
#
# Input: FSQCA_data.csv (raw). Needs the raw 'Unicorns' column.
#
# Calibration scale: for this analysis the nine causal conditions are
# calibrated at the 25th/50th/75th percentiles of their UNTRANSFORMED values
# (not of the z-score/exponential-transformed values used in Tables 3.4-3.6),
# as stated in the note to Table B7.
################################################################################

## ---- Packages ----------------------------------------------------------------
if (!requireNamespace("QCA", quietly = TRUE)) install.packages("QCA")
library(QCA)

## ---- Load raw data and clean column names ------------------------------------
data <- read.csv("FSQCA_data.csv", stringsAsFactors = FALSE, check.names = TRUE)

names(data) <- trimws(names(data))
names(data) <- gsub("\\.$", "", names(data))
data$SA4.region <- trimws(data$SA4.region)
rownames(data) <- data$SA4.region

outcome_A <- "Homeostasis"
outcome_B <- "Highgrowth.entrepreneurship"
condition_vars <- c(
  "Intermediaries", "Finance", "Digital.infrastructure",
  "Physical.infrastructure", "Talent", "Demand", "Culture",
  "New.knowledge", "Networks"
)
all_vars <- c(outcome_A, outcome_B, condition_vars)

missing_vars <- all_vars[!all_vars %in% names(data)]
if (length(missing_vars) > 0) {
  stop(sprintf("Missing expected column(s): %s. Check the raw file's column names.",
               paste(missing_vars, collapse = ", ")))
}
cat(sprintf("Loaded %d cases, %d causal conditions, 2 outcomes.\n",
            nrow(data), length(condition_vars)))

## ---- Transformation (applied BEFORE fuzzy calibration) -----------------------
## Conditions: z-score -> exponentiate -> normalise to mean = 1 -> cap at 5.
## Outcomes:   ratio to sample mean -> cap at 5.
transform_condition <- function(x) {
  z <- as.numeric(scale(x))
  e <- exp(z)
  pmin(e / mean(e), 5)
}
transform_outcome <- function(x) pmin(x / mean(x), 5)

transformed <- data.frame(row.names = rownames(data))
for (v in condition_vars) transformed[[v]] <- transform_condition(as.numeric(data[[v]]))
transformed[[outcome_A]] <- transform_outcome(as.numeric(data[[outcome_A]]))
transformed[[outcome_B]] <- transform_outcome(as.numeric(data[[outcome_B]]))

## ---- Direct-method fuzzy calibration (Ragin, 2008) ---------------------------
## Anchors are percentiles of the variable's own distribution:
##   full non-membership = p_low, crossover = p_mid, full membership = p_high.
## A constant of 0.001 is then added to every membership score below 1 (capped
## at 0.999) so that no case sits at exactly 0.50 and is dropped from the truth
## table (cf. Fiss, 2011). This is the adjustment used for ALL reported results.
calibrate_pct <- function(x, p_low, p_mid, p_high) {
  th <- quantile(x, probs = c(p_low, p_mid, p_high), na.rm = TRUE)
```

```
  if (length(unique(th)) < 3) stop("Percentile thresholds are not distinct.")
  fs <- calibrate(x, type = "fuzzy",
                  thresholds = c(e = th[[1]], cr = th[[2]], i = th[[3]]))
  below_one <- which(fs < 1)
  fs[below_one] <- pmin(fs[below_one] + 0.001, 0.999)
  fs
}

dir.exp <- setNames(rep(1, length(condition_vars)), condition_vars)  # presence expected

## ---- Helper: truth table + complex / parsimonious / intermediate solutions ---
run_solutions <- function(fs_df, outcome, incl.cut, pri.cut = 0.65, n.cut = 3, label) {
  cat("\n\n========================================\n")
  cat(label, "| incl.cut =", incl.cut, "| pri.cut =", pri.cut, "| n.cut =", n.cut, "\n")
  cat("========================================\n\n")

  tt <- truthTable(fs_df, outcome = outcome, conditions = condition_vars,
                   incl.cut = incl.cut, pri.cut = pri.cut, n.cut = n.cut,
                   show.cases = TRUE, sort.by = "incl")
  print(tt)

  n_retained <- sum(tt$tt$n >= n.cut)
  cat(sprintf("\nConfigurations meeting n.cut: %d; treated as remainders: %d of %d (%.0f%%)\n",
              n_retained, 2^length(condition_vars) - n_retained, 2^length(condition_vars),
              100 * (2^length(condition_vars) - n_retained) / 2^length(condition_vars)))

  sol_complex <- minimize(tt, details = TRUE, show.cases = TRUE)
  cat("\n--- COMPLEX SOLUTION ---\n"); print(sol_complex)

  sol_pars <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?", method = "CCubes"),
    error = function(e) { cat("PARSIMONIOUS SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_pars)) { cat("\n--- PARSIMONIOUS SOLUTION ---\n"); print(sol_pars) }

  sol_int <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?",
             dir.exp = dir.exp, method = "CCubes"),
    error = function(e) { cat("INTERMEDIATE SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_int)) { cat("\n--- INTERMEDIATE SOLUTION ---\n"); print(sol_int) }

  list(tt = tt, complex = sol_complex, parsimonious = sol_pars, intermediate = sol_int)
}

if (!"Unicorns" %in% names(data)) stop("Missing 'Unicorns' column in the raw data.")

## ---- Crisp outcome ------------------------------------------------------------
unicorn_crisp <- as.numeric(data$Unicorns > 0)
cat(sprintf("Regions with any unicorn firm: %d of %d\n", sum(unicorn_crisp), nrow(data)))
print(data.frame(region = rownames(data)[unicorn_crisp == 1],
                 unicorn_count = data$Unicorns[unicorn_crisp == 1]))

## ---- Conditions: 25/50/75 percentiles of the RAW values ----------------------
fs_raw <- as.data.frame(lapply(condition_vars, function(v)
  calibrate_pct(as.numeric(data[[v]]), 0.25, 0.50, 0.75)))
names(fs_raw) <- condition_vars
rownames(fs_raw) <- rownames(data)

## ---- Necessity (Table B7) -----------------------------------------------------
## RoN (relevance of necessity) is reported with inclN and covN: a condition can
## show high inclN simply because it is almost always present (trivial necessity).
nec_unicorn <- pof(fs_raw, unicorn_crisp, relation = "necessity")
cat("\nTABLE B7: necessary conditions for unicorn-firm presence\n")
print(nec_unicorn)

nec_tab <- as.data.frame(nec_unicorn$incl.cov)
cat("\nConditions with inclN >= 0.90:\n")
print(rownames(nec_tab)[nec_tab$inclN >= 0.90])

sink("fsQCA_TableB7_unicorn_necessity.txt")
print(nec_unicorn)
sink()
cat("\nDone. File written to:", getwd(), "\n")
```

I.4 Robustness: Frequency Threshold of Two Cases

Paper2_Script4_Robustness_Frequency2.R

Tables B1 and B2: the main analysis re-run with a frequency threshold (n.cut) of 2.

```
################################################################################
# PAPER 2 (Chapter 3) - SCRIPT 4 OF 4: ROBUSTNESS - FREQUENCY THRESHOLD OF TWO
#
# Produces: Tables B1 and B2 (solutions). The corresponding truth tables are
# printed to the console but are not reproduced in the thesis.
# Identical to the main analysis (Script 1) except n.cut = 2 instead of 3.
# Input: FSQCA_data.csv (raw).
################################################################################

## ---- Packages ----------------------------------------------------------------
if (!requireNamespace("QCA", quietly = TRUE)) install.packages("QCA")
library(QCA)

## ---- Load raw data and clean column names ------------------------------------
data <- read.csv("FSQCA_data.csv", stringsAsFactors = FALSE, check.names = TRUE)

names(data) <- trimws(names(data))
names(data) <- gsub("\\.$", "", names(data))
data$SA4.region <- trimws(data$SA4.region)
rownames(data) <- data$SA4.region

outcome_A <- "Homeostasis"
outcome_B <- "Highgrowth.entrepreneurship"
condition_vars <- c(
  "Intermediaries", "Finance", "Digital.infrastructure",
  "Physical.infrastructure", "Talent", "Demand", "Culture",
  "New.knowledge", "Networks"
)
all_vars <- c(outcome_A, outcome_B, condition_vars)

missing_vars <- all_vars[!all_vars %in% names(data)]
if (length(missing_vars) > 0) {
  stop(sprintf("Missing expected column(s): %s. Check the raw file's column names.",
               paste(missing_vars, collapse = ", ")))
}
cat(sprintf("Loaded %d cases, %d causal conditions, 2 outcomes.\n",
            nrow(data), length(condition_vars)))

## ---- Transformation (applied BEFORE fuzzy calibration) -----------------------
## Conditions: z-score -> exponentiate -> normalise to mean = 1 -> cap at 5.
## Outcomes:   ratio to sample mean -> cap at 5.
transform_condition <- function(x) {
  z <- as.numeric(scale(x))
  e <- exp(z)
  pmin(e / mean(e), 5)
}
transform_outcome <- function(x) pmin(x / mean(x), 5)

transformed <- data.frame(row.names = rownames(data))
for (v in condition_vars) transformed[[v]] <- transform_condition(as.numeric(data[[v]]))
transformed[[outcome_A]] <- transform_outcome(as.numeric(data[[outcome_A]]))
transformed[[outcome_B]] <- transform_outcome(as.numeric(data[[outcome_B]]))

## ---- Direct-method fuzzy calibration (Ragin, 2008) ---------------------------
## Anchors are percentiles of the variable's own distribution:
##   full non-membership = p_low, crossover = p_mid, full membership = p_high.
## A constant of 0.001 is then added to every membership score below 1 (capped
## at 0.999) so that no case sits at exactly 0.50 and is dropped from the truth
## table (cf. Fiss, 2011). This is the adjustment used for ALL reported results.
calibrate_pct <- function(x, p_low, p_mid, p_high) {
  th <- quantile(x, probs = c(p_low, p_mid, p_high), na.rm = TRUE)
  if (length(unique(th)) < 3) stop("Percentile thresholds are not distinct.")
  fs <- calibrate(x, type = "fuzzy",
                  thresholds = c(e = th[[1]], cr = th[[2]], i = th[[3]]))
  below_one <- which(fs < 1)
  fs[below_one] <- pmin(fs[below_one] + 0.001, 0.999)
  fs
}
```

```
dir.exp <- setNames(rep(1, length(condition_vars)), condition_vars)  # presence expected

## ---- Helper: truth table + complex / parsimonious / intermediate solutions ---
run_solutions <- function(fs_df, outcome, incl.cut, pri.cut = 0.65, n.cut = 3, label) {
  cat("\n\n========================================\n")
  cat(label, "| incl.cut =", incl.cut, "| pri.cut =", pri.cut, "| n.cut =", n.cut, "\n")
  cat("========================================\n\n")

  tt <- truthTable(fs_df, outcome = outcome, conditions = condition_vars,
                   incl.cut = incl.cut, pri.cut = pri.cut, n.cut = n.cut,
                   show.cases = TRUE, sort.by = "incl")
  print(tt)

  n_retained <- sum(tt$tt$n >= n.cut)
  cat(sprintf("\nConfigurations meeting n.cut: %d; treated as remainders: %d of %d (%.0f%%)\n",
              n_retained, 2^length(condition_vars) - n_retained, 2^length(condition_vars),
              100 * (2^length(condition_vars) - n_retained) / 2^length(condition_vars)))

  sol_complex <- minimize(tt, details = TRUE, show.cases = TRUE)
  cat("\n--- COMPLEX SOLUTION ---\n"); print(sol_complex)

  sol_pars <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?", method = "CCubes"),
    error = function(e) { cat("PARSIMONIOUS SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_pars)) { cat("\n--- PARSIMONIOUS SOLUTION ---\n"); print(sol_pars) }

  sol_int <- tryCatch(
    minimize(tt, details = TRUE, show.cases = TRUE, include = "?",
             dir.exp = dir.exp, method = "CCubes"),
    error = function(e) { cat("INTERMEDIATE SOLUTION FAILED:", conditionMessage(e), "\n"); NULL })
  if (!is.null(sol_int)) { cat("\n--- INTERMEDIATE SOLUTION ---\n"); print(sol_int) }

  list(tt = tt, complex = sol_complex, parsimonious = sol_pars, intermediate = sol_int)
}

fs_df <- as.data.frame(lapply(transformed, function(v) calibrate_pct(v, 0.25, 0.50, 0.75)))
rownames(fs_df) <- rownames(transformed)

tableB1 <- run_solutions(fs_df, outcome_A, incl.cut = 0.80, pri.cut = 0.65, n.cut = 2,
  label = "TABLE B1: Homeostasis - frequency threshold = 2")
tableB2 <- run_solutions(fs_df, outcome_B, incl.cut = 0.85, pri.cut = 0.65, n.cut = 2,
  label = "TABLE B2: High-growth entrepreneurship - frequency threshold = 2")

sink("fsQCA_robustness_frequency_threshold2.txt")
for (r in list(tableB1, tableB2)) {
  print(r$tt); print(r$complex); print(r$parsimonious); print(r$intermediate)
}
sink()
cat("\nDone. File written to:", getwd(), "\n")
```

Part II: Regression Analysis (Chapter 4)

II.A Primary (Lagged) Specification

Scripts 1–5 are run in sequence in the same R session. All explanatory variables and controls are measured up to 2020, the moderators come from the 2016 Census, and both outcomes are measured over 2021–2024.

II.A.1 Primary Specification

`Paper3_Script1_Primary_Specification.R`

Constructs all variables from the source data (explanatory variables and controls ≤ 2020; moderators from the 2016 Census; outcomes 2021–2024) and estimates Table 4.4 (VIFs) and Tables 4.5 and 4.6.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 1: PRIMARY (LAGGED) SPECIFICATION
#
# Builds every variable from the raw source files, assembles the analysis
# datasets and estimates the models reported in Table 4.4 (variance
# inflation factors) and Tables 4.5-4.6 (the two eight-model regression
# tables). Run top to bottom in a fresh R session; Scripts 2-5 are then run
# in the same session.
#
# Temporal design: the dependent variables are measured over a later period
# than every explanatory variable, moderator and control.
#   - Explanatory variables and controls: measured up to and including 2020.
#   - Moderators (Ethnic Diversity, Creative-Class Share): 2016 Census wave.
#   - Entrepreneurial Ecosystem Homeostasis: 2021-2024.
#   - Productive Entrepreneurship: firms founded 2021-2024, per 1,000
#     residents.
# ============================================================================


# ============================================================================
# PART 0. LIBRARIES, PATHS, HELPER FUNCTIONS
# ============================================================================
library(tidyverse)
library(readxl)
library(lubridate)
library(janitor)
library(igraph)
library(lmtest)
library(sandwich)
library(car)

# All input files are read from a single project data folder.
data_dir <- "data"

f_postal_sa4          <- file.path(data_dir, "Postal to SA4.xlsx")
f_overlap_poa         <- file.path(data_dir, "overlap_poa_details.csv")
f_sa4_lookup          <- file.path(data_dir, "SA4 codes and names .xlsx")
f_vc_deals            <- file.path(data_dir, "Final VC Deals.xlsx")
f_migration_17_21     <- file.path(data_dir, "Internal_migration_SA4_2017_2021.csv")
f_migration_22_25     <- file.path(data_dir, "Internal_migration_SA4_2022_2025.csv")
f_population          <- file.path(data_dir, "resident_population.csv")
f_business_churn      <- file.path(data_dir, "Business_Churn_SA4_202124.xlsx")
f_homeostasis_raw     <- file.path(data_dir, "Dependent Variable_Homeostasis.csv")
f_uv_anzsic           <- file.path(data_dir, "UV_ANZSIC_2021.xlsx")
f_mv1_2016            <- file.path(data_dir, "MV1_ED_SA4_2016.csv")
f_mv2_2016            <- file.path(data_dir, "mv2_CC_2016.csv")
f_specialisation_2016 <- file.path(data_dir, "specialisation_index_2016_FINAL.csv")
f_employed_pre2021    <- file.path(data_dir, "Employed_SA4_201425.xlsx")
f_income              <- file.path(data_dir, "md_hh_inc.csv")
f_age                 <- file.path(data_dir, "demographic 25-64.csv")
f_unemployment        <- file.path(data_dir, "unemployment_finalrate.csv")
f_org_parts           <- file.path(data_dir, paste0("organizations_part_", 1:7, ".csv"))
f_all_sa4_data        <- file.path(data_dir, "All SA4s.data.csv")

clean_sa4 <- function(x) x %>% str_to_lower() %>% str_trim() %>% str_squish()
zscore    <- function(x) as.numeric(scale(x))

robust_summary <- function(model) coeftest(model, vcov = vcovHC(model, type = "HC1"))

stars_from_p <- function(p) {
```

```
  case_when(p < 0.001 ~ "***", p < 0.01 ~ "**", p < 0.05 ~ "*", p < 0.1 ~ "+", TRUE ~ "")
}

compute_se_directed <- function(edges) {
  if (nrow(edges) == 0) return(data.frame(sa4_code21 = character(0), SE = numeric(0)))
  edge_weights <- edges %>% count(investor_sa4, investee_sa4, name = "w_ij")
  g <- graph_from_data_frame(edge_weights, directed = TRUE)
  adj <- as_adjacency_matrix(g, attr = "w_ij", sparse = FALSE)
  nodes <- V(g)$name
  se_values <- numeric(length(nodes)); names(se_values) <- nodes
  for (i in nodes) {
    neigh <- which(adj[i, ] > 0 | adj[, i] > 0)
    k <- length(neigh)
    if (k <= 1) { se_values[i] <- 0; next }
    sub_adj <- adj[neigh, neigh]
    v_i <- sum(sub_adj > 0); max_edges <- k * (k - 1)
    se_values[i] <- ifelse(max_edges == 0, 0, v_i / max_edges)
  }
  data.frame(sa4_code21 = names(se_values), SE = se_values)
}

fmt_cell <- function(raw_model, std_model, term_raw, term_z) {
  rs <- robust_summary(raw_model)
  if (!term_raw %in% rownames(rs)) return(c("", "", ""))
  est   <- rs[term_raw, "Estimate"]
  se    <- rs[term_raw, "Std. Error"]
  p     <- rs[term_raw, "Pr(>|t|)"]
  stars <- stars_from_p(p)
  beta  <- if (!is.null(std_model) && !is.null(term_z) && term_z %in% names(coef(std_model))) {
    sprintf("%.3f", coef(std_model)[term_z])
  } else { "" }
  c(sprintf("%.3f%s", est, stars), sprintf("(%.3f)", se), sprintf("[%s]", beta))
}

print_results_table <- function(raw_models, std_models, model_labels, var_specs) {
  for (v in seq_along(var_specs)) {
    label    <- names(var_specs)[v]
    term_raw <- var_specs[[v]]$raw
    term_z   <- var_specs[[v]]$z
    cat(sprintf("%-32s", label))
    lines <- lapply(seq_along(raw_models), function(m) {
      if (is.null(raw_models[[m]])) return(c("", "", ""))
      fmt_cell(raw_models[[m]], std_models[[m]], term_raw, term_z)
    })
    cat(paste(sapply(lines, `[`, 1), collapse = " | "), "\n")
    cat(sprintf("%-32s", ""))
    cat(paste(sapply(lines, `[`, 2), collapse = " | "), "\n")
    cat(sprintf("%-32s", ""))
    cat(paste(sapply(lines, `[`, 3), collapse = " | "), "\n")
  }
  cat(strrep("-", 100), "\n")
  cat(sprintf("%-32s", "Observations"))
  cat(paste(sapply(raw_models, function(m) if (is.null(m)) "" else nobs(m)), collapse = " | "), "\n")
  cat(sprintf("%-32s", "Adjusted R-squared"))
  cat(paste(sapply(raw_models, function(m) if (is.null(m)) "" else sprintf("%.3f",
summary(m)$adj.r.squared)), collapse = " | "), "\n")
  cat(sprintf("%-32s", "AIC"))
  cat(paste(sapply(raw_models, function(m) if (is.null(m)) "" else sprintf("%.1f", AIC(m))), collapse = " |
"), "\n")
  cat("\nModel columns:", paste(model_labels, collapse = " | "), "\n")
  cat("Robust (HC1) SE in parentheses (); standardised coefficient in brackets [].\n")
  cat("*** p<0.001  ** p<0.01  * p<0.05  + p<0.1\n\n")
}


# ============================================================================
# PART 1. SA4 LOOKUP + POSTAL-TO-SA4 CROSSWALK (static crosswalk; not subject
# to the temporal restriction)
# ============================================================================
sa4_lookup_clean <- read_excel(f_sa4_lookup) %>%
  transmute(sa4_code = as.character(SA4_CODE_2021),
            sa4_name = str_to_lower(str_trim(SA4_NAME_2021))) %>%
  distinct() %>%
  filter(!is.na(sa4_code), !is.na(sa4_name), sa4_name != "na",
         !str_detect(sa4_code, "ZZZ"),
```

```
         !str_detect(sa4_name, "migratory|offshore|no usual address"))

postal_sa4_raw <- read_excel(f_postal_sa4) %>%
  clean_names() %>%
  select(poa_code21, sa4_code21, sa4_name21) %>%
  mutate(poa_code21 = as.character(poa_code21)) %>%
  distinct()

overlap_override <- read_csv(f_overlap_poa, show_col_types = FALSE) %>%
  clean_names() %>%
  mutate(poa_code21 = str_pad(as.character(poa_code21), width = 4, pad = "0"))

postal_sa4 <- postal_sa4_raw %>%
  left_join(overlap_override, by = "poa_code21") %>%
  filter(is.na(sa4_region) | sa4_name21 == sa4_region) %>%
  select(poa_code21, sa4_code21, sa4_name21) %>%
  distinct()

stopifnot(!any(duplicated(postal_sa4$poa_code21)))

all_sa4   <- postal_sa4 %>% distinct(sa4_code21, sa4_name21)
all_nodes <- unique(as.character(all_sa4$sa4_code21))

population_clean <- read_csv(f_population, show_col_types = FALSE) %>%
  mutate(sa4_name = clean_sa4(region_name), year = as.integer(year),
         population = `Estimated resident population`) %>%
  select(sa4_name, year, population) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  select(sa4 = sa4_code, year, population)

population_avg_leq2020 <- population_clean %>%
  filter(year <= 2020) %>%
  group_by(sa4) %>% summarise(population_avg_leq2020 = mean(population, na.rm = TRUE), .groups = "drop")

cat(sprintf("[PART 1] postal_sa4: %d POAs, %d SA4s\n",
            n_distinct(postal_sa4$poa_code21), length(all_nodes)))


# ============================================================================
# PART 2. MIGRATION CONCENTRATION (se1)
# Herfindahl-type concentration of each region's bilateral migration flows,
# computed yearly and averaged over 2017-2020 (the migration data begin in
# 2017; the window is restricted to <=2020).
# ============================================================================
migration_raw <- bind_rows(
  read_csv(f_migration_17_21, show_col_types = FALSE),
  read_csv(f_migration_22_25, show_col_types = FALSE)
) %>%
  transmute(year = as.numeric(TIME_PERIOD), i = as.character(REGION_DEP),
            j = as.character(REGION_ARR), weight = as.numeric(OBS_VALUE)) %>%
  filter(!is.na(weight), weight > 0, i != j) %>%
  filter(i %in% all_nodes, j %in% all_nodes) %>%
  filter(year <= 2020)   # temporal restriction: predictors <= 2020

cat(sprintf("[PART 2] Migration data restricted to years: %d-%d\n",
            min(migration_raw$year), max(migration_raw$year)))

flows <- migration_raw %>% group_by(year, i, j) %>%
  summarise(x_ij = sum(weight, na.rm = TRUE), .groups = "drop")

pair_key <- flows %>% mutate(lo = pmin(i, j), hi = pmax(i, j),
                             direction = ifelse(i == lo, "lo_to_hi", "hi_to_lo"))

bilateral <- pair_key %>%
  select(year, lo, hi, direction, x_ij) %>%
  pivot_wider(names_from = direction, values_from = x_ij, values_fill = 0) %>%
  mutate(lo_to_hi = if ("lo_to_hi" %in% names(.)) lo_to_hi else 0,
         hi_to_lo = if ("hi_to_lo" %in% names(.)) hi_to_lo else 0) %>%
  transmute(year, i = lo, j = hi, w_ij = lo_to_hi + hi_to_lo) %>%
  filter(!is.na(i), !is.na(j), i != j)

se1_yearly <- bilateral %>%
  transmute(year, ego = i, partner = j, w_ij) %>%
  bind_rows(bilateral %>% transmute(year, ego = j, partner = i, w_ij)) %>%
  group_by(year, ego) %>%
```

```
  mutate(W_i = sum(w_ij, na.rm = TRUE), p_ij = ifelse(W_i > 0, w_ij / W_i, 0)) %>%
  summarise(SE = sum(p_ij^2, na.rm = TRUE), .groups = "drop")

se1_avg <- se1_yearly %>%
  group_by(ego) %>%
  summarise(se1_avg = mean(SE, na.rm = TRUE), .groups = "drop") %>%
  transmute(sa4 = as.character(ego), se1_avg)

cat(sprintf("[PART 2] se1_avg (2017-2020 only): %d regions\n", nrow(se1_avg)))

# ---- Network size (migration) - candidate control, not used in the models --
# Number of distinct partner regions, averaged over the same <=2020 panel,
# divided by 100 (see Table C8).
network_size_migration <- bilateral %>%
  transmute(year, ego = i, partner = j) %>%
  bind_rows(bilateral %>% transmute(year, ego = j, partner = i)) %>%
  distinct(year, ego, partner) %>%
  group_by(year, ego) %>%
  summarise(n_partners = n_distinct(partner), .groups = "drop") %>%
  group_by(ego) %>%
  summarise(network_size_migration_avg = mean(n_partners, na.rm = TRUE), .groups = "drop") %>%
  transmute(sa4 = as.character(ego), network_size_migration = network_size_migration_avg / 100)

cat(sprintf("[PART 2] network_size_migration (2017-2020 avg, /100): %d regions\n",
    nrow(network_size_migration)))


# ============================================================================
# PART 3. FINANCIAL NETWORK EMBEDDEDNESS (se2)
# Ego-network density of each region in the directed venture-capital
# investor-investee network. The window covers all VC deal years up to and
# including 2020, capped at ten years; with deal data beginning in 2014 this
# is 2014-2020, as reported in Table 4.1. The years used are printed below.
# ============================================================================
vc_edges <- read_excel(f_vc_deals) %>%
  clean_names() %>%
  separate_rows(investee_company_zip_code, sep = "\\|") %>%
  separate_rows(firm_investors_zip_code, sep = "\\|") %>%
  mutate(investee_company_zip_code = trimws(as.character(investee_company_zip_code)),
         firm_investors_zip_code   = trimws(as.character(firm_investors_zip_code))) %>%
  left_join(postal_sa4, by = c("investee_company_zip_code" = "poa_code21")) %>%
  rename(investee_sa4 = sa4_code21) %>%
  left_join(postal_sa4, by = c("firm_investors_zip_code" = "poa_code21")) %>%
  rename(investor_sa4 = sa4_code21) %>%
  mutate(year = year(ymd(investment_date))) %>%
  filter(!is.na(investor_sa4), !is.na(investee_sa4),
         investor_sa4 != investee_sa4, !is.na(year))

edges_year_all <- vc_edges %>% select(year, investor_sa4, investee_sa4) %>% distinct()
edges_year_leq2020 <- edges_year_all %>% filter(year <= 2020)

if (nrow(edges_year_leq2020) == 0) {
  stop("[PART 3] FATAL: no VC deal data exists at or before 2020 - Financial Network Embeddedness cannot be
computed.")
}

max_vc_year_lagged <- max(edges_year_leq2020$year)
min_vc_year_lagged <- min(edges_year_leq2020$year)
n_years_available <- max_vc_year_lagged - min_vc_year_lagged + 1
cat(sprintf("[PART 3] VC deal data at/before 2020 spans %d-%d (%d years available).\n",
            min_vc_year_lagged, max_vc_year_lagged, n_years_available))

# VC deal window: all years <= 2020, capped at ten years (2014-2020 here)
edges_10yr <- edges_year_leq2020 %>% filter(year >= max_vc_year_lagged - 9, year <= max_vc_year_lagged)

se2_avg <- compute_se_directed(edges_10yr) %>%
  transmute(sa4 = as.character(sa4_code21), se2_avg = SE)

cat(sprintf("[PART 3] se2_avg (<=2020 window): %d regions with observed VC ties (zero-filled for the rest at
merge)\n", nrow(se2_avg)))


# ============================================================================
# PART 3b. BURT'S CONSTRAINT AND VC DEAL VOLUME (controls, finance models)
# Computed on the same <=2020 VC edge window as Financial Network Embeddedness.
```

```
# ============================================================================
burt_constraint_raw <- edges_10yr %>%
  count(investor_sa4, investee_sa4, name = "w_ij") %>%
  graph_from_data_frame(directed = TRUE)

burt_constraint <- tryCatch({
  co <- constraint(burt_constraint_raw, weights = E(burt_constraint_raw)$w_ij)
  data.frame(sa4 = as.character(V(burt_constraint_raw)$name), burt_constraint = as.numeric(co))
}, error = function(e) {
  cat("[PART 3b] WARNING: constraint() failed -", conditionMessage(e), "\n")
  data.frame(sa4 = character(0), burt_constraint = numeric(0))
}) %>%
  filter(!is.na(burt_constraint))

in_deals_pc_final <- edges_10yr %>%
  group_by(investee_sa4) %>%
  summarise(in_deals = n(), .groups = "drop") %>%
  rename(sa4 = investee_sa4) %>%
  left_join(population_avg_leq2020, by = "sa4") %>%
  mutate(in_deals_pc = (in_deals / population_avg_leq2020) * 1000) %>%
  select(sa4, in_deals_pc)

cat(sprintf("[PART 3b] burt_constraint: %d regions | in_deals_pc: %d regions\n",
            nrow(burt_constraint), nrow(in_deals_pc_final)))

# ---- Network size (VC) - candidate control, not used in the models ---------
# Number of distinct tie partners in the <=2020 VC network, divided by 100
# (see Table C8).
network_size_vc <- edges_10yr %>%
  transmute(ego = investor_sa4, partner = investee_sa4) %>%
  bind_rows(edges_10yr %>% transmute(ego = investee_sa4, partner = investor_sa4)) %>%
  distinct(ego, partner) %>%
  count(ego, name = "n_partners") %>%
  transmute(sa4 = as.character(ego), network_size_vc = n_partners / 100)

cat(sprintf("[PART 3b] network_size_vc (<=2020 window, /100): %d regions\n", nrow(network_size_vc)))


# ============================================================================
# PART 4. DEPENDENT VARIABLE 1: ENTREPRENEURIAL ECOSYSTEM HOMEOSTASIS
# Sum of three standardised components (each z-scored across SA4 regions):
#   (i)   log(1 + business entries per 1,000 residents), 2021-2024;
#   (ii)  inverted-U transformation of industry unrelated variety
#         (min-max normalised entropy u, transformed as u - u^2);
#   (iii) log(1 + KIBS employment share).
# Standardising each component before summing gives the three components
# equal weight in the index (see Section 4.3 and the note to Table 4.2).
# ============================================================================
churn_raw <- read_excel(f_business_churn) %>%
  mutate(region = Region, year = as.integer(TIME_PERIOD)) %>%
  filter(`Data Item` == "Total business entries (no.)") %>%
  transmute(sa4_name = clean_sa4(region), year, entries = as.numeric(OBS_VALUE))

real_entries <- churn_raw %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, year, entries) %>%
  filter(!is.na(sa4))

ed_cross <- real_entries %>%
  filter(year %in% 2021:2024) %>%
  group_by(sa4) %>%
  summarise(entries_cumulative = sum(entries, na.rm = TRUE), .groups = "drop") %>%
  left_join(population_clean %>% filter(year == 2024) %>%
              select(sa4, population_2024 = population), by = "sa4") %>%
  mutate(ed_cross = (entries_cumulative / population_2024) * 1000)

uv_data_2digit <- read_excel(f_uv_anzsic) %>%
  clean_names() %>%
  rename(sa4 = sa4_ur, industry = x2_digit_level_indp_industry_of_employment, count = count) %>%
  select(sa4, industry, count) %>%
  filter(!str_detect(industry, regex("nfd", ignore_case = TRUE))) %>%
  mutate(industry_2d = case_when(
    str_detect(industry, "Agriculture|Forestry|Fishing") ~ "A", str_detect(industry, "Mining") ~ "B",
    str_detect(industry, "Manufacturing") ~ "C", str_detect(industry, "Electricity|Gas|Water|Waste") ~ "D",
    str_detect(industry, "Construction") ~ "E", str_detect(industry, "Wholesale") ~ "F",
```

```
    str_detect(industry, "Retail") ~ "G", str_detect(industry, "Accommodation|Food") ~ "H",
    str_detect(industry, "Transport|Postal|Warehousing") ~ "I",
    str_detect(industry, "Information|Media|Telecommunications") ~ "J",
    str_detect(industry, "Financial|Insurance") ~ "K", str_detect(industry, "Rental|Hiring|Real Estate") ~
"L",
    str_detect(industry, "Professional|Scientific|Technical") ~ "M",
    str_detect(industry, "Administrative|Support") ~ "N", str_detect(industry, "Public Administration") ~
"O",
    str_detect(industry, "Education") ~ "P", str_detect(industry, "Health") ~ "Q",
    str_detect(industry, "Arts|Recreation") ~ "R", str_detect(industry, "Other Services") ~ "S",
    TRUE ~ NA_character_
  )) %>%
  filter(!is.na(industry_2d))

uv_cross <- uv_data_2digit %>%
  group_by(sa4, industry_2d) %>% summarise(count = sum(count), .groups = "drop") %>%
  group_by(sa4) %>% mutate(total_emp = sum(count), p_g = count / total_emp) %>%
  summarise(UV = sum(p_g * log2(1 / pmax(p_g, 1e-12))), .groups = "drop") %>%
  mutate(sa4_name = str_to_lower(str_trim(sa4))) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, uv = UV) %>%
  filter(!is.na(sa4)) %>%
  mutate(uv_norm = (uv - min(uv, na.rm = TRUE)) / (max(uv, na.rm = TRUE) - min(uv, na.rm = TRUE)),
         uv_nl = uv_norm - uv_norm^2)

raw_homeo <- read_csv(f_homeostasis_raw, show_col_types = FALSE) %>%
  clean_names() %>% rename(sa4 = sa4_region) %>%
  mutate(sa4_name = str_to_lower(str_trim(sa4))) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  mutate(sa4 = sa4_code) %>% select(-sa4_name, -sa4_code)

kibs_2023_cross <- raw_homeo %>% select(sa4, kibs_2023) %>% rename(kibs = kibs_2023) %>% filter(!is.na(sa4))

homeostasis_cross <- ed_cross %>%
  select(sa4, ed_cross) %>%
  left_join(uv_cross %>% select(sa4, uv_nl), by = "sa4") %>%
  left_join(kibs_2023_cross, by = "sa4") %>%
  mutate(ed_log     = log(1 + ed_cross),
         kibs_log   = log(1 + kibs),
         z_ed_log   = as.numeric(scale(ed_log)),
         z_uv_nl    = as.numeric(scale(uv_nl)),
         z_kibs_log = as.numeric(scale(kibs_log)),
         homeostasis = z_ed_log + z_uv_nl + z_kibs_log)

cat(sprintf("[PART 4] homeostasis_cross (standardised components, 2021-2024): %d regions\n",
nrow(homeostasis_cross)))


# ============================================================================
# PART 5. DEPENDENT VARIABLE 2: PRODUCTIVE ENTREPRENEURSHIP
# New and innovative firms (Crunchbase) founded 2021-2024, matched to SA4 by
# postcode, per 1,000 residents (average population 2021-2024).
# ============================================================================
org_all <- map_dfr(f_org_parts, read.csv)

org_aus <- org_all %>%
  filter(country_code == "AUS") %>%
  mutate(founded_on = parse_date_time(founded_on, orders = c("ymd", "dmy", "mdy", "Y"), quiet = TRUE),
         year_founded = year(founded_on),
         postal_code_clean = str_extract(postal_code, "\\b\\d{4}\\b")) %>%
  filter(year_founded >= 2021, year_founded <= 2024)   # outcome window

org_sa4_valid <- org_aus %>%
  left_join(postal_sa4, by = c("postal_code_clean" = "poa_code21")) %>%
  filter(!is.na(sa4_code21)) %>%
  group_by(uuid, year_founded, sa4_code21, sa4_name21) %>%
  slice(1) %>% ungroup()

new_firms_total <- org_sa4_valid %>%
  distinct(uuid, sa4_code21) %>%
  count(sa4_code21, name = "total_firms") %>%
  transmute(sa4 = as.character(sa4_code21), total_firms)

population_avg_2021_2024 <- population_clean %>%
  filter(year >= 2021, year <= 2024) %>%
```

```
  group_by(sa4) %>% summarise(population_avg_2021_2024 = mean(population, na.rm = TRUE), .groups = "drop")

# ---- Zero-filling ------------------------------------------------------
# count() returns rows only for regions with at least one firm, so the
# complete SA4 list is used as the base and regions with no firms founded
# in 2021-2024 are assigned a count of zero (the same convention used for
# the VC-based measures).
all_sa4_for_firms <- sa4_lookup_clean %>% distinct(sa4 = sa4_code)

new_firms_percapita <- all_sa4_for_firms %>%
  left_join(new_firms_total, by = "sa4") %>%
  mutate(total_firms = replace_na(total_firms, 0)) %>%
  left_join(population_avg_2021_2024, by = "sa4") %>%
  mutate(new_firms_percapita = (total_firms / population_avg_2021_2024) * 1000)

cat(sprintf("[PART 5] new_firms_percapita (2021-2024 founding, zero-filled): %d total regions (%d with >=1
firm, %d zero-filled)\n",
            nrow(new_firms_percapita), sum(new_firms_percapita$total_firms > 0),
                sum(new_firms_percapita$total_firms == 0)))

# ---- Distribution of firm counts ---------------------------------------
all_sa4_codes <- sa4_lookup_clean$sa4_code
regions_with_zero_firms <- setdiff(all_sa4_codes, new_firms_total$sa4)
cat(sprintf("\n[PART 5] SPARSITY CHECK: %d of %d total SA4 regions have ZERO matched firms founded 2021-
2024.\n",
            length(regions_with_zero_firms), length(all_sa4_codes)))
cat("[PART 5] Distribution of firm counts among regions with >=1 firm:\n")
print(summary(new_firms_total$total_firms))
cat(sprintf("[PART 5] Regions with exactly 1 firm: %d | Regions with <=3 firms: %d\n",
            sum(new_firms_total$total_firms == 1), sum(new_firms_total$total_firms <= 3)))


# ============================================================================
# PART 6. MODERATORS (2016 CENSUS WAVE) AND POPULATION DENSITY
#   Ethnic Diversity: standardised entropy index (entropy_std) from
#     MV1_ED_SA4_2016.csv.
#   Creative-Class Share: creative_class_share from mv2_CC_2016.csv.
#   Population density (control): ABS Data by Region, measure ERP_21
#     (persons per km2), averaged over years <= 2020.
# ============================================================================
abs_regional_raw <- read_csv(f_all_sa4_data, show_col_types = FALSE)

# ---- Population density, averaged over years <= 2020 -----------------------
density_year_col <- intersect(c("TIME_PERIOD", "YEAR", "Year", "year"), names(abs_regional_raw))
if (length(density_year_col) > 0) {
  density_avg <- abs_regional_raw %>%
    filter(REGIONTYPE == "SA4", MEASURE == "ERP_21", !is.na(OBS_VALUE),
           .data[[density_year_col[1]]] <= 2020) %>%
    mutate(sa4 = as.character(ASGS_2021)) %>%
    group_by(sa4) %>%
    summarise(population_density_avg = mean(OBS_VALUE, na.rm = TRUE), .groups = "drop")
  cat(sprintf("[PART 6] Density restricted to <=2020 using column '%s'.\n", density_year_col[1]))
} else {
  density_avg <- abs_regional_raw %>%
    filter(REGIONTYPE == "SA4", MEASURE == "ERP_21", !is.na(OBS_VALUE)) %>%
    mutate(sa4 = as.character(ASGS_2021)) %>%
    group_by(sa4) %>%
    summarise(population_density_avg = mean(OBS_VALUE, na.rm = TRUE), .groups = "drop")
  cat("[PART 6] WARNING: no year column found in the ABS file; density averaged over all years.\n")
}

mv1_2016 <- read_csv(f_mv1_2016, show_col_types = FALSE) %>%
  clean_names() %>%
  transmute(sa4_name = str_to_lower(str_trim(sa4_ur)), entropy = entropy_std)

entropy_2016 <- mv1_2016 %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, entropy)

ccs_2016 <- read_csv(f_mv2_2016, show_col_types = FALSE) %>%
  clean_names() %>%
  mutate(sa4_name = str_to_lower(str_trim(sa4_ur))) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, creative_class_share)
```

```
cat(sprintf("[PART 6] density_avg: %d | entropy_2016: %d | ccs_2016: %d\n",
            nrow(density_avg), nrow(entropy_2016), nrow(ccs_2016)))


# ============================================================================
# PART 7. CONTROLS: median household income (log), unemployment rate,
# working-age population (share aged 25-64) and employment rate, each
# averaged over 2014-2020. The Specialisation Index (2016) is added in
# Part 7b.
# ============================================================================
employment_panel <- read_excel(f_employed_pre2021) %>%
  mutate(sa4_name = clean_sa4(`Region of usual residence (SA4)`), year = as.integer(year),
         employed = as.numeric(Employed) * 1000) %>%
  filter(year >= 2014, year <= 2020) %>%
  select(sa4_name, year, employed) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  select(sa4 = sa4_code, year, employed)

income_clean <- read_csv(f_income, show_col_types = FALSE) %>%
  mutate(sa4_name = clean_sa4(sa4_name), year = as.integer(year)) %>%
  filter(year <= 2020) %>%
  select(sa4_name, year, log_med_income) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  select(sa4 = sa4_code, year, log_med_income)

age_clean <- read_csv(f_age, show_col_types = FALSE) %>%
  mutate(sa4_name = clean_sa4(region_name), year = as.integer(date),
         age_25_64 = `Proportion of Estimated Resident Population that is 25-64 years old`) %>%
  filter(year <= 2020) %>%
  select(sa4_name, year, age_25_64) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  select(sa4 = sa4_code, year, age_25_64)

unemployment_clean <- read_csv(f_unemployment, show_col_types = FALSE) %>%
  mutate(sa4_code = str_extract(labour_market_region_sa4_asgs_2011, "^\\d+"), year = as.integer(year)) %>%
  filter(year <= 2020) %>%
  select(sa4 = sa4_code, year, unemployment_rate)

controls_all <- employment_panel %>%
  full_join(income_clean, by = c("sa4", "year")) %>%
  full_join(age_clean, by = c("sa4", "year")) %>%
  full_join(population_clean %>% filter(year <= 2020), by = c("sa4", "year")) %>%
  full_join(unemployment_clean, by = c("sa4", "year")) %>%
  mutate(employment_rate = employed / population)

controls_avg <- controls_all %>%
  group_by(sa4) %>%
  summarise(
    employment_rate_avg   = mean(employment_rate, na.rm = TRUE),
    log_med_income_avg    = mean(log_med_income, na.rm = TRUE),
    age_25_64_avg         = mean(age_25_64, na.rm = TRUE),
    unemployment_rate_avg = mean(unemployment_rate, na.rm = TRUE),
    .groups = "drop"
  )

cat(sprintf("[PART 7] controls_avg (ALL <=2020): %d regions\n", nrow(controls_avg)))

# ============================================================================
# PART 7b. SPECIALISATION INDEX (2016)
# Read from the 2016 specialisation index file (one row per SA4 region).
# ============================================================================
specialisation_2016 <- read_csv(f_specialisation_2016, show_col_types = FALSE) %>%
  transmute(sa4 = as.character(sa4), specialisation_index = specialisation_index_2016)

cat(sprintf("[PART 7b] specialisation_2016: %d regions\n", nrow(specialisation_2016)))
unmatched_spec_2016 <- specialisation_2016 %>% filter(!sa4 %in% sa4_lookup_clean$sa4_code)
if (nrow(unmatched_spec_2016) > 0) {
  cat(sprintf("[PART 7b] WARNING: %d specialisation_index_2016 SA4 codes do not match the pipeline's code
list:\n",
              nrow(unmatched_spec_2016)))
  print(unmatched_spec_2016)
} else {
  cat("[PART 7b] OK: all specialisation_index_2016 SA4 codes match.\n")
}
```

```r
# ============================================================================
# PART 7c. BUSINESS CHURN - NOT AVAILABLE FOR <= 2020
# Business entries and exits (flow measures) have no observations before
# 2021 in the ABS source file, so business churn cannot be measured within
# the predictor window and is not included (see Table C8).
# ============================================================================
business_churn <- tibble(sa4 = character(0), business_churn = numeric(0))
cat("[PART 7c] Business churn not available for <=2020; not included in the models.\n")


# ============================================================================
# PART 8. ASSEMBLE ANALYSIS DATASETS; MEAN-CENTRE AND STANDARDISE
# ============================================================================
homeostasis_analysis <- homeostasis_cross %>%
  left_join(se1_avg, by = "sa4") %>%
  left_join(density_avg, by = "sa4") %>%
  left_join(entropy_2016, by = "sa4") %>%
  left_join(ccs_2016, by = "sa4") %>%
  left_join(controls_avg, by = "sa4") %>%
  left_join(network_size_migration, by = "sa4") %>%
  left_join(business_churn, by = "sa4") %>%
  left_join(specialisation_2016, by = "sa4")

finance_analysis <- new_firms_percapita %>%
  left_join(se2_avg, by = "sa4") %>%
  mutate(se2_avg = replace_na(se2_avg, 0)) %>%
  left_join(density_avg, by = "sa4") %>%
  left_join(entropy_2016, by = "sa4") %>%
  left_join(ccs_2016, by = "sa4") %>%
  left_join(controls_avg, by = "sa4") %>%
  left_join(burt_constraint, by = "sa4") %>%
  mutate(burt_constraint = replace_na(burt_constraint, 0)) %>%
  left_join(in_deals_pc_final, by = "sa4") %>%
  mutate(in_deals_pc = replace_na(in_deals_pc, 0)) %>%
  left_join(network_size_vc, by = "sa4") %>%
  mutate(network_size_vc = replace_na(network_size_vc, 0)) %>%
  left_join(business_churn, by = "sa4") %>%
  left_join(specialisation_2016, by = "sa4")

homeostasis_analysis <- homeostasis_analysis %>% mutate(density_100 = population_density_avg / 100)
finance_analysis <- finance_analysis %>% mutate(density_100 = population_density_avg / 100)

homeostasis_analysis <- homeostasis_analysis %>%
  mutate(se1_c = se1_avg - mean(se1_avg, na.rm = TRUE),
         ccs_c = creative_class_share - mean(creative_class_share, na.rm = TRUE),
         entropy_c = entropy - mean(entropy, na.rm = TRUE))

finance_analysis <- finance_analysis %>%
  mutate(se2_c = se2_avg - mean(se2_avg, na.rm = TRUE),
         ccs_c = creative_class_share - mean(creative_class_share, na.rm = TRUE),
         entropy_c = entropy - mean(entropy, na.rm = TRUE))

homeostasis_analysis <- homeostasis_analysis %>%
  mutate(z_homeostasis = zscore(homeostasis), z_se1 = zscore(se1_avg),
         z_ccs = zscore(creative_class_share), z_entropy = zscore(entropy),
         z_density = zscore(population_density_avg),
         z_income = zscore(log_med_income_avg), z_unemp = zscore(unemployment_rate_avg),
         z_age = zscore(age_25_64_avg), z_emprate = zscore(employment_rate_avg),
         z_network_size_mig = zscore(network_size_migration), z_specialisation =
             zscore(specialisation_index))

finance_analysis <- finance_analysis %>%
  mutate(z_new_firms = zscore(new_firms_percapita), z_se2 = zscore(se2_avg),
         z_ccs = zscore(creative_class_share), z_entropy = zscore(entropy),
         z_density = zscore(population_density_avg),
         z_income = zscore(log_med_income_avg), z_unemp = zscore(unemployment_rate_avg),
         z_age = zscore(age_25_64_avg), z_emprate = zscore(employment_rate_avg),
         z_burtc = zscore(burt_constraint), z_indeals = zscore(in_deals_pc),
         z_network_size_vc = zscore(network_size_vc), z_specialisation = zscore(specialisation_index))

cat(sprintf("[PART 8] homeostasis_analysis: %d rows | finance_analysis: %d rows\n",
            nrow(homeostasis_analysis), nrow(finance_analysis)))

# ---- Missing-value check for the network-size variables -------------------
```

```
cat("\n[PART 8] Missing values in the network-size variables:\n")
cat(sprintf("  homeostasis_analysis$network_size_migration: %d NA of %d\n",
            sum(is.na(homeostasis_analysis$network_size_migration)), nrow(homeostasis_analysis)))
cat(sprintf("  finance_analysis$network_size_vc:             %d NA of %d\n",
            sum(is.na(finance_analysis$network_size_vc)), nrow(finance_analysis)))
if (all(is.na(homeostasis_analysis$network_size_migration))) {
  cat("  network_size_migration is entirely missing - check the Part 2 edge list.\n")
}

# ---- Variance inflation factors (Table 4.4) -------------------------------
vif_check_homeo <- lm(homeostasis ~ se1_c + ccs_c + entropy_c +
                        log_med_income_avg + unemployment_rate_avg + age_25_64_avg +
                        employment_rate_avg + density_100 + specialisation_index,
                      data = homeostasis_analysis)
cat("\n[PART 8] VIF - Homeostasis main-effects model (Table 4.4):\n")
homeo_vif <- vif(vif_check_homeo)
print(homeo_vif)
cat(sprintf("[PART 8] MEAN VIF (Homeostasis): %.3f\n", mean(homeo_vif)))

vif_check_finance <- lm(new_firms_percapita ~ se2_c + ccs_c + entropy_c +
                          log_med_income_avg + unemployment_rate_avg + age_25_64_avg +
                          employment_rate_avg + density_100 + burt_constraint + in_deals_pc +
specialisation_index,
                        data = finance_analysis)
cat("\n[PART 8] VIF - Productive Entrepreneurship main-effects model (Table 4.4):\n")
fin_vif <- vif(vif_check_finance)
print(fin_vif)
cat(sprintf("[PART 8] MEAN VIF (Productive Entrepreneurship): %.3f\n", mean(fin_vif)))


# ============================================================================
# PART 9. TABLE 4.5: HOMEOSTASIS ~ MIGRATION CONCENTRATION x ETHNIC
# DIVERSITY x CREATIVE-CLASS SHARE (Models 1-8)
# ============================================================================
# Business churn and the two network-size variables are not included in the
# models (Table C8): business churn is unavailable for <= 2020, migration
# network size has almost no variance across regions, and VC network size is
# highly correlated with the Productive Entrepreneurship outcome.
controls_formula_base   <- "density_100 + log_med_income_avg + unemployment_rate_avg + age_25_64_avg +
employment_rate_avg + specialisation_index"
controls_formula_base_z <- "z_density + z_income + z_unemp + z_age + z_emprate + z_specialisation"
controls_formula_homeo   <- controls_formula_base
controls_formula_homeo_z <- controls_formula_base_z

f_h1 <- as.formula(paste("homeostasis ~ se1_c +", controls_formula_homeo))
f_h2 <- as.formula(paste("homeostasis ~ ccs_c +", controls_formula_homeo))
f_h3 <- as.formula(paste("homeostasis ~ entropy_c +", controls_formula_homeo))
f_h4 <- as.formula(paste("homeostasis ~ se1_c + ccs_c + entropy_c +", controls_formula_homeo))
f_h5 <- as.formula(paste("homeostasis ~ se1_c * entropy_c + ccs_c +", controls_formula_homeo))
f_h6 <- as.formula(paste("homeostasis ~ entropy_c * ccs_c + se1_c +", controls_formula_homeo))
f_h7 <- as.formula(paste("homeostasis ~ se1_c * ccs_c + entropy_c +", controls_formula_homeo))
f_h8 <- as.formula(paste("homeostasis ~ se1_c * entropy_c * ccs_c +", controls_formula_homeo))

f_h1z <- as.formula(paste("z_homeostasis ~ z_se1 +", controls_formula_homeo_z))
f_h2z <- as.formula(paste("z_homeostasis ~ z_ccs +", controls_formula_homeo_z))
f_h3z <- as.formula(paste("z_homeostasis ~ z_entropy +", controls_formula_homeo_z))
f_h4z <- as.formula(paste("z_homeostasis ~ z_se1 + z_ccs + z_entropy +", controls_formula_homeo_z))
f_h5z <- as.formula(paste("z_homeostasis ~ z_se1 * z_entropy + z_ccs +", controls_formula_homeo_z))
f_h6z <- as.formula(paste("z_homeostasis ~ z_entropy * z_ccs + z_se1 +", controls_formula_homeo_z))
f_h7z <- as.formula(paste("z_homeostasis ~ z_se1 * z_ccs + z_entropy +", controls_formula_homeo_z))
f_h8z <- as.formula(paste("z_homeostasis ~ z_se1 * z_entropy * z_ccs +", controls_formula_homeo_z))

homeo_raw <- lapply(list(f_h1, f_h2, f_h3, f_h4, f_h5, f_h6, f_h7, f_h8), lm, data =
    homeostasis_analysis)
homeo_std <- lapply(list(f_h1z, f_h2z, f_h3z, f_h4z, f_h5z, f_h6z, f_h7z, f_h8z), lm, data =
homeostasis_analysis)

homeo_var_specs <- list(
  "Migration Concentration"         = list(raw = "se1_c",                  z = "z_se1"),
  "Creative-Class Share"            = list(raw = "ccs_c",                  z = "z_ccs"),
  "Ethnic Diversity"                = list(raw = "entropy_c",              z = "z_entropy"),
  "MC x Ethnic Diversity"           = list(raw = "se1_c:entropy_c",        z = "z_se1:z_entropy"),
  "Ethnic Diversity x CCS"          = list(raw = "entropy_c:ccs_c",        z = "z_entropy:z_ccs"),
  "MC x CCS"                        = list(raw = "se1_c:ccs_c",            z = "z_se1:z_ccs"),
  "MC x Ethnic Diversity x CCS"     = list(raw = "se1_c:entropy_c:ccs_c",  z = "z_se1:z_entropy:z_ccs"),
```

```
  "Population Density"              = list(raw = "density_100",              z = "z_density"),
  "Median Income (log)"             = list(raw = "log_med_income_avg",       z = "z_income"),
  "Unemployment Rate"               = list(raw = "unemployment_rate_avg",    z = "z_unemp"),
  "Working-Age Population"          = list(raw = "age_25_64_avg",            z = "z_age"),
  "Employment Rate"                 = list(raw = "employment_rate_avg",      z = "z_emprate"),
  "Specialisation Index"            = list(raw = "specialisation_index",     z = "z_specialisation")
)

cat("\n\n==========================================================================\n")
cat("TABLE 4.5. Effects of Migration Concentration on Entrepreneurial Ecosystem\n")
cat("Homeostasis (primary lagged specification).\n")
cat("==========================================================================\n")
print_results_table(
  homeo_raw, homeo_std,
  model_labels = c("(1)", "(2)", "(3)", "(4)", "(5)", "(6)", "(7)", "(8)"),
  var_specs = homeo_var_specs
)


# ============================================================================
# PART 10. TABLE 4.6: PRODUCTIVE ENTREPRENEURSHIP ~ FINANCIAL NETWORK
# EMBEDDEDNESS x ETHNIC DIVERSITY x CREATIVE-CLASS SHARE (Models 1-8)
# ============================================================================
controls_formula_fin   <- paste(controls_formula_base, "+ burt_constraint + in_deals_pc")
controls_formula_fin_z <- paste(controls_formula_base_z, "+ z_burtc + z_indeals")

f_f1 <- as.formula(paste("new_firms_percapita ~ se2_c +", controls_formula_fin))
f_f2 <- as.formula(paste("new_firms_percapita ~ ccs_c +", controls_formula_fin))
f_f3 <- as.formula(paste("new_firms_percapita ~ entropy_c +", controls_formula_fin))
f_f4 <- as.formula(paste("new_firms_percapita ~ se2_c + ccs_c + entropy_c +", controls_formula_fin))
f_f5 <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c + ccs_c +", controls_formula_fin))
f_f6 <- as.formula(paste("new_firms_percapita ~ entropy_c * ccs_c + se2_c +", controls_formula_fin))
f_f7 <- as.formula(paste("new_firms_percapita ~ se2_c * ccs_c + entropy_c +", controls_formula_fin))
f_f8 <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +", controls_formula_fin))

f_f1z <- as.formula(paste("z_new_firms ~ z_se2 +", controls_formula_fin_z))
f_f2z <- as.formula(paste("z_new_firms ~ z_ccs +", controls_formula_fin_z))
f_f3z <- as.formula(paste("z_new_firms ~ z_entropy +", controls_formula_fin_z))
f_f4z <- as.formula(paste("z_new_firms ~ z_se2 + z_ccs + z_entropy +", controls_formula_fin_z))
f_f5z <- as.formula(paste("z_new_firms ~ z_se2 * z_entropy + z_ccs +", controls_formula_fin_z))
f_f6z <- as.formula(paste("z_new_firms ~ z_entropy * z_ccs + z_se2 +", controls_formula_fin_z))
f_f7z <- as.formula(paste("z_new_firms ~ z_se2 * z_ccs + z_entropy +", controls_formula_fin_z))
f_f8z <- as.formula(paste("z_new_firms ~ z_se2 * z_entropy * z_ccs +", controls_formula_fin_z))

fin_raw <- lapply(list(f_f1, f_f2, f_f3, f_f4, f_f5, f_f6, f_f7, f_f8), lm, data = finance_analysis)
fin_std <- lapply(list(f_f1z, f_f2z, f_f3z, f_f4z, f_f5z, f_f6z, f_f7z, f_f8z), lm, data = finance_analysis)

fin_var_specs <- list(
  "Financial Network Embeddedness"  = list(raw = "se2_c",                    z = "z_se2"),
  "Creative-Class Share"            = list(raw = "ccs_c",                    z = "z_ccs"),
  "Ethnic Diversity"                = list(raw = "entropy_c",                z = "z_entropy"),
  "FNE x Ethnic Diversity"          = list(raw = "se2_c:entropy_c",          z = "z_se2:z_entropy"),
  "Ethnic Diversity x CCS"          = list(raw = "entropy_c:ccs_c",          z = "z_entropy:z_ccs"),
  "FNE x CCS"                       = list(raw = "se2_c:ccs_c",              z = "z_se2:z_ccs"),
  "FNE x Ethnic Diversity x CCS"    = list(raw = "se2_c:entropy_c:ccs_c",    z = "z_se2:z_entropy:z_ccs"),
  "Population Density"              = list(raw = "density_100",              z = "z_density"),
  "Median Income (log)"             = list(raw = "log_med_income_avg",       z = "z_income"),
  "Unemployment Rate"               = list(raw = "unemployment_rate_avg",    z = "z_unemp"),
  "Working-Age Population"          = list(raw = "age_25_64_avg",            z = "z_age"),
  "Employment Rate"                 = list(raw = "employment_rate_avg",      z = "z_emprate"),
  "Burt's Constraint (VC Network)"  = list(raw = "burt_constraint",          z = "z_burtc"),
  "VC Deal Volume, per capita"      = list(raw = "in_deals_pc",            z = "z_indeals"),
  "Specialisation Index"            = list(raw = "specialisation_index",     z = "z_specialisation")
)

cat("\n\n==========================================================================\n")
cat("TABLE 4.6. Effects of Financial Network Embeddedness on Productive\n")
cat("Entrepreneurship (primary lagged specification).\n")
cat("==========================================================================\n")
print_results_table(
  fin_raw, fin_std,
  model_labels = c("(1)", "(2)", "(3)", "(4)", "(5)", "(6)", "(7)", "(8)"),
  var_specs = fin_var_specs
)
```

II.A.2 Cross-Effect Checks

Paper3_Script2_Cross_Effects_TablesC1_C2.R

Tables C1 and C2: Financial Network Embeddedness → Homeostasis and Migration Concentration → Productive Entrepreneurship, using the same eight-model sequence.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 2: CROSS-EFFECT CHECKS (TABLES C1 AND C2)
#
# Estimates the two remaining explanatory-variable / outcome combinations
# with the same eight-model sequence as Tables 4.5 and 4.6:
#   Table C1: Financial Network Embeddedness -> Homeostasis
#   Table C2: Migration Concentration        -> Productive Entrepreneurship
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session. The script adds the
# cross-pairing columns (se2_c to homeostasis_analysis and se1_c to
# finance_analysis) that Scripts 3 and 4 also use.
# ============================================================================
required_objects <- c("homeostasis_analysis", "finance_analysis",
                      "controls_formula_homeo", "controls_formula_fin",
                      "se1_avg", "se2_avg", "print_results_table")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script1_Primary_Specification.R first, in the same
R session.",
               paste(missing_objects, collapse = ", ")))
}
# ================================================
# Add the cross-pairing explanatory variables (mean-centred)
# ================================================
library(dplyr)
if (!"se2_c" %in% names(homeostasis_analysis)) {
  se2_avg_dedup <- se2_avg %>%
    mutate(sa4 = as.character(sa4)) %>%
    group_by(sa4) %>%
    summarise(se2_val = mean(se2_avg, na.rm = TRUE), .groups = "drop")
  homeostasis_analysis <- homeostasis_analysis %>% mutate(sa4 = as.character(sa4))
  homeostasis_analysis <- merge(homeostasis_analysis, se2_avg_dedup, by = "sa4", all.x = TRUE)
  homeostasis_analysis$se2_val[is.na(homeostasis_analysis$se2_val)] <- 0
  homeostasis_analysis$se2_c <- homeostasis_analysis$se2_val - mean(homeostasis_analysis$se2_val, na.rm =
TRUE)
  cat("[SETUP] Added se2_c to homeostasis_analysis.\n")
} else {
  cat("[SETUP] se2_c already present in homeostasis_analysis.\n")
}
if (!"se1_c" %in% names(finance_analysis)) {
  se1_avg_dedup <- se1_avg %>%
    mutate(sa4 = as.character(sa4)) %>%
    group_by(sa4) %>%
    summarise(se1_val = mean(se1_avg, na.rm = TRUE), .groups = "drop")
  finance_analysis <- finance_analysis %>% mutate(sa4 = as.character(sa4))
  finance_analysis <- merge(finance_analysis, se1_avg_dedup, by = "sa4", all.x = TRUE)
  finance_analysis$se1_c <- finance_analysis$se1_val - mean(finance_analysis$se1_val, na.rm = TRUE)
  cat("[SETUP] Added se1_c to finance_analysis.\n")
} else {
  cat("[SETUP] se1_c already present in finance_analysis.\n")
}
# Standardised versions for the standardised-coefficient columns
if (!"z_se2" %in% names(homeostasis_analysis) || any(is.na(homeostasis_analysis$z_se2))) {
  homeostasis_analysis$z_se2 <- as.numeric(scale(homeostasis_analysis$se2_c))
}
if (!"z_se1" %in% names(finance_analysis) || any(is.na(finance_analysis$z_se1))) {
  finance_analysis$z_se1 <- as.numeric(scale(finance_analysis$se1_c))
}
# ============================================================================
# TABLE C1: HOMEOSTASIS ~ FINANCIAL NETWORK EMBEDDEDNESS x ETHNIC DIVERSITY x CCS
# ============================================================================
f_c1 <- as.formula(paste("homeostasis ~ se2_c +", controls_formula_homeo))
f_c2 <- as.formula(paste("homeostasis ~ ccs_c +", controls_formula_homeo))
f_c3 <- as.formula(paste("homeostasis ~ entropy_c +", controls_formula_homeo))
f_c4 <- as.formula(paste("homeostasis ~ se2_c + ccs_c + entropy_c +", controls_formula_homeo))
f_c5 <- as.formula(paste("homeostasis ~ se2_c * entropy_c + ccs_c +", controls_formula_homeo))
f_c6 <- as.formula(paste("homeostasis ~ entropy_c * ccs_c + se2_c +", controls_formula_homeo))
```

```
f_c7 <- as.formula(paste("homeostasis ~ se2_c * ccs_c + entropy_c +", controls_formula_homeo))
f_c8 <- as.formula(paste("homeostasis ~ se2_c * entropy_c * ccs_c +", controls_formula_homeo))
f_c1z <- as.formula(paste("z_homeostasis ~ z_se2 +", controls_formula_homeo_z))
f_c2z <- as.formula(paste("z_homeostasis ~ z_ccs +", controls_formula_homeo_z))
f_c3z <- as.formula(paste("z_homeostasis ~ z_entropy +", controls_formula_homeo_z))
f_c4z <- as.formula(paste("z_homeostasis ~ z_se2 + z_ccs + z_entropy +", controls_formula_homeo_z))
f_c5z <- as.formula(paste("z_homeostasis ~ z_se2 * z_entropy + z_ccs +", controls_formula_homeo_z))
f_c6z <- as.formula(paste("z_homeostasis ~ z_entropy * z_ccs + z_se2 +", controls_formula_homeo_z))
f_c7z <- as.formula(paste("z_homeostasis ~ z_se2 * z_ccs + z_entropy +", controls_formula_homeo_z))
f_c8z <- as.formula(paste("z_homeostasis ~ z_se2 * z_entropy * z_ccs +", controls_formula_homeo_z))
homeoC_raw <- lapply(list(f_c1, f_c2, f_c3, f_c4, f_c5, f_c6, f_c7, f_c8), lm, data = homeostasis_analysis)
homeoC_std <- lapply(list(f_c1z, f_c2z, f_c3z, f_c4z, f_c5z, f_c6z, f_c7z, f_c8z), lm, data =
homeostasis_analysis)
homeoC_var_specs <- list(
  "Financial Network Embeddedness"  = list(raw = "se2_c",                    z = "z_se2"),
  "Creative-Class Share"            = list(raw = "ccs_c",                    z = "z_ccs"),
  "Ethnic Diversity"                = list(raw = "entropy_c",                z = "z_entropy"),
  "FNE x Ethnic Diversity"          = list(raw = "se2_c:entropy_c",          z = "z_se2:z_entropy"),
  "Ethnic Diversity x CCS"          = list(raw = "entropy_c:ccs_c",          z = "z_entropy:z_ccs"),
  "FNE x CCS"                       = list(raw = "se2_c:ccs_c",              z = "z_se2:z_ccs"),
  "FNE x Ethnic Diversity x CCS"    = list(raw = "se2_c:entropy_c:ccs_c",   z = "z_se2:z_entropy:z_ccs"),
  "Population Density"              = list(raw = "density_100",         z = "z_density"),
  "Median Income (log)"             = list(raw = "log_med_income_avg",       z = "z_income"),
  "Unemployment Rate"               = list(raw = "unemployment_rate_avg",    z = "z_unemp"),
  "Working-Age Population"          = list(raw = "age_25_64_avg",            z = "z_age"),
  "Employment Rate"                 = list(raw = "employment_rate_avg",      z = "z_emprate"),
  "Specialisation Index"            = list(raw = "specialisation_index",     z = "z_specialisation")
)
cat("\n\n================================================================================\n")
cat("TABLE C1. Effects of Financial Network Embeddedness on Entrepreneurial\n")
cat("Ecosystem Homeostasis (cross-effect check).\n")
cat("================================================================================\n")
print_results_table(
  homeoC_raw, homeoC_std,
  model_labels = c("(1)", "(2)", "(3)", "(4)", "(5)", "(6)", "(7)", "(8)"),
  var_specs = homeoC_var_specs
)
# ================================================================================
# TABLE C2: PRODUCTIVE ENTREPRENEURSHIP ~ MIGRATION CONCENTRATION x ETHNIC
# DIVERSITY x CCS
# ================================================================================
f_d1 <- as.formula(paste("new_firms_percapita ~ se1_c +", controls_formula_fin))
f_d2 <- as.formula(paste("new_firms_percapita ~ ccs_c +", controls_formula_fin))
f_d3 <- as.formula(paste("new_firms_percapita ~ entropy_c +", controls_formula_fin))
f_d4 <- as.formula(paste("new_firms_percapita ~ se1_c + ccs_c + entropy_c +", controls_formula_fin))
f_d5 <- as.formula(paste("new_firms_percapita ~ se1_c * entropy_c + ccs_c +", controls_formula_fin))
f_d6 <- as.formula(paste("new_firms_percapita ~ entropy_c * ccs_c + se1_c +", controls_formula_fin))
f_d7 <- as.formula(paste("new_firms_percapita ~ se1_c * ccs_c + entropy_c +", controls_formula_fin))
f_d8 <- as.formula(paste("new_firms_percapita ~ se1_c * entropy_c * ccs_c +", controls_formula_fin))
f_d1z <- as.formula(paste("z_new_firms ~ z_se1 +", controls_formula_fin_z))
f_d2z <- as.formula(paste("z_new_firms ~ z_ccs +", controls_formula_fin_z))
f_d3z <- as.formula(paste("z_new_firms ~ z_entropy +", controls_formula_fin_z))
f_d4z <- as.formula(paste("z_new_firms ~ z_se1 + z_ccs + z_entropy +", controls_formula_fin_z))
f_d5z <- as.formula(paste("z_new_firms ~ z_se1 * z_entropy + z_ccs +", controls_formula_fin_z))
f_d6z <- as.formula(paste("z_new_firms ~ z_entropy * z_ccs + z_se1 +", controls_formula_fin_z))
f_d7z <- as.formula(paste("z_new_firms ~ z_se1 * z_ccs + z_entropy +", controls_formula_fin_z))
f_d8z <- as.formula(paste("z_new_firms ~ z_se1 * z_entropy * z_ccs +", controls_formula_fin_z))
finD_raw <- lapply(list(f_d1, f_d2, f_d3, f_d4, f_d5, f_d6, f_d7, f_d8), lm, data = finance_analysis)
finD_std <- lapply(list(f_d1z, f_d2z, f_d3z, f_d4z, f_d5z, f_d6z, f_d7z, f_d8z), lm, data = finance_analysis)
finD_var_specs <- list(
  "Migration Concentration"         = list(raw = "se1_c",                    z = "z_se1"),
  "Creative-Class Share"            = list(raw = "ccs_c",                    z = "z_ccs"),
  "Ethnic Diversity"                = list(raw = "entropy_c",                z = "z_entropy"),
  "MC x Ethnic Diversity"           = list(raw = "se1_c:entropy_c",          z = "z_se1:z_entropy"),
  "Ethnic Diversity x CCS"          = list(raw = "entropy_c:ccs_c",          z = "z_entropy:z_ccs"),
  "MC x CCS"                        = list(raw = "se1_c:ccs_c",              z = "z_se1:z_ccs"),
  "MC x Ethnic Diversity x CCS"     = list(raw = "se1_c:entropy_c:ccs_c",   z = "z_se1:z_entropy:z_ccs"),
  "Population Density"              = list(raw = "density_100",         z = "z_density"),
  "Median Income (log)"             = list(raw = "log_med_income_avg",       z = "z_income"),
  "Unemployment Rate"               = list(raw = "unemployment_rate_avg",    z = "z_unemp"),
  "Working-Age Population"          = list(raw = "age_25_64_avg",            z = "z_age"),
  "Employment Rate"                 = list(raw = "employment_rate_avg",      z = "z_emprate"),
  "Specialisation Index"            = list(raw = "specialisation_index",     z = "z_specialisation"),
  "Burt's Constraint (VC Network)"  = list(raw = "burt_constraint",  z = "z_burtc"),
  "VC Deal Volume, per capita"      = list(raw = "in_deals_pc",      z = "z_indeals")
```

```
)
cat("\n\n================================================================================\n")
cat("TABLE C2. Effects of Migration Concentration on Productive\n")
cat("Entrepreneurship (cross-effect check).\n")
cat("================================================================================\n")
print_results_table(
  finD_raw, finD_std,
  model_labels = c("(1)", "(2)", "(3)", "(4)", "(5)", "(6)", "(7)", "(8)"),
  var_specs = finD_var_specs
)
```

II.A.3 Predicted Margins

Paper3_Script3_Predicted_Margins_Figures4.2_4.3.R

Figures 4.2 and 4.3: predicted margins of each three-way interaction for the four region types (HE-HC, HE-LC, LE-HC, LE-LC), with 95% confidence bands.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 3: PREDICTED MARGINS FOR THE THREE-WAY
# INTERACTIONS (FIGURES 4.2 AND 4.3)
#
# For each three-way interaction model, predicts the outcome across the
# range of the focal explanatory variable at low (mean - 1 SD) and high
# (mean + 1 SD) values of the two moderators, giving four region types:
#   HH = high Ethnic Diversity, high Creative-Class Share  (HE-HC)
#   HL = high Ethnic Diversity, low Creative-Class Share   (HE-LC)
#   LH = low Ethnic Diversity, high Creative-Class Share   (LE-HC)
#   LL = low Ethnic Diversity, low Creative-Class Share    (LE-LC)
# Moderator values are bounded by the observed range.
#   Model A: Migration Concentration -> Homeostasis (Figure 4.2)
#   Model B: Financial Network Embeddedness -> Productive Entrepreneurship
#            (Figure 4.3)
#   Models C and D: the cross-effect models of Tables C1 and C2.
# Each model also reports the HH versus LL slope-difference test.
#
# Run after Paper3_Script1_Primary_Specification.R and Paper3_Script2_Cross_Effects_TablesC1_C2.R,
# in the same R session.
# ============================================================================
library(dplyr)
library(ggplot2)
library(marginaleffects)

options(marginaleffects_safe = FALSE)

required_objects <- c("homeostasis_analysis", "finance_analysis", "controls_formula_homeo",
"controls_formula_fin")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script1_Primary_Specification.R first.",
               paste(missing_objects, collapse = ", ")))
}
if (!"se2_c" %in% names(homeostasis_analysis) || !"se1_c" %in% names(finance_analysis)) {
  stop("se2_c not found in homeostasis_analysis, or se1_c not found in finance_analysis. Run
Paper3_Script2_Cross_Effects_TablesC1_C2.R first.")
}

# ================================================
# HELPER: Low/High cutoffs (mean +/- 1 SD, clipped to observed range)
# ================================================
get_loval <- function(x) { x <- x[!is.na(x)]; max(min(x), mean(x) - sd(x)) }
get_hival <- function(x) { x <- x[!is.na(x)]; min(max(x), mean(x) + sd(x)) }

# ================================================
# GENERIC PLOT + TEST BUILDER
# ================================================
build_marginal_plots <- function(model, data, focal_var, dv_label, focal_label, dv_var_name, tag) {

  entropy_lo <- get_loval(data$entropy_c)
  entropy_hi <- get_hival(data$entropy_c)
  cc_lo      <- get_loval(data$ccs_c)
  cc_hi      <- get_hival(data$ccs_c)

  focal_range <- quantile(data[[focal_var]], probs = c(0.05, 0.95), na.rm = TRUE)
  focal_seq   <- seq(focal_range[1], focal_range[2], length.out = 100)

  grid_args <- setNames(list(model, focal_seq, c(entropy_lo, entropy_hi), c(cc_lo, cc_hi)),
                        c("model", focal_var, "entropy_c", "ccs_c"))
  grid <- do.call(datagrid, c(grid_args, grid_type = "counterfactual"))

  pred <- predictions(model, newdata = grid, vcov = vcov(model)) %>% as.data.frame()

  pred <- pred %>%
    mutate(moderator_type = case_when(
      near(entropy_c, entropy_lo) & near(ccs_c, cc_lo) ~ "LL (low diversity, low CCS)",
```

```
      near(entropy_c, entropy_lo) & near(ccs_c, cc_hi) ~ "LH",
      near(entropy_c, entropy_hi) & near(ccs_c, cc_lo) ~ "HL",
      near(entropy_c, entropy_hi) & near(ccs_c, cc_hi) ~ "HH (high diversity, high CCS)"
    ))

  # --- 4-line plot ---
  plot_all4 <- ggplot(pred, aes(x = .data[[focal_var]], y = estimate, color = moderator_type, fill =
      moderator_type)) +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.15, color = NA) +
    geom_line(linewidth = 1) +
    labs(title = sprintf("%s: Predicted %s by %s,\nmoderated by Ethnic Diversity x Creative-Class Share",
tag, dv_label, focal_label),
         subtitle = "All four moderator combinations (mean +/- 1 SD)",
         x = focal_label, y = sprintf("Predicted %s", dv_label), color = "Region type", fill = "Region type")
+
    theme_minimal(base_size = 12) + theme(legend.position = "bottom")

  # --- 2-line plot (extremes only) ---
  pred_extremes <- pred %>% filter(moderator_type %in% c("LL (low diversity, low CCS)", "HH (high diversity,
high CCS)"))
  plot_extremes <- ggplot(pred_extremes, aes(x = .data[[focal_var]], y = estimate, color = moderator_type,
fill = moderator_type)) +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.15, color = NA) +
    geom_line(linewidth = 1.2) +
    scale_color_manual(values = c("LL (low diversity, low CCS)" = "#2166AC", "HH (high diversity, high CCS)"
= "#B2182B")) +
    scale_fill_manual(values  = c("LL (low diversity, low CCS)" = "#2166AC", "HH (high diversity, high CCS)"
= "#B2182B")) +
    labs(title = sprintf("%s: High-high (HH) versus low-low (LL) region type", tag),
         subtitle = sprintf("Effect of %s on %s", focal_label, dv_label),
         x = focal_label, y = sprintf("Predicted %s", dv_label), color = NULL, fill = NULL) +
    theme_minimal(base_size = 13) + theme(legend.position = "bottom")

  print(plot_extremes)
  print(plot_all4)

  # --- Significance test: LL slope vs HH slope ---
  slope_grid_args <- setNames(list(model, c(entropy_lo, entropy_hi), c(cc_lo, cc_hi)),
                              c("model", "entropy_c", "ccs_c"))
  slopes <- avg_slopes(model, variables = focal_var,
                       newdata = do.call(datagrid, c(slope_grid_args, grid_type = "counterfactual")),
                       by = c("entropy_c", "ccs_c"), vcov = vcov(model))
  cat(sprintf("\n[%s] Slopes by region type:\n", tag)); print(slopes)

  ll_row <- which(near(slopes$entropy_c, entropy_lo) & near(slopes$ccs_c, cc_lo))
  hh_row <- which(near(slopes$entropy_c, entropy_hi) & near(slopes$ccs_c, cc_hi))
  if (length(ll_row) == 1 && length(hh_row) == 1) {
    test_result <- hypotheses(slopes, hypothesis = sprintf("b%d - b%d = 0", hh_row, ll_row))
    cat(sprintf("[%s] HH vs LL slope difference test:\n", tag)); print(test_result)
  } else {
    cat(sprintf("[%s] WARNING: could not uniquely identify LL/HH rows (LL=%d, HH=%d found) - inspect slopes
above manually.\n", tag, length(ll_row), length(hh_row)))
  }

  list(plot_all4 = plot_all4, plot_extremes = plot_extremes)
}

# ================================================
# A) Migration Concentration x Ethnic Diversity x CCS -> Homeostasis
# ================================================
model_A <- lm(as.formula(paste("homeostasis ~ se1_c * entropy_c * ccs_c +", controls_formula_homeo)), data =
homeostasis_analysis)
res_A <- build_marginal_plots(model_A, homeostasis_analysis, "se1_c", "Homeostasis", "Migration
Concentration", "homeostasis", "A")
ggsave("A_SE1_Homeostasis_extremes.png", res_A$plot_extremes, width = 8, height = 6, dpi = 300)
ggsave("A_SE1_Homeostasis_all4.png", res_A$plot_all4, width = 9, height = 6, dpi = 300)

# ================================================
# B) Financial Network Embeddedness x Ethnic Diversity x CCS -> Productive
#    Entrepreneurship
# ================================================
model_B <- lm(as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +",
    controls_formula_fin)), data = finance_analysis)
res_B <- build_marginal_plots(model_B, finance_analysis, "se2_c", "Productive Entrepreneurship", "Financial
Network Embeddedness", "new_firms_percapita", "B")
```

```
ggsave("B_SE2_NewFirms_extremes.png", res_B$plot_extremes, width = 8, height = 6, dpi = 300)
ggsave("B_SE2_NewFirms_all4.png", res_B$plot_all4, width = 9, height = 6, dpi = 300)

# ================================================
# C) Financial Network Embeddedness x Ethnic Diversity x CCS -> Homeostasis
#    (cross-effect, Table C1)
# ================================================
model_C <- lm(as.formula(paste("homeostasis ~ se2_c * entropy_c * ccs_c +", controls_formula_homeo)), data =
homeostasis_analysis)
res_C <- build_marginal_plots(model_C, homeostasis_analysis, "se2_c", "Homeostasis", "Financial Network
Embeddedness", "homeostasis", "C")
ggsave("C_SE2_Homeostasis_extremes.png", res_C$plot_extremes, width = 8, height = 6, dpi = 300)
ggsave("C_SE2_Homeostasis_all4.png", res_C$plot_all4, width = 9, height = 6, dpi = 300)

# ================================================
# D) Migration Concentration x Ethnic Diversity x CCS -> Productive
#    Entrepreneurship (cross-effect, Table C2)
# ================================================
model_D <- lm(as.formula(paste("new_firms_percapita ~ se1_c * entropy_c * ccs_c +",
    controls_formula_fin)), data = finance_analysis)
res_D <- build_marginal_plots(model_D, finance_analysis, "se1_c", "Productive Entrepreneurship", "Migration
Concentration", "new_firms_percapita", "D")
ggsave("D_SE1_NewFirms_extremes.png", res_D$plot_extremes, width = 8, height = 6, dpi = 300)
ggsave("D_SE1_NewFirms_all4.png", res_D$plot_all4, width = 9, height = 6, dpi = 300)

cat("\nAll eight plots saved (four models x two versions).\n")
```

II.A.4 Pairwise Comparisons of Average Marginal Effects

Paper3_Script4_Pairwise_Comparisons_Tables4.7_C9.R

Tables 4.7 and C9: slopes of the focal explanatory variable by region type and all six pairwise differences.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 4: PAIRWISE COMPARISONS OF AVERAGE MARGINAL
# EFFECTS (TABLES 4.7 AND C9)
#
# For each three-way interaction model (A-D, as in Script 3), computes the
# average marginal effect (slope) of the focal explanatory variable in each
# of the four region types (HH, HL, LH, LL; see Script 3) and tests all six
# pairwise differences between those slopes. Table C9 reports all six
# contrasts; Table 4.7 reports the two most informative contrasts per model.
#
# Run after Paper3_Script1_Primary_Specification.R and Paper3_Script2_Cross_Effects_TablesC1_C2.R,
# in the same R session.
# ============================================================================
library(marginaleffects)
library(dplyr)
options(marginaleffects_safe = FALSE)
required <- c("homeostasis_analysis", "finance_analysis", "controls_formula_homeo",
    "controls_formula_fin")
missing <- required[!sapply(required, exists)]
if (length(missing) > 0) stop(sprintf("Missing: %s. Run Paper3_Script1_Primary_Specification.R first, in the
same R session.", paste(missing, collapse=", ")))
if (!"se2_c" %in% names(homeostasis_analysis) || !"se1_c" %in% names(finance_analysis)) {
  stop("Cross-effect columns missing - run Paper3_Script2_Cross_Effects_TablesC1_C2.R first.")
}
get_loval <- function(x) { x <- x[!is.na(x)]; max(min(x), mean(x) - sd(x)) }
get_hival <- function(x) { x <- x[!is.na(x)]; min(max(x), mean(x) + sd(x)) }
pairwise_slopes <- function(model, data, focal_var, tag, dv_label, iv_label) {
  entropy_lo <- get_loval(data$entropy_c); entropy_hi <- get_hival(data$entropy_c)
  cc_lo <- get_loval(data$ccs_c);          cc_hi <- get_hival(data$ccs_c)
  grid_args <- setNames(list(model, c(entropy_lo, entropy_hi), c(cc_lo, cc_hi)),
                        c("model", "entropy_c", "ccs_c"))
  grid <- do.call(datagrid, c(grid_args, grid_type = "counterfactual"))
  slopes <- avg_slopes(model, variables = focal_var, newdata = grid,
                       by = c("entropy_c", "ccs_c"), vcov = vcov(model))
  # Group labels are kept in a separate vector so that the `slopes` object
  # retains the attributes that hypotheses() requires.
  group_labels <- case_when(
    near(slopes$entropy_c, entropy_lo) & near(slopes$ccs_c, cc_lo) ~ "LL",
    near(slopes$entropy_c, entropy_lo) & near(slopes$ccs_c, cc_hi) ~ "LH",
    near(slopes$entropy_c, entropy_hi) & near(slopes$ccs_c, cc_lo) ~ "HL",
    near(slopes$entropy_c, entropy_hi) & near(slopes$ccs_c, cc_hi) ~ "HH"
  )
  cat(sprintf("\n\n================================================================================\n"))
  cat(sprintf("MODEL %s: Effect of %s on %s, by regional profile\n", tag, iv_label, dv_label))
  cat(sprintf("================================================================================\n"))
  print_df <- slopes[, c("estimate", "std.error", "statistic", "p.value", "conf.low", "conf.high")]
  print_df$group <- group_labels
  print(print_df[, c("group", "estimate", "std.error", "statistic", "p.value", "conf.low", "conf.high")])
  row_idx <- setNames(seq_len(nrow(slopes)), group_labels)
  pairs <- list(c("HH","HL"), c("HH","LH"), c("HH","LL"), c("HL","LH"), c("HL","LL"), c("LH","LL"))
  cat(sprintf("\n--- Pairwise comparisons of slopes (Model %s) ---\n", tag))
  for (p in pairs) {
    i <- row_idx[p[1]]; j <- row_idx[p[2]]
    test <- hypotheses(slopes, hypothesis = sprintf("b%d - b%d = 0", i, j))
    cat(sprintf("%s vs %s:  diff = %.3f, SE = %.3f, p = %.3f %s\n",
                p[1], p[2], test$estimate, test$std.error, test$p.value,
                ifelse(test$p.value < 0.001, "***", ifelse(test$p.value < 0.01, "**",
                ifelse(test$p.value < 0.05, "*", ifelse(test$p.value < 0.1, "+", ""))))))
  }
  invisible(slopes)
}
# Model 8 of Tables 4.5, 4.6, C1 and C2
model_A <- lm(as.formula(paste("homeostasis ~ se1_c * entropy_c * ccs_c +", controls_formula_homeo)), data =
homeostasis_analysis)
model_B <- lm(as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +",
    controls_formula_fin)), data = finance_analysis)
model_C <- lm(as.formula(paste("homeostasis ~ se2_c * entropy_c * ccs_c +", controls_formula_homeo)), data =
homeostasis_analysis)
```

```
model_D <- lm(as.formula(paste("new_firms_percapita ~ se1_c * entropy_c * ccs_c +",
    controls_formula_fin)), data = finance_analysis)
pairwise_slopes(model_A, homeostasis_analysis, "se1_c", "A", "Homeostasis", "Migration Concentration")
pairwise_slopes(model_B, finance_analysis,     "se2_c", "B", "Productive Entrepreneurship", "Financial
Network Embeddedness")
pairwise_slopes(model_C, homeostasis_analysis, "se2_c", "C", "Homeostasis", "Financial Network Embeddedness")
pairwise_slopes(model_D, finance_analysis,     "se1_c", "D", "Productive Entrepreneurship", "Migration
Concentration")
```

II.A.5 Robustness and Diagnostic Checks

`Paper3_Script5_Robustness_Diagnostics_TableC7.R`

Table C7 and Section 4.4.5: alternative measures, negative binomial model, Cook's distance, collinearity check and moderator stability across Census waves.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 5: ROBUSTNESS AND DIAGNOSTIC CHECKS
# (TABLE C7 AND SECTION 4.4.5)
#
#   1. Reference estimates from the primary models (Tables 4.5 and 4.6)
#   2. Cross-effect main effects
#   3. Alternative measures of Financial Network Embeddedness (Burt's
#      constraint in the VC network; PageRank centrality)
#   4. Alternative measures for the Homeostasis models (occupation-based
#      (ANZSCO) unrelated variety; religious diversity in place of ethnic
#      diversity; Burt's constraint in the migration network)
#   5. Negative binomial specification (count of new firms)
#   6. Influence diagnostics (Cook's distance)
#   7. Collinearity check (median income omitted)
#   8. Moderator temporal stability (2016 versus 2021 Census wave)
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session.
# ============================================================================
library(lmtest); library(sandwich); library(car); library(readxl); library(igraph); library(MASS)

required_objects <- c("homeostasis_analysis", "finance_analysis", "homeo_raw", "fin_raw",
                      "controls_formula_homeo", "controls_formula_fin", "sa4_lookup_clean",
                      "se1_avg", "se2_avg", "edges_10yr", "bilateral")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script1_Primary_Specification.R first, in the same
R session.",
               paste(missing_objects, collapse = ", ")))
}


robust_summary <- function(model) coeftest(model, vcov = vcovHC(model, type = "HC1"))
clean_sa4 <- function(x) x %>% str_to_lower() %>% str_trim() %>% str_squish()
data_dir <- "data"

# ============================================================================
# PART 1: CORE MODEL REFERENCE VALUES
# ============================================================================
cat("################################################################################\n")
cat("# PART 1: CORE MODEL REFERENCE VALUES\n")
cat("################################################################################\n")
orig_se1_main <- robust_summary(homeo_raw[[4]])["se1_c", ]
orig_homeo_3way <- robust_summary(homeo_raw[[8]])["se1_c:entropy_c:ccs_c", ]
orig_se2_main <- robust_summary(fin_raw[[4]])["se2_c", ]
orig_fin_3way <- robust_summary(fin_raw[[8]])["se2_c:entropy_c:ccs_c", ]
cat(sprintf("MC -> Homeostasis (model 4):         est=%.4f, p=%.4f\n", orig_se1_main["Estimate"],
    orig_se1_main["Pr(>|t|)"]))
cat(sprintf("MC x ED x CCS -> Homeostasis (m8):   est=%.4f, p=%.4f\n", orig_homeo_3way["Estimate"],
    orig_homeo_3way["Pr(>|t|)"]))
cat(sprintf("FNE -> Productive Ent. (model 4):    est=%.4f, p=%.4f\n", orig_se2_main["Estimate"],
    orig_se2_main["Pr(>|t|)"]))
cat(sprintf("FNE x ED x CCS -> Productive Ent. (m8):est=%.4f, p=%.4f\n\n", orig_fin_3way["Estimate"],
    orig_fin_3way["Pr(>|t|)"]))

# ============================================================================
# PART 2: CROSS-EFFECT CHECKS
# ============================================================================
cat("################################################################################\n")
cat("# PART 2: CROSS-EFFECT CHECKS\n")
cat("################################################################################\n")
fin_cross <- finance_analysis %>% left_join(se1_avg, by = "sa4") %>%
  mutate(se1_c_cross = se1_avg - mean(se1_avg, na.rm = TRUE))
f_cross1 <- as.formula(paste("new_firms_percapita ~ se1_c_cross +", controls_formula_fin))
m_cross1 <- lm(f_cross1, data = fin_cross)
cross1_res <- robust_summary(m_cross1)["se1_c_cross", ]
cat(sprintf("MC -> Productive Entrepreneurship: est=%.4f, p=%.4f (N=%d)\n", cross1_res["Estimate"],
    cross1_res["Pr(>|t|)"], nobs(m_cross1)))
```

```
homeo_cross <- homeostasis_analysis %>% left_join(se2_avg, by = "sa4") %>%
  mutate(se2_avg = replace_na(se2_avg, 0), se2_c_cross = se2_avg - mean(se2_avg, na.rm = TRUE))
f_cross2 <- as.formula(paste("homeostasis ~ se2_c_cross +", controls_formula_homeo))
m_cross2 <- lm(f_cross2, data = homeo_cross)
cross2_res <- robust_summary(m_cross2)["se2_c_cross", ]
cat(sprintf("FNE -> Homeostasis:                est=%.4f, p=%.4f (N=%d)\n\n", cross2_res["Estimate"],
cross2_res["Pr(>|t|)"], nobs(m_cross2)))

# ============================================================================
# PART 3: ALTERNATIVE MEASURES OF FINANCIAL NETWORK EMBEDDEDNESS
# ============================================================================
cat("############################################################################\n")
cat("# PART 3: ALTERNATIVE MEASURES OF FINANCIAL NETWORK EMBEDDEDNESS\n")
cat("############################################################################\n")

# -- Burt's Constraint (VC) --
burt_vc_graph <- edges_10yr %>% count(investor_sa4, investee_sa4, name = "w_ij") %>%
    graph_from_data_frame(directed = TRUE)
burt_vc <- tryCatch({
  co <- constraint(burt_vc_graph, weights = E(burt_vc_graph)$w_ij)
  data.frame(sa4 = as.character(V(burt_vc_graph)$name), burtc_alt = as.numeric(co))
}, error = function(e) data.frame(sa4 = character(0), burtc_alt = numeric(0))) %>% filter(!is.na(burtc_alt))

fin_burt <- finance_analysis %>% left_join(burt_vc, by = "sa4") %>%
  mutate(burtc_alt = replace_na(burtc_alt, 0),
         burtc_alt_c = burtc_alt - mean(burtc_alt, na.rm = TRUE))
f_burt_alt <- as.formula(paste("new_firms_percapita ~ burtc_alt_c * entropy_c * ccs_c + se2_c +",
    controls_formula_fin))
m_burt_alt <- lm(f_burt_alt, data = fin_burt)
burt_alt_res <- robust_summary(m_burt_alt)["burtc_alt_c:entropy_c:ccs_c", ]
cat(sprintf("Burt's Constraint (VC) swap, 3-way: est=%.4f, p=%.4f (N=%d)\n", burt_alt_res["Estimate"],
    burt_alt_res["Pr(>|t|)"], nobs(m_burt_alt)))

# -- PageRank --
pagerank_scores <- page_rank(burt_vc_graph, directed = TRUE, weights = E(burt_vc_graph)$w_ij)$vector
pagerank_df <- data.frame(sa4 = names(pagerank_scores), pagerank = as.numeric(pagerank_scores))
fin_pr <- finance_analysis %>% left_join(pagerank_df, by = "sa4") %>%
  mutate(pagerank = replace_na(pagerank, 0), pagerank_c = pagerank - mean(pagerank, na.rm = TRUE))
f_pr_alt <- as.formula(paste("new_firms_percapita ~ pagerank_c * entropy_c * ccs_c + se2_c +",
    controls_formula_fin))
m_pr_alt <- lm(f_pr_alt, data = fin_pr)
pr_alt_res <- robust_summary(m_pr_alt)["pagerank_c:entropy_c:ccs_c", ]
cat(sprintf("PageRank swap, 3-way:              est=%.6f, p=%.4f (N=%d)\n\n", pr_alt_res["Estimate"],
    pr_alt_res["Pr(>|t|)"], nobs(m_pr_alt)))

# ============================================================================
# PART 4: ALTERNATIVE MEASURES, HOMEOSTASIS MODELS
# ============================================================================
cat("############################################################################\n")
cat("# PART 4: ALTERNATIVE MEASURES, HOMEOSTASIS MODELS\n")
cat("############################################################################\n")

# -- ANZSCO variety swap --
f_anzsco_uv <- file.path(data_dir, "ANZSCOUV.csv")
anzsco_raw <- read_csv(f_anzsco_uv, show_col_types = FALSE) %>% clean_names() %>%
  transmute(sa4_name = clean_sa4(sa4_region), y2014 = x2014_anzsco, y2015 = x2015, y2016 = x2016,
            y2017 = x2017, y2018 = x2018, y2019 = x2019, y2020 = x2020) %>%
  mutate(across(starts_with("y2"), ~ ifelse(.x < 1, NA_real_, .x)))
anzsco_uv_avg <- anzsco_raw %>% rowwise() %>%
  mutate(anzsco_uv_raw = mean(c(y2014, y2015, y2016, y2017, y2018, y2019, y2020), na.rm = TRUE)) %>%
  ungroup() %>% left_join(sa4_lookup_clean, by = "sa4_name") %>% transmute(sa4 = sa4_code, anzsco_uv_raw)
homeo_anzsco <- homeostasis_analysis %>% left_join(anzsco_uv_avg, by = "sa4") %>%
  mutate(anzsco_norm = (anzsco_uv_raw - min(anzsco_uv_raw, na.rm=TRUE)) / (max(anzsco_uv_raw, na.rm=TRUE) -
min(anzsco_uv_raw, na.rm=TRUE)),
         anzsco_uv_nl = anzsco_norm - anzsco_norm^2, ed_plus_kibs = homeostasis - uv_nl,
         homeostasis_alt1 = ed_plus_kibs + anzsco_uv_nl)
f_anzsco_alt <- as.formula(paste("homeostasis_alt1 ~ se1_c + ccs_c + entropy_c +",
    controls_formula_homeo))
m_anzsco_alt <- lm(f_anzsco_alt, data = homeo_anzsco)
anzsco_res <- robust_summary(m_anzsco_alt)["se1_c", ]
cat(sprintf("ANZSCO variety swap, MC main:      est=%.4f, p=%.4f (N=%d)\n", anzsco_res["Estimate"],
    anzsco_res["Pr(>|t|)"], nobs(m_anzsco_alt)))

# -- Religious Diversity swap --
```

```
f_controls_xlsx <- file.path(data_dir, "CONTROLS.xlsx")
religion_raw <- read_excel(f_controls_xlsx, sheet = "relegious affiliation") %>% filter(TIME_PERIOD == 2016)
%>%
  mutate(category = str_trim(str_extract(`Data Item`, "(?<=:).*(?= \\()")), sa4 =
      as.character(ASGS_2021))
religion_wide <- religion_raw %>% filter(!str_detect(category, "Inadequately")) %>%
  dplyr::select(sa4, category, OBS_VALUE) %>% pivot_wider(names_from = category, values_from = OBS_VALUE) %>%
  mutate(across(-sa4, ~ replace_na(.x, 0)))
n_cats <- ncol(religion_wide) - 1
religious_diversity <- religion_wide %>% rowwise() %>%
  mutate(total = sum(c_across(-sa4), na.rm = TRUE),
         entropy_raw = { props <- c_across(-sa4) / total; props <- props[props > 0]; -sum(props * log(props))
},
         religious_diversity = (entropy_raw / log(n_cats)) * 100) %>% ungroup() %>% dplyr::select(sa4,
             religious_diversity)
homeo_reldiv <- homeostasis_analysis %>% left_join(religious_diversity, by = "sa4") %>%
  mutate(reldiv_c = religious_diversity - mean(religious_diversity, na.rm = TRUE))
f_reldiv_alt <- as.formula(paste("homeostasis ~ se1_c * reldiv_c * ccs_c +", controls_formula_homeo))
m_reldiv_alt <- lm(f_reldiv_alt, data = homeo_reldiv)
reldiv_res <- robust_summary(m_reldiv_alt)["se1_c:reldiv_c:ccs_c", ]
cat(sprintf("Religious Diversity swap, 3-way:    est=%.4f, p=%.4f (N=%d)\n", reldiv_res["Estimate"],
    reldiv_res["Pr(>|t|)"], nobs(m_reldiv_alt)))

# -- Burt's Constraint (Migration) swap --
bilateral_pooled_mig <- bilateral %>% group_by(i, j) %>% summarise(w_ij = sum(w_ij, na.rm = TRUE), .groups =
"drop") %>% filter(w_ij > 0)
g_migration <- graph_from_data_frame(bilateral_pooled_mig, directed = FALSE)
burt_constraint_mig <- tryCatch({
  co <- constraint(g_migration, weights = E(g_migration)$w_ij)
  data.frame(sa4 = as.character(V(g_migration)$name), burt_constraint_mig = as.numeric(co))
}, error = function(e) data.frame(sa4 = character(0), burt_constraint_mig = numeric(0))) %>%
filter(!is.na(burt_constraint_mig))
homeo_burtmig <- homeostasis_analysis %>% left_join(burt_constraint_mig, by = "sa4") %>%
  mutate(burtc_mig_c = burt_constraint_mig - mean(burt_constraint_mig, na.rm = TRUE))
f_burtmig_main <- as.formula(paste("homeostasis ~ burtc_mig_c + ccs_c + entropy_c +",
controls_formula_homeo))
m_burtmig_main <- lm(f_burtmig_main, data = homeo_burtmig)
burtmig_main_res <- robust_summary(m_burtmig_main)["burtc_mig_c", ]
f_burtmig_3way <- as.formula(paste("homeostasis ~ burtc_mig_c * entropy_c * ccs_c +",
controls_formula_homeo))
m_burtmig_3way <- lm(f_burtmig_3way, data = homeo_burtmig)
burtmig_3way_res <- robust_summary(m_burtmig_3way)["burtc_mig_c:entropy_c:ccs_c", ]
cat(sprintf("Burt's Constraint (Mig) swap, main: est=%.4f, p=%.4f (N=%d)\n",
    burtmig_main_res["Estimate"], burtmig_main_res["Pr(>|t|)"], nobs(m_burtmig_main)))
cat(sprintf("Burt's Constraint (Mig) swap, 3way: est=%.4f, p=%.4f (N=%d)\n\n",
    burtmig_3way_res["Estimate"], burtmig_3way_res["Pr(>|t|)"], nobs(m_burtmig_3way)))

# ============================================================================
# PART 5: NEGATIVE BINOMIAL CHECK (Productive Entrepreneurship)
# ============================================================================
cat("############################################################################\n")
cat("# PART 5: NEGATIVE BINOMIAL SPECIFICATION CHECK\n")
cat("############################################################################\n")
fin_nb <- finance_analysis %>% mutate(total_firms_count = round(new_firms_percapita *
population_avg_2021_2024 / 1000))
f_nb <- as.formula(paste("total_firms_count ~ se2_c * entropy_c * ccs_c +", controls_formula_fin))
m_nb <- tryCatch(glm.nb(f_nb, data = fin_nb), error = function(e) { cat("NB model failed:",
conditionMessage(e), "\n"); NULL })
if (!is.null(m_nb)) {
  nb_res <- summary(m_nb)$coefficients["se2_c:entropy_c:ccs_c", ]
  cat(sprintf("Negative Binomial, 3-way: est=%.4f, p=%.4f (N=%d)\n\n", nb_res["Estimate"],
nb_res["Pr(>|z|)"], nobs(m_nb)))
} else {
  cat("Negative binomial model did not converge.\n\n")
}

# ============================================================================
# PART 6: INFLUENCE DIAGNOSTICS (Cook's distance)
# ============================================================================
cat("############################################################################\n")
cat("# PART 6: INFLUENCE DIAGNOSTICS\n")
cat("############################################################################\n")
cooksd <- cooks.distance(fin_raw[[8]])
max_cooksd_idx <- which.max(cooksd)
cat(sprintf("Max Cook's distance: %.3f (row %d, SA4=%s)\n", cooksd[max_cooksd_idx], max_cooksd_idx,
```

```
    finance_analysis$sa4[max_cooksd_idx]))

fin_no_outlier <- finance_analysis[-max_cooksd_idx, ]
f8_fin <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +", controls_formula_fin))
m_no_outlier <- lm(f8_fin, data = fin_no_outlier)
no_outlier_res <- robust_summary(m_no_outlier)["se2_c:entropy_c:ccs_c", ]
cat(sprintf("Excluding highest-Cook's-D region: est=%.4f, p=%.4f (N=%d)\n\n",
    no_outlier_res["Estimate"], no_outlier_res["Pr(>|t|)"], nobs(m_no_outlier)))

# ============================================================================
# PART 7: COLLINEARITY CHECK (median income omitted)
# ============================================================================
cat("############################################################################\n")
cat("# PART 7: COLLINEARITY CHECK (median income omitted)\n")
cat("############################################################################\n")
controls_no_income_homeo <- gsub("log_med_income_avg \\+ ", "", controls_formula_homeo)
controls_no_income_fin <- gsub("log_med_income_avg \\+ ", "", controls_formula_fin)

f_se1_noincome <- as.formula(paste("homeostasis ~ se1_c + ccs_c + entropy_c +",
    controls_no_income_homeo))
m_se1_noincome <- lm(f_se1_noincome, data = homeostasis_analysis)
cat(sprintf("MC main effect, income omitted:    est=%.4f, p=%.4f\n", robust_summary(m_se1_noincome)
["se1_c","Estimate"], robust_summary(m_se1_noincome)["se1_c","Pr(>|t|)"]))

f_se2_noincome <- as.formula(paste("new_firms_percapita ~ se2_c + ccs_c + entropy_c +",
controls_no_income_fin))
m_se2_noincome <- lm(f_se2_noincome, data = finance_analysis)
cat(sprintf("FNE main effect, income omitted:   est=%.4f, p=%.4f\n", robust_summary(m_se2_noincome)
["se2_c","Estimate"], robust_summary(m_se2_noincome)["se2_c","Pr(>|t|)"]))

f_3way_noincome <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +",
controls_no_income_fin))
m_3way_noincome <- lm(f_3way_noincome, data = finance_analysis)
cat(sprintf("Productive Ent. 3-way, income omitted:est=%.4f, p=%.4f\n\n", robust_summary(m_3way_noincome)
["se2_c:entropy_c:ccs_c","Estimate"], robust_summary(m_3way_noincome)["se2_c:entropy_c:ccs_c","Pr(>|t|)"]))

# ============================================================================
# PART 8: MODERATOR TEMPORAL STABILITY (2016 vs 2021)
# ============================================================================
cat("############################################################################\n")
cat("# PART 8: MODERATOR TEMPORAL STABILITY (2016 vs 2021 Census wave)\n")
cat("############################################################################\n")
f_mv1_2021 <- file.path(data_dir, "MV1_Entropy ED2021.csv")
if (file.exists(f_mv1_2021)) {
  mv1_2021_check <- tryCatch({
    read_csv(f_mv1_2021, show_col_types = FALSE) %>%
      clean_names() %>% transmute(sa4_name = str_to_lower(str_trim(region)), entropy_2021 = e_std) %>%
      left_join(sa4_lookup_clean, by = "sa4_name") %>% transmute(sa4 = sa4_code, entropy_2021)
  }, error = function(e) NULL)
  if (!is.null(mv1_2021_check)) {
    compare_div <- homeostasis_analysis %>% dplyr::select(sa4, entropy) %>% left_join(mv1_2021_check, by =
"sa4")
    cat(sprintf("r(Ethnic Diversity 2016, 2021) = %.3f\n", cor(compare_div$entropy,
        compare_div$entropy_2021, use = "pairwise.complete.obs")))
  } else {
    cat("2021 moderator file could not be read.\n")
  }
} else {
  cat("2021 moderator file not found - check skipped.\n")
}

cat("\n############################################################################\n")
cat("# ALL CHECKS COMPLETE\n")
cat("############################################################################\n")
```

II.B Robustness Specification

This script is self-contained and is run in a new R session. It produces the overlapping-window robustness specification reported in Tables C3–C6.

II.B.1 Robustness (Overlapping-Window) Specification

`Paper3_Script6_Robustness_Specification_TablesC3_C6.R`

Tables C3–C6: full observation windows, 2021 Census moderators and cumulative 2014–2024 Productive Entrepreneurship. Run in a fresh R session.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 6: ROBUSTNESS (OVERLAPPING-WINDOW)
# SPECIFICATION (TABLES C3-C6)
#
# Self-contained: run in a fresh R session (not after Script 1, because it
# re-creates the same object names). Differences from the primary
# specification (Section 4.4.5 and the notes to Tables C3 and C4):
#   - Migration Concentration and Financial Network Embeddedness use the full
#     observation windows (migration 2017-2025; the latest ten years of VC
#     deals), with no <= 2020 restriction.
#   - Moderators (Ethnic Diversity, Creative-Class Share) use the 2021 Census.
#   - Productive Entrepreneurship is cumulative: firms founded 2014-2024 per
#     1,000 residents.
#   - Controls are averaged over all available years.
# Homeostasis is constructed exactly as in the primary specification (sum of
# three standardised components). Model structure, controls and HC1 robust
# standard errors are identical to Tables 4.5 and 4.6.
#
# Bachelor-degree share, household net worth, Burt's constraint in the
# migration network and large-firm share are loaded for the diagnostic VIF
# checks below but are not included in the reported models (Table C8).
#
# STRUCTURE:
#   PART 0   Libraries, paths, helper functions
#   PART 1   SA4 lookup and postcode-to-SA4 crosswalk
#   PART 2   Migration Concentration (full window)
#   PART 3   Financial Network Embeddedness (latest ten-year VC window)
#   PART 4   Homeostasis (standardised components)
#   PART 5   Productive Entrepreneurship (cumulative 2014-2024)
#   PART 6   Moderators (2021 Census) and population density
#   PART 7   Controls
#   PART 8   Analysis datasets, centring, standardisation and VIFs (Tables
#            C5 and C6)
#   PART 9   Table C3 - Homeostasis
#   PART 10  Table C4 - Productive Entrepreneurship
# ============================================================================
# ============================================================================
# PART 0. LIBRARIES, PATHS, HELPER FUNCTIONS
# ============================================================================
library(tidyverse)
library(readxl)
library(lubridate)
library(janitor)
library(igraph)
library(lmtest)
library(sandwich)
library(car)
# All input files are read from a single project data folder.
data_dir <- "data"
f_postal_sa4        <- file.path(data_dir, "Postal to SA4.xlsx")
f_overlap_poa       <- file.path(data_dir, "overlap_poa_details.csv")
f_sa4_lookup        <- file.path(data_dir, "SA4 codes and names .xlsx")
f_vc_deals          <- file.path(data_dir, "Final VC Deals.xlsx")
f_migration_17_21   <- file.path(data_dir, "Internal_migration_SA4_2017_2021.csv")
f_migration_22_25   <- file.path(data_dir, "Internal_migration_SA4_2022_2025.csv")
f_population        <- file.path(data_dir, "resident_population.csv")
f_business_churn    <- file.path(data_dir, "Business_Churn_SA4_202124.xlsx")
f_homeostasis_raw   <- file.path(data_dir, "Dependent Variable_Homeostasis.csv")
f_uv_anzsic         <- file.path(data_dir, "UV_ANZSIC_2021.xlsx")
f_mv1_2021          <- file.path(data_dir, "MV1_Entropy ED2021.csv")
```

```
f_mv2_2021          <- file.path(data_dir, "mv2_CC_2021.csv")
f_employed_pre2021  <- file.path(data_dir, "Employed_SA4_201425.xlsx")
f_employed_post2021 <- file.path(data_dir, "employed_SA4.csv")
f_income            <- file.path(data_dir, "md_hh_inc.csv")
f_age               <- file.path(data_dir, "demographic 25-64.csv")
f_unemployment      <- file.path(data_dir, "unemployment_finalrate.csv")
f_org_parts         <- file.path(data_dir, paste0("organizations_part_", 1:7, ".csv"))
f_all_sa4_data      <- file.path(data_dir, "All SA4s.data.csv")
f_dashboard         <- file.path(data_dir, "progressdashboard_2024data (2).csv")
clean_sa4 <- function(x) x %>% str_to_lower() %>% str_trim() %>% str_squish()
zscore    <- function(x) as.numeric(scale(x))
robust_summary <- function(model) coeftest(model, vcov = vcovHC(model, type = "HC1"))
stars_from_p <- function(p) {
  case_when(p < 0.001 ~ "***", p < 0.01 ~ "**", p < 0.05 ~ "*", p < 0.1 ~ "+", TRUE ~ "")
}
compute_se_directed <- function(edges) {
  if (nrow(edges) == 0) return(data.frame(sa4_code21 = character(0), SE = numeric(0)))
  edge_weights <- edges %>% count(investor_sa4, investee_sa4, name = "w_ij")
  g <- graph_from_data_frame(edge_weights, directed = TRUE)
  adj <- as_adjacency_matrix(g, attr = "w_ij", sparse = FALSE)
  nodes <- V(g)$name
  se_values <- numeric(length(nodes)); names(se_values) <- nodes
  for (i in nodes) {
    neigh <- which(adj[i, ] > 0 | adj[, i] > 0)
    k <- length(neigh)
    if (k <= 1) { se_values[i] <- 0; next }
    sub_adj <- adj[neigh, neigh]
    v_i <- sum(sub_adj > 0); max_edges <- k * (k - 1)
    se_values[i] <- ifelse(max_edges == 0, 0, v_i / max_edges)
  }
  data.frame(sa4_code21 = names(se_values), SE = se_values)
}
fmt_cell <- function(raw_model, std_model, term_raw, term_z) {
  rs <- robust_summary(raw_model)
  if (!term_raw %in% rownames(rs)) return(c("", "", ""))
  est   <- rs[term_raw, "Estimate"]
  se    <- rs[term_raw, "Std. Error"]
  p     <- rs[term_raw, "Pr(>|t|)"]
  stars <- stars_from_p(p)
  beta  <- if (!is.null(std_model) && !is.null(term_z) && term_z %in% names(coef(std_model))) {
    sprintf("%.3f", coef(std_model)[term_z])
  } else { "" }
  c(sprintf("%.3f%s", est, stars), sprintf("(%.3f)", se), sprintf("[%s]", beta))
}
print_results_table <- function(raw_models, std_models, model_labels, var_specs) {
  for (v in seq_along(var_specs)) {
    label    <- names(var_specs)[v]
    term_raw <- var_specs[[v]]$raw
    term_z   <- var_specs[[v]]$z
    cat(sprintf("%-32s", label))
    lines <- lapply(seq_along(raw_models), function(m) {
      if (is.null(raw_models[[m]])) return(c("", "", ""))
      fmt_cell(raw_models[[m]], std_models[[m]], term_raw, term_z)
    })
    cat(paste(sapply(lines, `[`, 1), collapse = " | "), "\n")
    cat(sprintf("%-32s", ""))
    cat(paste(sapply(lines, `[`, 2), collapse = " | "), "\n")
    cat(sprintf("%-32s", ""))
    cat(paste(sapply(lines, `[`, 3), collapse = " | "), "\n")
  }
  cat(strrep("-", 100), "\n")
  cat(sprintf("%-32s", "Observations"))
  cat(paste(sapply(raw_models, function(m) if (is.null(m)) "" else nobs(m)), collapse = " | "), "\n")
  cat(sprintf("%-32s", "Adjusted R-squared"))
  cat(paste(sapply(raw_models, function(m) if (is.null(m)) "" else sprintf("%.3f",
summary(m)$adj.r.squared)), collapse = " | "), "\n")
  cat(sprintf("%-32s", "AIC"))
  cat(paste(sapply(raw_models, function(m) if (is.null(m)) "" else sprintf("%.1f", AIC(m))), collapse = " |
"), "\n")
  cat("\nModel columns:", paste(model_labels, collapse = " | "), "\n")
  cat("Robust (HC1) SE in parentheses (); standardised coefficient in brackets [].\n")
  cat("*** p<0.001  ** p<0.01  * p<0.05  + p<0.1\n\n")
}
# ============================================================================
# PART 1. SA4 LOOKUP + POSTAL-TO-SA4 CROSSWALK
```

```
# ============================================================================
sa4_lookup_clean <- read_excel(f_sa4_lookup) %>%
  transmute(sa4_code = as.character(SA4_CODE_2021),
            sa4_name = str_to_lower(str_trim(SA4_NAME_2021))) %>%
  distinct() %>%
  filter(!is.na(sa4_code), !is.na(sa4_name), sa4_name != "na",
         !str_detect(sa4_code, "ZZZ"),
         !str_detect(sa4_name, "migratory|offshore|no usual address"))
postal_sa4_raw <- read_excel(f_postal_sa4) %>%
  clean_names() %>%
  dplyr::select(poa_code21, sa4_code21, sa4_name21) %>%
  mutate(poa_code21 = as.character(poa_code21)) %>%
  distinct()
overlap_override <- read_csv(f_overlap_poa, show_col_types = FALSE) %>%
  clean_names() %>%
  mutate(poa_code21 = str_pad(as.character(poa_code21), width = 4, pad = "0"))
postal_sa4 <- postal_sa4_raw %>%
  left_join(overlap_override, by = "poa_code21") %>%
  filter(is.na(sa4_region) | sa4_name21 == sa4_region) %>%
  dplyr::select(poa_code21, sa4_code21, sa4_name21) %>%
  distinct()
stopifnot(!any(duplicated(postal_sa4$poa_code21)))
all_sa4   <- postal_sa4 %>% distinct(sa4_code21, sa4_name21)
all_nodes <- unique(as.character(all_sa4$sa4_code21))
population_clean <- read_csv(f_population, show_col_types = FALSE) %>%
  mutate(sa4_name = clean_sa4(region_name), year = as.integer(year),
         population = `Estimated resident population`) %>%
  dplyr::select(sa4_name, year, population) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  dplyr::select(sa4 = sa4_code, year, population)
population_avg <- population_clean %>%
  group_by(sa4) %>% summarise(population_avg = mean(population, na.rm = TRUE), .groups = "drop")
cat(sprintf("[PART 1] postal_sa4: %d POAs, %d SA4s\n",
            n_distinct(postal_sa4$poa_code21), length(all_nodes)))
# ============================================================================
# PART 2. MIGRATION CONCENTRATION (se1; averaged over the full window)
# ============================================================================
migration_raw <- bind_rows(
  read_csv(f_migration_17_21, show_col_types = FALSE),
  read_csv(f_migration_22_25, show_col_types = FALSE)
) %>%
  transmute(year = as.numeric(TIME_PERIOD), i = as.character(REGION_DEP),
            j = as.character(REGION_ARR), weight = as.numeric(OBS_VALUE)) %>%
  filter(!is.na(weight), weight > 0, i != j) %>%
  filter(i %in% all_nodes, j %in% all_nodes)
flows <- migration_raw %>% group_by(year, i, j) %>%
  summarise(x_ij = sum(weight, na.rm = TRUE), .groups = "drop")
pair_key <- flows %>% mutate(lo = pmin(i, j), hi = pmax(i, j),
                             direction = ifelse(i == lo, "lo_to_hi", "hi_to_lo"))
bilateral <- pair_key %>%
  dplyr::select(year, lo, hi, direction, x_ij) %>%
  pivot_wider(names_from = direction, values_from = x_ij, values_fill = 0) %>%
  mutate(lo_to_hi = if ("lo_to_hi" %in% names(.)) lo_to_hi else 0,
         hi_to_lo = if ("hi_to_lo" %in% names(.)) hi_to_lo else 0) %>%
  transmute(year, i = lo, j = hi, w_ij = lo_to_hi + hi_to_lo) %>%
  filter(!is.na(i), !is.na(j), i != j)
se1_yearly <- bilateral %>%
  transmute(year, ego = i, partner = j, w_ij) %>%
  bind_rows(bilateral %>% transmute(year, ego = j, partner = i, w_ij)) %>%
  group_by(year, ego) %>%
  mutate(W_i = sum(w_ij, na.rm = TRUE), p_ij = ifelse(W_i > 0, w_ij / W_i, 0)) %>%
  summarise(SE = sum(p_ij^2, na.rm = TRUE), .groups = "drop")
se1_avg <- se1_yearly %>%
  group_by(ego) %>%
  summarise(se1_avg = mean(SE, na.rm = TRUE), .groups = "drop") %>%
  transmute(sa4 = as.character(ego), se1_avg)
cat(sprintf("[PART 2] se1_avg: %d regions\n", nrow(se1_avg)))
# ============================================================================
# PART 2b. BURT'S CONSTRAINT - MIGRATION NETWORK (diagnostic only)
# ============================================================================
bilateral_pooled_mig <- bilateral %>%
  group_by(i, j) %>%
  summarise(w_ij = sum(w_ij, na.rm = TRUE), .groups = "drop") %>%
  filter(w_ij > 0)
g_migration <- graph_from_data_frame(bilateral_pooled_mig, directed = FALSE)
```

```
burt_constraint_mig <- tryCatch({
  co <- constraint(g_migration, weights = E(g_migration)$w_ij)
  data.frame(sa4 = as.character(V(g_migration)$name), burt_constraint_mig = as.numeric(co))
}, error = function(e) {
  cat("[PART 2b] WARNING: constraint() failed -", conditionMessage(e), "\n")
  data.frame(sa4 = character(0), burt_constraint_mig = numeric(0))
}) %>%
  filter(!is.na(burt_constraint_mig))
cat(sprintf("[PART 2b] burt_constraint_mig: %d regions (NaN cases dropped, not zero-filled)\n",
            nrow(burt_constraint_mig)))
# ============================================================================
# PART 3. FINANCIAL NETWORK EMBEDDEDNESS (se2; latest ten years of VC deals)
# ============================================================================
vc_edges <- read_excel(f_vc_deals) %>%
  clean_names() %>%
  separate_rows(investee_company_zip_code, sep = "\\|") %>%
  separate_rows(firm_investors_zip_code, sep = "\\|") %>%
  mutate(investee_company_zip_code = trimws(as.character(investee_company_zip_code)),
         firm_investors_zip_code   = trimws(as.character(firm_investors_zip_code))) %>%
  left_join(postal_sa4, by = c("investee_company_zip_code" = "poa_code21")) %>%
  rename(investee_sa4 = sa4_code21) %>%
  left_join(postal_sa4, by = c("firm_investors_zip_code" = "poa_code21")) %>%
  rename(investor_sa4 = sa4_code21) %>%
  mutate(year = year(ymd(investment_date))) %>%
  filter(!is.na(investor_sa4), !is.na(investee_sa4),
         investor_sa4 != investee_sa4, !is.na(year))
edges_year <- vc_edges %>% dplyr::select(year, investor_sa4, investee_sa4) %>% distinct()
max_vc_year  <- max(edges_year$year)
edges_10yr   <- edges_year %>% filter(year >= max_vc_year - 9, year <= max_vc_year)
se2_avg <- compute_se_directed(edges_10yr) %>%
  transmute(sa4 = as.character(sa4_code21), se2_avg = SE)
cat(sprintf("[PART 3] se2_avg: %d regions with observed VC ties (zero-filled for the rest at merge)\n",
nrow(se2_avg)))
# ============================================================================
# PART 3b. BURT'S CONSTRAINT AND VC DEAL VOLUME (controls)
# ============================================================================
burt_constraint_raw <- edges_10yr %>%
  count(investor_sa4, investee_sa4, name = "w_ij") %>%
  graph_from_data_frame(directed = TRUE)
burt_constraint <- tryCatch({
  co <- constraint(burt_constraint_raw, weights = E(burt_constraint_raw)$w_ij)
  data.frame(sa4 = as.character(V(burt_constraint_raw)$name), burt_constraint = as.numeric(co))
}, error = function(e) {
  cat("[PART 3b] WARNING: constraint() failed -", conditionMessage(e), "\n")
  data.frame(sa4 = character(0), burt_constraint = numeric(0))
}) %>%
  filter(!is.na(burt_constraint))
in_deals_pc_final <- edges_10yr %>%
  group_by(investee_sa4) %>%
  summarise(in_deals = n(), .groups = "drop") %>%
  rename(sa4 = investee_sa4) %>%
  left_join(population_avg, by = "sa4") %>%
  mutate(in_deals_pc = (in_deals / population_avg) * 1000) %>%
  dplyr::select(sa4, in_deals_pc)
cat(sprintf("[PART 3b] burt_constraint: %d regions (NaN cases dropped, not zero-filled) | in_deals_pc: %d
regions\n",
            nrow(burt_constraint), nrow(in_deals_pc_final)))
# ============================================================================
# PART 4. DEPENDENT VARIABLE 1: ENTREPRENEURIAL ECOSYSTEM HOMEOSTASIS
# Sum of three standardised components, as in the primary specification.
# ============================================================================
churn_raw <- read_excel(f_business_churn) %>%
  mutate(region = Region, year = as.integer(TIME_PERIOD)) %>%
  filter(`Data Item` == "Total business entries (no.)") %>%
  transmute(sa4_name = clean_sa4(region), year, entries = as.numeric(OBS_VALUE))
real_entries <- churn_raw %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, year, entries) %>%
  filter(!is.na(sa4))
ed_cross <- real_entries %>%
  filter(year %in% 2021:2024) %>%
  group_by(sa4) %>%
  summarise(entries_cumulative = sum(entries, na.rm = TRUE), .groups = "drop") %>%
  left_join(population_clean %>% filter(year == 2024) %>%
              dplyr::select(sa4, population_2024 = population), by = "sa4") %>%
```

```
  mutate(ed_cross = (entries_cumulative / population_2024) * 1000)
uv_data_2digit <- read_excel(f_uv_anzsic) %>%
  clean_names() %>%
  rename(sa4 = sa4_ur, industry = x2_digit_level_indp_industry_of_employment, count = count) %>%
  dplyr::select(sa4, industry, count) %>%
  filter(!str_detect(industry, regex("nfd", ignore_case = TRUE))) %>%
  mutate(industry_2d = case_when(
    str_detect(industry, "Agriculture|Forestry|Fishing") ~ "A", str_detect(industry, "Mining") ~ "B",
    str_detect(industry, "Manufacturing") ~ "C", str_detect(industry, "Electricity|Gas|Water|Waste") ~ "D",
    str_detect(industry, "Construction") ~ "E", str_detect(industry, "Wholesale") ~ "F",
    str_detect(industry, "Retail") ~ "G", str_detect(industry, "Accommodation|Food") ~ "H",
    str_detect(industry, "Transport|Postal|Warehousing") ~ "I",
    str_detect(industry, "Information|Media|Telecommunications") ~ "J",
    str_detect(industry, "Financial|Insurance") ~ "K", str_detect(industry, "Rental|Hiring|Real Estate") ~
"L",
    str_detect(industry, "Professional|Scientific|Technical") ~ "M",
    str_detect(industry, "Administrative|Support") ~ "N", str_detect(industry, "Public Administration") ~
"O",
    str_detect(industry, "Education") ~ "P", str_detect(industry, "Health") ~ "Q",
    str_detect(industry, "Arts|Recreation") ~ "R", str_detect(industry, "Other Services") ~ "S",
    TRUE ~ NA_character_
  )) %>%
  filter(!is.na(industry_2d))
uv_cross <- uv_data_2digit %>%
  group_by(sa4, industry_2d) %>% summarise(count = sum(count), .groups = "drop") %>%
  group_by(sa4) %>% mutate(total_emp = sum(count), p_g = count / total_emp) %>%
  summarise(UV = sum(p_g * log2(1 / pmax(p_g, 1e-12))), .groups = "drop") %>%
  mutate(sa4_name = str_to_lower(str_trim(sa4))) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, uv = UV) %>%
  filter(!is.na(sa4)) %>%
  mutate(uv_norm = (uv - min(uv, na.rm = TRUE)) / (max(uv, na.rm = TRUE) - min(uv, na.rm = TRUE)),
         uv_nl = uv_norm - uv_norm^2)
raw_homeo <- read_csv(f_homeostasis_raw, show_col_types = FALSE) %>%
  clean_names() %>% rename(sa4 = sa4_region) %>%
  mutate(sa4_name = str_to_lower(str_trim(sa4))) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  mutate(sa4 = sa4_code) %>% dplyr::select(-sa4_name, -sa4_code)
kibs_2023_cross <- raw_homeo %>% dplyr::select(sa4, kibs_2023) %>% rename(kibs = kibs_2023) %>%
filter(!is.na(sa4))
homeostasis_cross <- ed_cross %>%
  dplyr::select(sa4, ed_cross) %>%
  left_join(uv_cross %>% dplyr::select(sa4, uv_nl), by = "sa4") %>%
  left_join(kibs_2023_cross, by = "sa4") %>%
  mutate(ed_log = log(1 + ed_cross), kibs_log = log(1 + kibs),
         z_ed_log = as.numeric(scale(ed_log)),
         z_uv_nl  = as.numeric(scale(uv_nl)),
         z_kibs_log = as.numeric(scale(kibs_log)),
         homeostasis = z_ed_log + z_uv_nl + z_kibs_log)
cat(sprintf("[PART 4] homeostasis_cross (standardised components): %d regions\n", nrow(homeostasis_cross)))
# ================================================================================
# PART 5. DEPENDENT VARIABLE 2: PRODUCTIVE ENTREPRENEURSHIP (CUMULATIVE
# 2014-2024, PER 1,000 RESIDENTS)
# ================================================================================
org_all <- map_dfr(f_org_parts, read.csv)
org_aus <- org_all %>%
  filter(country_code == "AUS") %>%
  mutate(founded_on = parse_date_time(founded_on, orders = c("ymd", "dmy", "mdy", "Y"), quiet = TRUE),
         year_founded = year(founded_on),
         postal_code_clean = str_extract(postal_code, "\\b\\d{4}\\b")) %>%
  filter(year_founded >= 2014, year_founded <= 2024)
org_sa4_valid <- org_aus %>%
  left_join(postal_sa4, by = c("postal_code_clean" = "poa_code21")) %>%
  filter(!is.na(sa4_code21)) %>%
  group_by(uuid, year_founded, sa4_code21, sa4_name21) %>%
  slice(1) %>% ungroup()
new_firms_total <- org_sa4_valid %>%
  distinct(uuid, sa4_code21) %>%
  count(sa4_code21, name = "total_firms") %>%
  transmute(sa4 = as.character(sa4_code21), total_firms)
new_firms_percapita <- new_firms_total %>%
  left_join(population_avg, by = "sa4") %>%
  mutate(new_firms_percapita = (total_firms / population_avg) * 1000)
cat(sprintf("[PART 5] new_firms_percapita: %d regions\n", nrow(new_firms_percapita)))
# ================================================================================
```

```
# PART 6. MODERATORS (2021 CENSUS) AND POPULATION DENSITY
# ============================================================================
abs_regional_raw <- read_csv(f_all_sa4_data, show_col_types = FALSE)
density_avg <- abs_regional_raw %>%
  filter(REGIONTYPE == "SA4", MEASURE == "ERP_21", !is.na(OBS_VALUE)) %>%
  mutate(sa4 = as.character(ASGS_2021)) %>%
  group_by(sa4) %>%
  summarise(population_density_avg = mean(OBS_VALUE, na.rm = TRUE), .groups = "drop")
bachelor_2021 <- abs_regional_raw %>%
  filter(REGIONTYPE == "SA4", MEASURE == "SCHOOL_5", !is.na(OBS_VALUE)) %>%
  mutate(sa4 = as.character(ASGS_2021)) %>%
  transmute(sa4, bachelor_pct = OBS_VALUE)
mv1_2021 <- read_csv(f_mv1_2021, show_col_types = FALSE) %>% clean_names() %>%
  transmute(sa4_name = str_to_lower(str_trim(region)), entropy = e_std)
entropy_2021 <- mv1_2021 %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, entropy)
ccs_2021 <- read_csv(f_mv2_2021, show_col_types = FALSE) %>%
  clean_names() %>%
  mutate(sa4_name = str_to_lower(str_trim(sa4_ur))) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, creative_class_share)
cat(sprintf("[PART 6] density_avg: %d | bachelor_2021: %d | entropy_2021: %d | ccs_2021: %d\n",
            nrow(density_avg), nrow(bachelor_2021), nrow(entropy_2021), nrow(ccs_2021)))
# ============================================================================
# PART 7. CONTROLS
# ============================================================================
employment_panel <- bind_rows(
  read_excel(f_employed_pre2021) %>%
    mutate(sa4_name = clean_sa4(`Region of usual residence (SA4)`), year = as.integer(year),
           employed = as.numeric(Employed) * 1000) %>%
    filter(year >= 2014, year <= 2020) %>% dplyr::select(sa4_name, year, employed),
  read_csv(f_employed_post2021, show_col_types = FALSE) %>%
    mutate(sa4_name = clean_sa4(region_name), year = as.integer(str_extract(date, "\\d{4}")),
           employed = as.numeric(value)) %>%
    filter(year >= 2021, year <= 2024) %>% dplyr::select(sa4_name, year, employed)
) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  dplyr::select(sa4 = sa4_code, year, employed)
income_clean <- read_csv(f_income, show_col_types = FALSE) %>%
  mutate(sa4_name = clean_sa4(sa4_name), year = as.integer(year)) %>%
  dplyr::select(sa4_name, year, log_med_income) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  dplyr::select(sa4 = sa4_code, year, log_med_income)
age_clean <- read_csv(f_age, show_col_types = FALSE) %>%
  mutate(sa4_name = clean_sa4(region_name), year = as.integer(date),
         age_25_64 = `Proportion of Estimated Resident Population that is 25-64 years old`) %>%
  dplyr::select(sa4_name, year, age_25_64) %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  dplyr::select(sa4 = sa4_code, year, age_25_64)
unemployment_clean <- read_csv(f_unemployment, show_col_types = FALSE) %>%
  mutate(sa4_code = str_extract(labour_market_region_sa4_asgs_2011, "^\\d+"), year = as.integer(year)) %>%
  dplyr::select(sa4 = sa4_code, year, unemployment_rate)
controls_all <- employment_panel %>%
  full_join(income_clean, by = c("sa4", "year")) %>%
  full_join(age_clean, by = c("sa4", "year")) %>%
  full_join(population_clean, by = c("sa4", "year")) %>%
  full_join(unemployment_clean, by = c("sa4", "year")) %>%
  mutate(employment_rate = employed / population)
controls_avg <- controls_all %>%
  group_by(sa4) %>%
  summarise(
    employment_rate_avg   = mean(employment_rate, na.rm = TRUE),
    log_med_income_avg    = mean(log_med_income, na.rm = TRUE),
    age_25_64_avg         = mean(age_25_64, na.rm = TRUE),
    unemployment_rate_avg = mean(unemployment_rate, na.rm = TRUE),
    .groups = "drop"
  )
dashboard_raw <- read_csv(f_dashboard, show_col_types = FALSE)
net_worth_avg <- dashboard_raw %>%
  filter(region_type == "sa4_code_2021", variable == "real_household_net_worth", !is.na(value)) %>%
  mutate(sa4 = as.character(region_id)) %>%
  group_by(sa4) %>%
  summarise(household_net_worth_avg = mean(value, na.rm = TRUE), .groups = "drop")
cat(sprintf("[PART 7] controls_avg: %d | net_worth_avg: %d\n", nrow(controls_avg), nrow(net_worth_avg)))
```

```
# ============================================================================
# PART 7b. SPECIALISATION INDEX
# ============================================================================
f_specialisation <- file.path(data_dir, "specialisation_index.csv")
specialisation_raw <- read_csv(f_specialisation, show_col_types = FALSE) %>%
  clean_names()
specialisation_clean <- specialisation_raw %>%
  transmute(sa4 = as.character(asgs_2021), specialisation_index = specialisation)
cat(sprintf("[PART 7b] specialisation_clean: %d regions\n", nrow(specialisation_clean)))
unmatched_spec <- specialisation_clean %>% filter(!sa4 %in% sa4_lookup_clean$sa4_code)
if (nrow(unmatched_spec) > 0) {
  cat(sprintf("[PART 7b] WARNING: %d specialisation_index SA4 codes do not match the pipeline's code
list:\n",
              nrow(unmatched_spec)))
  print(unmatched_spec)
} else {
  cat("[PART 7b] OK: all specialisation_index SA4 codes match.\n")
}
# ============================================================================
# PART 7c. LARGE-FIRM SHARE (diagnostic only)
# ============================================================================
f_firm_size <- file.path(data_dir, "firm_size_shares.csv")
firm_size_clean <- read_csv(f_firm_size, show_col_types = FALSE) %>%
  transmute(sa4 = as.character(sa4), share_large_20plus = share_large_20plus_avg)
cat(sprintf("[PART 7c] firm_size_clean: %d regions\n", nrow(firm_size_clean)))
unmatched_firmsize <- firm_size_clean %>% filter(!sa4 %in% sa4_lookup_clean$sa4_code)
if (nrow(unmatched_firmsize) > 0) {
  cat(sprintf("[PART 7c] WARNING: %d firm-size SA4 codes do not match the pipeline's code list:\n",
              nrow(unmatched_firmsize)))
  print(unmatched_firmsize)
} else {
  cat("[PART 7c] OK: all firm-size SA4 codes match.\n")
}
# ============================================================================
# PART 8. ASSEMBLE ANALYSIS DATASETS; MEAN-CENTRE AND STANDARDISE; VIFs
# ============================================================================
homeostasis_analysis <- homeostasis_cross %>%
  left_join(se1_avg, by = "sa4") %>%
  left_join(density_avg, by = "sa4") %>%
  left_join(bachelor_2021, by = "sa4") %>%
  left_join(entropy_2021, by = "sa4") %>%
  left_join(ccs_2021, by = "sa4") %>%
  left_join(controls_avg, by = "sa4") %>%
  left_join(net_worth_avg, by = "sa4") %>%
  left_join(burt_constraint_mig, by = "sa4") %>%
  mutate(burt_constraint_mig = replace_na(burt_constraint_mig, 0)) %>%
  left_join(specialisation_clean, by = "sa4") %>%
  left_join(firm_size_clean, by = "sa4")
finance_analysis <- new_firms_percapita %>%
  left_join(se2_avg, by = "sa4") %>%
  mutate(se2_avg = replace_na(se2_avg, 0)) %>%
  left_join(density_avg, by = "sa4") %>%
  left_join(bachelor_2021, by = "sa4") %>%
  left_join(entropy_2021, by = "sa4") %>%
  left_join(ccs_2021, by = "sa4") %>%
  left_join(controls_avg, by = "sa4") %>%
  left_join(net_worth_avg, by = "sa4") %>%
  left_join(burt_constraint, by = "sa4") %>%
  mutate(burt_constraint = replace_na(burt_constraint, 0)) %>%
  left_join(in_deals_pc_final, by = "sa4") %>%
  mutate(in_deals_pc = replace_na(in_deals_pc, 0)) %>%
  left_join(specialisation_clean, by = "sa4") %>%
  left_join(firm_size_clean, by = "sa4")
homeostasis_analysis <- homeostasis_analysis %>%
  mutate(density_100 = population_density_avg / 100,
         networth_10k = household_net_worth_avg / 10000)
finance_analysis <- finance_analysis %>%
  mutate(density_100 = population_density_avg / 100,
         networth_10k = household_net_worth_avg / 10000)
homeostasis_analysis <- homeostasis_analysis %>%
  mutate(se1_c = se1_avg - mean(se1_avg, na.rm = TRUE),
         ccs_c = creative_class_share - mean(creative_class_share, na.rm = TRUE),
         entropy_c = entropy - mean(entropy, na.rm = TRUE))
finance_analysis <- finance_analysis %>%
  mutate(se2_c = se2_avg - mean(se2_avg, na.rm = TRUE),
```

```
        ccs_c = creative_class_share - mean(creative_class_share, na.rm = TRUE),
        entropy_c = entropy - mean(entropy, na.rm = TRUE))
homeostasis_analysis <- homeostasis_analysis %>%
  mutate(z_homeostasis = zscore(homeostasis), z_se1 = zscore(se1_avg),
        z_ccs = zscore(creative_class_share), z_entropy = zscore(entropy),
        z_density = zscore(population_density_avg),
        z_bachelor = zscore(bachelor_pct), z_income = zscore(log_med_income_avg),
        z_unemp = zscore(unemployment_rate_avg), z_age = zscore(age_25_64_avg),
        z_emprate = zscore(employment_rate_avg), z_networth = zscore(household_net_worth_avg),
        z_burtc_mig = zscore(burt_constraint_mig), z_specialisation = zscore(specialisation_index),
        z_firmsize = zscore(share_large_20plus))
finance_analysis <- finance_analysis %>%
  mutate(z_new_firms = zscore(new_firms_percapita), z_se2 = zscore(se2_avg),
        z_ccs = zscore(creative_class_share), z_entropy = zscore(entropy),
        z_density = zscore(population_density_avg),
        z_bachelor = zscore(bachelor_pct), z_income = zscore(log_med_income_avg),
        z_unemp = zscore(unemployment_rate_avg), z_age = zscore(age_25_64_avg),
        z_emprate = zscore(employment_rate_avg), z_networth = zscore(household_net_worth_avg),
        z_burtc = zscore(burt_constraint), z_indeals = zscore(in_deals_pc), z_specialisation =
zscore(specialisation_index),
        z_firmsize = zscore(share_large_20plus))
cat(sprintf("[PART 8] homeostasis_analysis: %d rows | finance_analysis: %d rows\n",
           nrow(homeostasis_analysis), nrow(finance_analysis)))
homeo_model_vars <- c("se1_c", "ccs_c", "entropy_c", "density_100", "log_med_income_avg",
                      "unemployment_rate_avg", "age_25_64_avg", "employment_rate_avg",
                      "specialisation_index")
cat("\n[PART 8] Missingness by variable, homeostasis_analysis (out of", nrow(homeostasis_analysis),
"regions):\n")
for (v in homeo_model_vars) {
  n_na <- sum(is.na(homeostasis_analysis[[v]]))
  if (n_na > 0) {
    missing_sa4 <- homeostasis_analysis$sa4[is.na(homeostasis_analysis[[v]])]
    cat(sprintf("  %-25s %d missing (sa4: %s)\n", v, n_na, paste(missing_sa4, collapse = ", ")))
  }
}
vif_check_homeo_diagnostic <- lm(homeostasis ~ se1_c + ccs_c + entropy_c +
                                   log_med_income_avg + unemployment_rate_avg + age_25_64_avg +
                                   employment_rate_avg + networth_10k + density_100 +
                                   burt_constraint_mig, data = homeostasis_analysis)
cat("\n[PART 8] Diagnostic VIF - Homeostasis model including Burt's constraint (migration) and net worth:\n")
print(vif(vif_check_homeo_diagnostic))
vif_check_homeo <- lm(homeostasis ~ se1_c + ccs_c + entropy_c +
                        log_med_income_avg + unemployment_rate_avg + age_25_64_avg +
                        employment_rate_avg + density_100 +
                        specialisation_index, data = homeostasis_analysis)
cat("\n[PART 8] VIF - Homeostasis main-effects model (Table C5):\n")
homeo_vif_values <- vif(vif_check_homeo)
print(homeo_vif_values)
cat(sprintf("[PART 8] MEAN VIF (Homeostasis): %.3f\n", mean(homeo_vif_values)))
vif_check_finance <- lm(new_firms_percapita ~ se2_c + ccs_c + entropy_c +
                          log_med_income_avg + unemployment_rate_avg + age_25_64_avg +
                          employment_rate_avg + density_100 +
                          burt_constraint + in_deals_pc + specialisation_index,
                        data = finance_analysis)
cat("\n[PART 8] VIF - Productive Entrepreneurship main-effects model (Table C6):\n")
fin_vif_values <- vif(vif_check_finance)
print(fin_vif_values)
cat(sprintf("[PART 8] MEAN VIF (Productive Entrepreneurship): %.3f\n", mean(fin_vif_values)))
# ============================================================================
# PART 9. TABLE C3: HOMEOSTASIS ~ MIGRATION CONCENTRATION x ETHNIC DIVERSITY x CCS
# ============================================================================
controls_formula_base    <- "density_100 + log_med_income_avg + unemployment_rate_avg + age_25_64_avg +
employment_rate_avg + specialisation_index"
controls_formula_base_z  <- "z_density + z_income + z_unemp + z_age + z_emprate + z_specialisation"
controls_formula_homeo   <- controls_formula_base
controls_formula_homeo_z <- controls_formula_base_z
f_h1 <- as.formula(paste("homeostasis ~ se1_c +", controls_formula_homeo))
f_h2 <- as.formula(paste("homeostasis ~ ccs_c +", controls_formula_homeo))
f_h3 <- as.formula(paste("homeostasis ~ entropy_c +", controls_formula_homeo))
f_h4 <- as.formula(paste("homeostasis ~ se1_c + ccs_c + entropy_c +", controls_formula_homeo))
f_h5 <- as.formula(paste("homeostasis ~ se1_c * entropy_c + ccs_c +", controls_formula_homeo))
f_h6 <- as.formula(paste("homeostasis ~ entropy_c * ccs_c + se1_c +", controls_formula_homeo))
f_h7 <- as.formula(paste("homeostasis ~ se1_c * ccs_c + entropy_c +", controls_formula_homeo))
f_h8 <- as.formula(paste("homeostasis ~ se1_c * entropy_c * ccs_c +", controls_formula_homeo))
f_h1z <- as.formula(paste("z_homeostasis ~ z_se1 +", controls_formula_homeo_z))
```

```
f_h2z <- as.formula(paste("z_homeostasis ~ z_ccs +", controls_formula_homeo_z))
f_h3z <- as.formula(paste("z_homeostasis ~ z_entropy +", controls_formula_homeo_z))
f_h4z <- as.formula(paste("z_homeostasis ~ z_se1 + z_ccs + z_entropy +", controls_formula_homeo_z))
f_h5z <- as.formula(paste("z_homeostasis ~ z_se1 * z_entropy + z_ccs +", controls_formula_homeo_z))
f_h6z <- as.formula(paste("z_homeostasis ~ z_entropy * z_ccs + z_se1 +", controls_formula_homeo_z))
f_h7z <- as.formula(paste("z_homeostasis ~ z_se1 * z_ccs + z_entropy +", controls_formula_homeo_z))
f_h8z <- as.formula(paste("z_homeostasis ~ z_se1 * z_entropy * z_ccs +", controls_formula_homeo_z))
homeo_raw <- lapply(list(f_h1, f_h2, f_h3, f_h4, f_h5, f_h6, f_h7, f_h8), lm, data =
    homeostasis_analysis)
homeo_std <- lapply(list(f_h1z, f_h2z, f_h3z, f_h4z, f_h5z, f_h6z, f_h7z, f_h8z), lm, data =
homeostasis_analysis)
homeo_var_specs <- list(
  "Migration Concentration"         = list(raw = "se1_c",                    z = "z_se1"),
  "Creative-Class Share"            = list(raw = "ccs_c",                    z = "z_ccs"),
  "Ethnic Diversity"                = list(raw = "entropy_c",                z = "z_entropy"),
  "MC x Ethnic Diversity"           = list(raw = "se1_c:entropy_c",          z = "z_se1:z_entropy"),
  "Ethnic Diversity x CCS"          = list(raw = "entropy_c:ccs_c",          z = "z_entropy:z_ccs"),
  "MC x CCS"                        = list(raw = "se1_c:ccs_c",              z = "z_se1:z_ccs"),
  "MC x Ethnic Diversity x CCS"     = list(raw = "se1_c:entropy_c:ccs_c", z = "z_se1:z_entropy:z_ccs"),
  "Population Density"              = list(raw = "density_100",        z = "z_density"),
  "Median Income (log)"             = list(raw = "log_med_income_avg",       z = "z_income"),
  "Unemployment Rate"               = list(raw = "unemployment_rate_avg",    z = "z_unemp"),
  "Working-Age Population"          = list(raw = "age_25_64_avg",            z = "z_age"),
  "Employment Rate"                 = list(raw = "employment_rate_avg",      z = "z_emprate"),
  "Specialisation Index"            = list(raw = "specialisation_index",      z = "z_specialisation")
)
cat("\n\n================================================================================\n")
cat("TABLE C3. Effects of Migration Concentration on Entrepreneurial Ecosystem\n")
cat("Homeostasis (robustness specification).\n")
cat("================================================================================\n")
print_results_table(
  homeo_raw, homeo_std,
  model_labels = c("(1)", "(2)", "(3)", "(4)", "(5)", "(6)", "(7)", "(8)"),
  var_specs = homeo_var_specs
)
# ================================================================================
# PART 10. TABLE C4: PRODUCTIVE ENTREPRENEURSHIP ~ FINANCIAL NETWORK
# EMBEDDEDNESS x ETHNIC DIVERSITY x CCS
# ================================================================================
controls_formula_fin   <- paste(controls_formula_base, "+ burt_constraint + in_deals_pc")
controls_formula_fin_z <- paste(controls_formula_base_z, "+ z_burtc + z_indeals")
f_f1 <- as.formula(paste("new_firms_percapita ~ se2_c +", controls_formula_fin))
f_f2 <- as.formula(paste("new_firms_percapita ~ ccs_c +", controls_formula_fin))
f_f3 <- as.formula(paste("new_firms_percapita ~ entropy_c +", controls_formula_fin))
f_f4 <- as.formula(paste("new_firms_percapita ~ se2_c + ccs_c + entropy_c +", controls_formula_fin))
f_f5 <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c + ccs_c +", controls_formula_fin))
f_f6 <- as.formula(paste("new_firms_percapita ~ entropy_c * ccs_c + se2_c +", controls_formula_fin))
f_f7 <- as.formula(paste("new_firms_percapita ~ se2_c * ccs_c + entropy_c +", controls_formula_fin))
f_f8 <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +", controls_formula_fin))
f_f1z <- as.formula(paste("z_new_firms ~ z_se2 +", controls_formula_fin_z))
f_f2z <- as.formula(paste("z_new_firms ~ z_ccs +", controls_formula_fin_z))
f_f3z <- as.formula(paste("z_new_firms ~ z_entropy +", controls_formula_fin_z))
f_f4z <- as.formula(paste("z_new_firms ~ z_se2 + z_ccs + z_entropy +", controls_formula_fin_z))
f_f5z <- as.formula(paste("z_new_firms ~ z_se2 * z_entropy + z_ccs +", controls_formula_fin_z))
f_f6z <- as.formula(paste("z_new_firms ~ z_entropy * z_ccs + z_se2 +", controls_formula_fin_z))
f_f7z <- as.formula(paste("z_new_firms ~ z_se2 * z_ccs + z_entropy +", controls_formula_fin_z))
f_f8z <- as.formula(paste("z_new_firms ~ z_se2 * z_entropy * z_ccs +", controls_formula_fin_z))
fin_raw <- lapply(list(f_f1, f_f2, f_f3, f_f4, f_f5, f_f6, f_f7, f_f8), lm, data = finance_analysis)
fin_std <- lapply(list(f_f1z, f_f2z, f_f3z, f_f4z, f_f5z, f_f6z, f_f7z, f_f8z), lm, data = finance_analysis)
fin_var_specs <- list(
  "Financial Network Embeddedness"  = list(raw = "se2_c",                    z = "z_se2"),
  "Creative-Class Share"            = list(raw = "ccs_c",                    z = "z_ccs"),
  "Ethnic Diversity"                = list(raw = "entropy_c",                z = "z_entropy"),
  "FNE x Ethnic Diversity"          = list(raw = "se2_c:entropy_c",          z = "z_se2:z_entropy"),
  "Ethnic Diversity x CCS"          = list(raw = "entropy_c:ccs_c",          z = "z_entropy:z_ccs"),
  "FNE x CCS"                       = list(raw = "se2_c:ccs_c",              z = "z_se2:z_ccs"),
  "FNE x Ethnic Diversity x CCS"    = list(raw = "se2_c:entropy_c:ccs_c",   z = "z_se2:z_entropy:z_ccs"),
  "Population Density"              = list(raw = "density_100",          z = "z_density"),
  "Median Income (log)"             = list(raw = "log_med_income_avg",       z = "z_income"),
  "Unemployment Rate"               = list(raw = "unemployment_rate_avg",    z = "z_unemp"),
  "Working-Age Population"          = list(raw = "age_25_64_avg",            z = "z_age"),
  "Employment Rate"                 = list(raw = "employment_rate_avg",      z = "z_emprate"),
  "Specialisation Index"            = list(raw = "specialisation_index",     z = "z_specialisation"),
  "Burt's Constraint (VC Network)"  = list(raw = "burt_constraint",   z = "z_burtc"),
  "VC Deal Volume, per capita"      = list(raw = "in_deals_pc",       z = "z_indeals")
```

```
)
cat("\n\n================================================================================\n")
cat("TABLE C4. Effects of Financial Network Embeddedness on Productive\n")
cat("Entrepreneurship (robustness specification).\n")
cat("================================================================================\n")
print_results_table(
  fin_raw, fin_std,
  model_labels = c("(1)", "(2)", "(3)", "(4)", "(5)", "(6)", "(7)", "(8)"),
  var_specs = fin_var_specs
)
```

II.C Supplementary Checks and Candidate Controls

These scripts produce the supplementary checks and the candidate-control tests summarised in Table C8 and Section 4.4.5.

II.C.1 Alternative Moderator: KIBS Employment Share

`Paper3_Script7_Alternative_Moderator_KIBS.R`

Supplementary check: KIBS employment share replaces Creative-Class Share in the Productive Entrepreneurship three-way interaction.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 7: ALTERNATIVE MODERATOR (KIBS IN PLACE OF
# CREATIVE-CLASS SHARE; TABLE C7)
#
# Replaces Creative-Class Share with KIBS employment share inside the
# three-way interaction of the Productive Entrepreneurship model; Creative-
# Class Share is retained as an additive control. The check is not run for
# Homeostasis because KIBS is one of the three components of that outcome.
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session.
# ============================================================================
library(lmtest); library(sandwich); library(car)

required_objects <- c("finance_analysis", "fin_raw", "controls_formula_fin", "kibs_2023_cross")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script1_Primary_Specification.R first, in the same
R session.",
               paste(missing_objects, collapse = ", ")))
}


robust_summary <- function(model) coeftest(model, vcov = vcovHC(model, type = "HC1"))

# ============================================================================
# JOIN KIBS SHARE AND MEAN-CENTRE
# ============================================================================
fin_test <- finance_analysis %>%
  left_join(kibs_2023_cross, by = "sa4") %>%
  mutate(kibs_c = kibs - mean(kibs, na.rm = TRUE))

cat(sprintf("[CHECK] kibs joined: %d regions | r(kibs, creative_class_share) = %.3f\n",
            sum(!is.na(fin_test$kibs)),
            cor(fin_test$kibs, fin_test$creative_class_share, use = "pairwise.complete.obs")))

# ============================================================================
# ALTERNATIVE MODEL: FNE x Ethnic Diversity x KIBS, CCS as additive control
# ============================================================================
f_alt <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * kibs_c + ccs_c +",
controls_formula_fin))
m_alt <- lm(f_alt, data = fin_test)

cat(" Productive Entrepreneurship: FNE x Ethnic Diversity x KIBS (CCS as additive control) ")
print(robust_summary(m_alt)[c("se2_c","entropy_c","kibs_c","ccs_c","se2_c:entropy_c:kibs_c"), ])
cat(sprintf("N=%d | Adj R2=%.3f\n", nobs(m_alt), summary(m_alt)$adj.r.squared))

# ============================================================================
# COMPARISON WITH THE PRIMARY MODEL
# ============================================================================
orig_3way <- coeftest(fin_raw[[8]], vcov = vcovHC(fin_raw[[8]], type="HC1"))["se2_c:entropy_c:ccs_c", ]
alt_3way  <- robust_summary(m_alt)["se2_c:entropy_c:kibs_c", ]

cat("\n============================================================================\n")
cat("Three-way interaction: Creative-Class Share versus KIBS as moderator\n")
cat("============================================================================\n")
cat(sprintf("  Primary (FNE x ED x CCS):     est=%.4f, p=%.4f\n", orig_3way["Estimate"],
orig_3way["Pr(>|t|)"]))
cat(sprintf("  Alternative (FNE x ED x KIBS): est=%.4f, p=%.4f\n", alt_3way["Estimate"],
    alt_3way["Pr(>|t|)"]))
```

II.C.2 Candidate Control: Business Churn

Paper3_Script8_Candidate_Business_Churn.R

Table C8 (panel A): confirms that business entries and exits are not observed before 2021.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 8: CANDIDATE CONTROL - BUSINESS CHURN (TABLE C8)
#
# Lists the data items and the year range in the ABS business entries and
# exits file. Entries and exits are observed only from 2021, so business
# churn cannot be measured within the <= 2020 predictor window.
# ============================================================================
library(readxl)
library(dplyr)

f_business_churn <- file.path("data", "Business_Churn_SA4_202124.xlsx")

churn_check <- read_excel(f_business_churn)
cat("[CHECK] Unique 'Data Item' values in this file:\n")
print(unique(churn_check$`Data Item`))
cat(sprintf("\n[CHECK] Year range: %d-%d\n", min(churn_check$TIME_PERIOD, na.rm=TRUE),
max(churn_check$TIME_PERIOD, na.rm=TRUE)))
```

II.C.3 Candidate Controls: Net Migration Rate and Household Net Worth

`Paper3_Script9_Candidate_NetMigration_NetWorth.R`

Table C8: correlation checks and Models 4 and 8 with both candidates added.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 9: CANDIDATE CONTROLS - NET MIGRATION RATE
# AND HOUSEHOLD NET WORTH (TABLE C8)
#
#   - Net migration rate (net inflow per 1,000 residents, <= 2020) is built
#     from the same migration flows as Migration Concentration; its
#     correlation with the focal variables is reported to assess overlap.
#   - Household net worth (average of the 2016-2020 observations) is checked
#     against the existing socio-economic controls.
# Both are then added to Models 4 and 8 of Tables 4.5 and 4.6, with VIFs and
# a comparison of the three-way interaction.
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session.
# ============================================================================
library(lmtest); library(sandwich); library(car)

required_objects <- c("migration_raw", "population_avg_leq2020", "homeostasis_analysis",
                      "finance_analysis", "sa4_lookup_clean", "controls_formula_homeo",
                      "controls_formula_fin", "controls_formula_homeo_z", "controls_formula_fin_z")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script1_Primary_Specification.R first, in the same
R session.",
               paste(missing_objects, collapse = ", ")))
}

f_progress_dashboard <- file.path("data", "progressdashboard_2024data (2).csv")
robust_summary <- function(model) coeftest(model, vcov = vcovHC(model, type = "HC1"))
zscore <- function(x) as.numeric(scale(x))

# ============================================================================
# PART 1. NET MIGRATION RATE (from migration_raw, <= 2020)
# ============================================================================
inflow <- migration_raw %>% group_by(j) %>% summarise(inflow = sum(weight, na.rm = TRUE)) %>% rename(sa4 = j)
outflow <- migration_raw %>% group_by(i) %>% summarise(outflow = sum(weight, na.rm = TRUE)) %>% rename(sa4 =
i)

net_migration_rate <- sa4_lookup_clean %>% distinct(sa4 = sa4_code) %>%
  left_join(inflow, by = "sa4") %>% left_join(outflow, by = "sa4") %>%
  mutate(inflow = replace_na(inflow, 0), outflow = replace_na(outflow, 0), net_flow = inflow - outflow) %>%
  left_join(population_avg_leq2020, by = "sa4") %>%
  mutate(net_migration_rate = (net_flow / population_avg_leq2020) * 1000) %>%
  select(sa4, net_migration_rate)

cat(sprintf("[PART 1] net_migration_rate: %d regions | mean=%.2f, sd=%.2f, min=%.2f, max=%.2f\n",
            nrow(net_migration_rate), mean(net_migration_rate$net_migration_rate, na.rm=TRUE),
            sd(net_migration_rate$net_migration_rate, na.rm=TRUE),
                min(net_migration_rate$net_migration_rate, na.rm=TRUE),
            max(net_migration_rate$net_migration_rate, na.rm=TRUE)))

# ============================================================================
# PART 2. HOUSEHOLD NET WORTH (average of observations <= 2020)
# ============================================================================
pd_raw <- read_csv(f_progress_dashboard, show_col_types = FALSE)

household_net_worth <- pd_raw %>%
  filter(region_type == "sa4_code_2021", full_variable_name == "Real household net worth",
         as.numeric(date) <= 2020) %>%
  group_by(sa4 = as.character(region_id)) %>%
  summarise(household_net_worth = mean(value, na.rm = TRUE), .groups = "drop")

cat(sprintf("[PART 2] household_net_worth: %d regions | mean=%.1f, sd=%.1f, min=%.1f, max=%.1f\n",
            nrow(household_net_worth), mean(household_net_worth$household_net_worth, na.rm=TRUE),
            sd(household_net_worth$household_net_worth, na.rm=TRUE),
                min(household_net_worth$household_net_worth, na.rm=TRUE),
            max(household_net_worth$household_net_worth, na.rm=TRUE)))

# ============================================================================
```

```
# PART 3. MERGE AND CORRELATION CHECKS
# ============================================================================
homeostasis_expanded <- homeostasis_analysis %>%
  left_join(net_migration_rate, by = "sa4") %>%
  left_join(household_net_worth, by = "sa4")
finance_expanded <- finance_analysis %>%
  left_join(net_migration_rate, by = "sa4") %>%
  left_join(household_net_worth, by = "sa4")

cat("\n============================================================================\n")
cat("Correlation of net migration rate with the focal explanatory variables\n")
cat("============================================================================\n")
cat(sprintf("  r(net_migration_rate, se1_avg) = %.3f  [Migration Concentration]\n",
            cor(homeostasis_expanded$net_migration_rate, homeostasis_expanded$se1_avg, use =
                "pairwise.complete.obs")))
cat(sprintf("  r(net_migration_rate, se2_avg) = %.3f  [Financial Network Embeddedness]\n",
            cor(finance_expanded$net_migration_rate, finance_expanded$se2_avg, use =
                "pairwise.complete.obs")))

cat("\n============================================================================\n")
cat("Correlation of household net worth with existing controls\n")
cat("============================================================================\n")
for (v in c("density_100", "log_med_income_avg", "creative_class_share", "employment_rate_avg")) {
  cat(sprintf("  r(household_net_worth, %s) = %.3f\n", v,
              cor(homeostasis_expanded$household_net_worth, homeostasis_expanded[[v]], use =
                  "pairwise.complete.obs")))
}

# ============================================================================
# PART 4. MODELS 4 AND 8 WITH THE TWO CANDIDATE CONTROLS ADDED
# ============================================================================
homeostasis_expanded <- homeostasis_expanded %>%
  mutate(z_netmig = zscore(net_migration_rate), z_hnw = zscore(household_net_worth))
finance_expanded <- finance_expanded %>%
  mutate(z_netmig = zscore(net_migration_rate), z_hnw = zscore(household_net_worth))

controls_expanded_homeo   <- paste(controls_formula_homeo, "+ net_migration_rate + household_net_worth")
controls_expanded_homeo_z <- paste(controls_formula_homeo_z, "+ z_netmig + z_hnw")
controls_expanded_fin     <- paste(controls_formula_fin, "+ net_migration_rate + household_net_worth")
controls_expanded_fin_z   <- paste(controls_formula_fin_z, "+ z_netmig + z_hnw")

f_h4_exp <- as.formula(paste("homeostasis ~ se1_c +", controls_expanded_homeo))
f_h8_exp <- as.formula(paste("homeostasis ~ se1_c * entropy_c * ccs_c +", controls_expanded_homeo))
f_f4_exp <- as.formula(paste("new_firms_percapita ~ se2_c +", controls_expanded_fin))
f_f8_exp <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +",
    controls_expanded_fin))

m_h4_exp <- lm(f_h4_exp, data = homeostasis_expanded)
m_h8_exp <- lm(f_h8_exp, data = homeostasis_expanded)
m_f4_exp <- lm(f_f4_exp, data = finance_expanded)
m_f8_exp <- lm(f_f8_exp, data = finance_expanded)

cat("\n============================================================================\n")
cat("Homeostasis, Model 4 + candidate controls\n")
cat("============================================================================\n")
print(robust_summary(m_h4_exp))
cat(sprintf("N = %d | Adj R2 = %.3f\n", nobs(m_h4_exp), summary(m_h4_exp)$adj.r.squared))

cat("\n============================================================================\n")
cat("Homeostasis, Model 8 + candidate controls\n")
cat("============================================================================\n")
print(robust_summary(m_h8_exp))
cat(sprintf("N = %d | Adj R2 = %.3f\n", nobs(m_h8_exp), summary(m_h8_exp)$adj.r.squared))

cat("\n============================================================================\n")
cat("Productive Entrepreneurship, Model 4 + candidate controls\n")
cat("============================================================================\n")
print(robust_summary(m_f4_exp))
cat(sprintf("N = %d | Adj R2 = %.3f\n", nobs(m_f4_exp), summary(m_f4_exp)$adj.r.squared))

cat("\n============================================================================\n")
cat("Productive Entrepreneurship, Model 8 + candidate controls\n")
cat("============================================================================\n")
print(robust_summary(m_f8_exp))
cat(sprintf("N = %d | Adj R2 = %.3f\n", nobs(m_f8_exp), summary(m_f8_exp)$adj.r.squared))
```

```
# ============================================================================
# PART 5. VARIANCE INFLATION FACTORS
# ============================================================================
cat("\n============================================================================\n")
cat("VIF: Homeostasis, Model 4 + candidate controls\n")
cat("============================================================================\n")
print(vif(m_h4_exp))
cat(sprintf("MEAN VIF: %.3f\n", mean(vif(m_h4_exp))))

cat("\n============================================================================\n")
cat("VIF: Productive Entrepreneurship, Model 4 + candidate controls\n")
cat("============================================================================\n")
print(vif(m_f4_exp))
cat(sprintf("MEAN VIF: %.3f\n", mean(vif(m_f4_exp))))

# ============================================================================
# PART 6. THREE-WAY INTERACTION WITH AND WITHOUT THE CANDIDATE CONTROLS
# ============================================================================
orig_3way_f <- coeftest(fin_raw[[8]], vcov = vcovHC(fin_raw[[8]], type="HC1"))["se2_c:entropy_c:ccs_c", ]
exp_3way_f  <- coeftest(m_f8_exp, vcov = vcovHC(m_f8_exp, type="HC1"))["se2_c:entropy_c:ccs_c", ]
cat("\n============================================================================\n")
cat("Productive Entrepreneurship three-way interaction, with and without candidates\n")
cat("============================================================================\n")
cat(sprintf("  Primary specification:          estimate=%.4f, p=%.4f\n", orig_3way_f["Estimate"],
    orig_3way_f["Pr(>|t|)"]))
cat(sprintf("  + net migration + net worth:    estimate=%.4f, p=%.4f\n", exp_3way_f["Estimate"],
    exp_3way_f["Pr(>|t|)"]))
```

II.C.4 Candidate Controls: Extended Set

Paper3_Script10_Candidate_Extended_Controls.R

Table C8: bachelor-degree share, SEIFA IRSAD, population density and household net worth.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 10: CANDIDATE CONTROLS - EXTENDED SET
# (TABLE C8)
#
# Sources:
#   ABS Data by Region (All SA4s.data.csv; long format, MEASURE codes):
#     ERP_21   population density (persons per km2), averaged
#     SCHOOL_5 bachelor degree or higher (%), 2021
#     IHAD_7   SEIFA Index of Relative Socio-economic Advantage and
#              Disadvantage, 2021
#   Regional progress dashboard (region_id = SA4 code):
#     real_household_net_worth, averaged over 2016, 2018 and 2020
# Both sources are joined on the numeric SA4 code.
#
# Other dashboard variables were not tested because they duplicate existing
# controls (qualifications, household income, unemployment) or overlap with
# the outcomes (business entry rate; knowledge-intensive services, which
# form part of Homeostasis).
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session.
# ============================================================================

f_all_sa4_data  <- file.path("data", "All SA4s.data.csv")
f_dashboard     <- file.path("data", "progressdashboard_2024data (2).csv")

# ---- (i) Population density (averaged) ----------------------------------
abs_regional_raw <- read_csv(f_all_sa4_data, show_col_types = FALSE)

density_avg <- abs_regional_raw %>%
  filter(REGIONTYPE == "SA4", MEASURE == "ERP_21", !is.na(OBS_VALUE)) %>%
  mutate(sa4 = as.character(ASGS_2021)) %>%
  group_by(sa4) %>%
  summarise(population_density_avg = mean(OBS_VALUE, na.rm = TRUE), .groups = "drop")

# ---- (ii) Bachelor degree share, 2021 -----------------------------------
bachelor_2021 <- abs_regional_raw %>%
  filter(REGIONTYPE == "SA4", MEASURE == "SCHOOL_5", !is.na(OBS_VALUE)) %>%
  mutate(sa4 = as.character(ASGS_2021)) %>%
  transmute(sa4, bachelor_pct = OBS_VALUE)

# ---- (iii) SEIFA IRSAD, 2021 --------------------------------------------
seifa_2021 <- abs_regional_raw %>%
  filter(REGIONTYPE == "SA4", MEASURE == "IHAD_7", !is.na(OBS_VALUE)) %>%
  mutate(sa4 = as.character(ASGS_2021)) %>%
  transmute(sa4, seifa_irsad = OBS_VALUE)

# ---- (iv) Real household net worth (2016, 2018, 2020 average) -----------
dashboard_raw <- read_csv(f_dashboard, show_col_types = FALSE)

net_worth_avg <- dashboard_raw %>%
  filter(region_type == "sa4_code_2021", variable == "real_household_net_worth",
         !is.na(value)) %>%
  mutate(sa4 = as.character(region_id)) %>%
  group_by(sa4) %>%
  summarise(household_net_worth_avg = mean(value, na.rm = TRUE), .groups = "drop")

# ---- (v) Combine --------------------------------------------------------
controls_extended <- density_avg %>%
  full_join(bachelor_2021, by = "sa4") %>%
  full_join(seifa_2021, by = "sa4") %>%
  full_join(net_worth_avg, by = "sa4")

cat(sprintf("[CHECK] controls_extended: %d regions\n", nrow(controls_extended)))
cat(sprintf("  population_density_avg  : %d non-missing\n", sum(!
    is.na(controls_extended$population_density_avg))))
cat(sprintf("  bachelor_pct            : %d non-missing\n", sum(!
    is.na(controls_extended$bachelor_pct))))
cat(sprintf("  seifa_irsad             : %d non-missing\n", sum(!is.na(controls_extended$seifa_irsad))))
```

```
cat(sprintf("  household_net_worth_avg : %d non-missing\n", sum(!
    is.na(controls_extended$household_net_worth_avg))))

# ---- (vi) Check that all SA4 codes match the analysis datasets -------------
unmatched <- controls_extended %>%
  filter(!sa4 %in% sa4_lookup_clean$sa4_code)

if (nrow(unmatched) > 0) {
  cat(sprintf("\n[CHECK] %d SA4 codes do not match the analysis datasets:\n", nrow(unmatched)))
  print(unmatched)
} else {
  cat("\n[CHECK] All SA4 codes match.\n")
}

# ============================================================================
# MERGE INTO THE ANALYSIS DATASETS
# Only variables not already present are added, so the population density
# used in the primary models is retained.
# ============================================================================
homeostasis_analysis <- homeostasis_analysis %>%
  left_join(controls_extended %>%
              dplyr::select(sa4, any_of(setdiff(names(controls_extended), names(homeostasis_analysis)))),
            by = "sa4")

finance_analysis <- finance_analysis %>%
  left_join(controls_extended %>%
              dplyr::select(sa4, any_of(setdiff(names(controls_extended), names(finance_analysis)))),
            by = "sa4")

cat(sprintf(
  "\n[MERGE] homeostasis_analysis: %d columns | finance_analysis: %d columns\n",
  ncol(homeostasis_analysis), ncol(finance_analysis)
))

# ============================================================================
# MAIN-EFFECTS MODELS WITH THE RETAINED CANDIDATES
# ============================================================================
# With all four candidates, bachelor-degree share and SEIFA IRSAD were highly
# collinear with Creative-Class Share and median income (VIFs above 20), so
# the models below add population density and household net worth only.
# ============================================================================
model_homeostasis_extended <- lm(
  homeostasis ~ se1_avg + entropy + creative_class_share +
    employment_rate_avg + log_med_income_avg + age_25_64_avg + unemployment_rate_avg +
    population_density_avg + household_net_worth_avg,
  data = homeostasis_analysis
)

cat("\nHomeostasis ~ Migration Concentration + extended controls (density, net worth):\n")
print(robust_summary(model_homeostasis_extended))

model_finance_extended <- lm(
  new_firms_percapita ~ se2_avg + entropy + creative_class_share +
    employment_rate_avg + log_med_income_avg + age_25_64_avg + unemployment_rate_avg +
    population_density_avg + household_net_worth_avg,
  data = finance_analysis
)

cat("\nProductive Entrepreneurship ~ FNE + extended controls (density, net worth):\n")
print(robust_summary(model_finance_extended))
```

II.C.5 Candidate Controls in the Robustness Specification

`Paper3_Script11_Candidates_Robustness_Specification.R`

Section 4.4.5: net migration rate, voluntary work, homelessness and business density added together (robustness specification). Run after Script 6.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 11: CANDIDATE CONTROLS IN THE ROBUSTNESS
# SPECIFICATION (TABLE C8)
#
# Adds four candidate controls (net migration rate, voluntary work,
# homelessness, business density) to the robustness specification. Each
# candidate uses the window that matches that specification:
#   - Net migration rate: full migration period
#   - Voluntary work and homelessness: 2021 Census
#   - Business density (businesses per 1,000 residents): 2024
#
# Run after Paper3_Script6_Robustness_Specification_TablesC3_C6.R, in the same R session.
# ============================================================================
library(lmtest); library(sandwich); library(car)

required_objects <- c("migration_raw", "homeostasis_analysis",
                      "finance_analysis", "homeo_raw", "fin_raw", "sa4_lookup_clean",
                      "controls_formula_homeo", "controls_formula_fin")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script6_Robustness_Specification_TablesC3_C6.R
first, in the same R session.",
               paste(missing_objects, collapse = ", ")))
}

# population_avg is rebuilt from population_clean if it is not present.
if (!exists("population_avg")) {
  if (exists("population_clean")) {
    cat("[CHECK] Rebuilding population_avg from population_clean.\n")
    population_avg <- population_clean %>%
      group_by(sa4) %>% summarise(population_avg = mean(population, na.rm = TRUE), .groups = "drop")
  } else {
    stop("Neither population_avg nor population_clean exist. Run
Paper3_Script6_Robustness_Specification_TablesC3_C6.R first, in the same R session.")
  }
}


f_progress_dashboard <- file.path("data", "progressdashboard_2024data (2).csv")
f_total_businesses <- file.path("data", "total_businesses_SA4.csv")
robust_summary <- function(model) coeftest(model, vcov = vcovHC(model, type = "HC1"))

# ============================================================================
# CANDIDATE CONTROLS
# ============================================================================
inflow <- migration_raw %>% group_by(j) %>% summarise(inflow = sum(weight, na.rm = TRUE)) %>% rename(sa4 = j)
outflow <- migration_raw %>% group_by(i) %>% summarise(outflow = sum(weight, na.rm = TRUE)) %>% rename(sa4 =
i)
net_migration_rate <- sa4_lookup_clean %>% distinct(sa4 = sa4_code) %>%
  left_join(inflow, by = "sa4") %>% left_join(outflow, by = "sa4") %>%
  mutate(inflow = replace_na(inflow, 0), outflow = replace_na(outflow, 0), net_flow = inflow - outflow) %>%
  left_join(population_avg, by = "sa4") %>%
  mutate(net_migration_rate = (net_flow / population_avg) * 1000) %>%
  select(sa4, net_migration_rate)

pd_raw <- read_csv(f_progress_dashboard, show_col_types = FALSE)

voluntary_work_2021 <- pd_raw %>%
  filter(region_type == "sa4_code_2021", full_variable_name == "Voluntary work", as.numeric(date) == 2021)
%>%
  transmute(sa4 = as.character(region_id), voluntary_work = value)

homelessness_2021 <- pd_raw %>%
  filter(region_type == "sa4_code_2021", full_variable_name == "Homelessness", as.numeric(date) == 2021) %>%
  transmute(sa4 = as.character(region_id), homelessness_log = log(1 + value))

total_businesses_2024 <- read_csv(f_total_businesses, show_col_types = FALSE) %>%
```

```
  filter(MEASURE == "CABEE_5", TIME_PERIOD == 2024) %>%
  mutate(sa4 = as.character(ASGS_2021)) %>%
  transmute(sa4, total_businesses_count = OBS_VALUE) %>%
  left_join(population_avg, by = "sa4") %>%
  mutate(total_businesses_pc = (total_businesses_count / population_avg) * 1000)

cat(sprintf("[CHECK] net_migration_rate: %d | voluntary_work_2021: %d | homelessness_2021: %d |
total_businesses_2024: %d\n",
            nrow(net_migration_rate), nrow(voluntary_work_2021), nrow(homelessness_2021),
                nrow(total_businesses_2024)))

homeo_test <- homeostasis_analysis %>%
  left_join(net_migration_rate, by = "sa4") %>% left_join(voluntary_work_2021, by = "sa4") %>%
  left_join(homelessness_2021, by = "sa4") %>% left_join(total_businesses_2024, by = "sa4")
fin_test <- finance_analysis %>%
  left_join(net_migration_rate, by = "sa4") %>% left_join(voluntary_work_2021, by = "sa4") %>%
  left_join(homelessness_2021, by = "sa4") %>% left_join(total_businesses_2024, by = "sa4")

# ============================================================================
# CORRELATION CHECKS
# ============================================================================
cat("\n============================================================================\n")
cat("Correlation checks\n")
cat("============================================================================\n")
cat(sprintf("  r(net_migration_rate, se1_avg) = %.3f | r(net_migration_rate, se2_avg) = %.3f\n",
            cor(homeo_test$net_migration_rate, homeo_test$se1_avg, use="pairwise.complete.obs"),
            cor(fin_test$net_migration_rate, fin_test$se2_avg, use="pairwise.complete.obs")))
cat(sprintf("  r(total_businesses_pc, new_firms_percapita) = %.3f  [overlap with the outcome]\n",
            cor(fin_test$total_businesses_pc, fin_test$new_firms_percapita,
                use="pairwise.complete.obs")))
cat(sprintf("  r(voluntary_work, entropy) = %.3f  [overlap with Ethnic Diversity]\n",
            cor(homeo_test$voluntary_work, homeo_test$entropy, use="pairwise.complete.obs")))

# ============================================================================
# MODELS WITH ALL CANDIDATES
# ============================================================================
f_h4_test <- as.formula(paste("homeostasis ~ se1_c + net_migration_rate + voluntary_work + homelessness_log
+", controls_formula_homeo))
f_f4_test <- as.formula(paste("new_firms_percapita ~ se2_c + net_migration_rate + voluntary_work +
homelessness_log + total_businesses_pc +", controls_formula_fin))
f_f8_test <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c + net_migration_rate +
voluntary_work + homelessness_log + total_businesses_pc +", controls_formula_fin))

m_h4_test <- lm(f_h4_test, data = homeo_test)
m_f4_test <- lm(f_f4_test, data = fin_test)
m_f8_test <- lm(f_f8_test, data = fin_test)

cat("\n============================================================================\nHomeostasis, Model 4 +
candidates\n============================================================================\n")
print(robust_summary(m_h4_test))
cat(sprintf("N=%d\n", nobs(m_h4_test)))

cat("\n============================================================================\nProductive
Entrepreneurship, Model 4 +
candidates\n============================================================================\n")
print(robust_summary(m_f4_test))
cat(sprintf("N=%d\n", nobs(m_f4_test)))

cat("\n============================================================================\nProductive
Entrepreneurship, Model 8 +
candidates\n============================================================================\n")
print(robust_summary(m_f8_test))
cat(sprintf("N=%d\n", nobs(m_f8_test)))

# ============================================================================
# THREE-WAY INTERACTION WITH AND WITHOUT THE CANDIDATES
# ============================================================================
orig_3way <- coeftest(fin_raw[[8]], vcov = vcovHC(fin_raw[[8]], type="HC1"))["se2_c:entropy_c:ccs_c", ]
exp_3way  <- robust_summary(m_f8_test)["se2_c:entropy_c:ccs_c", ]
cat("\n============================================================================\n")
cat("Three-way interaction (robustness specification), with and without candidates\n")
cat("============================================================================\n")
cat(sprintf("  Robustness specification:       est=%.4f, p=%.4f\n", orig_3way["Estimate"],
    orig_3way["Pr(>|t|)"]))
```

```
cat(sprintf("  + four candidate controls:       est=%.4f, p=%.4f\n", exp_3way["Estimate"],
exp_3way["Pr(>|t|)"]))
```

II.C.6 Candidate Control: Homelessness

Paper3_Script12_Candidate_Homelessness.R

Table C8 (panel G): homelessness added to Models 4 and 8, with and without the highest-homelessness region.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 12: CANDIDATE CONTROL - HOMELESSNESS
# (TABLE C8)
#
# Adds log(1 + homelessness rate), 2016 Census, to Models 4 and 8 of the
# primary specification. One region has a homelessness rate far above all
# others, so the models are re-estimated with that region excluded.
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session.
# ============================================================================
library(lmtest); library(sandwich); library(car)

required_objects <- c("homeostasis_analysis", "finance_analysis", "sa4_lookup_clean",
                      "controls_formula_homeo", "controls_formula_fin",
                      "homeo_raw", "fin_raw")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script1_Primary_Specification.R first, in the same
R session.",
               paste(missing_objects, collapse = ", ")))
}

f_progress_dashboard <- file.path("data", "progressdashboard_2024data (2).csv")
robust_summary <- function(model) coeftest(model, vcov = vcovHC(model, type = "HC1"))

pd_raw <- read_csv(f_progress_dashboard, show_col_types = FALSE)

homelessness_2016 <- pd_raw %>%
  filter(region_type == "sa4_code_2021", full_variable_name == "Homelessness",
         as.numeric(date) == 2016) %>%
  transmute(sa4 = as.character(region_id), homelessness_raw = value,
            homelessness_log = log(1 + value))

cat(sprintf("[CHECK] homelessness_2016: %d regions\n", nrow(homelessness_2016)))

homeostasis_hml <- homeostasis_analysis %>% left_join(homelessness_2016, by = "sa4")
finance_hml     <- finance_analysis %>% left_join(homelessness_2016, by = "sa4")

# ============================================================================
# PART 1. ALL REGIONS
# ============================================================================
f_h4_hml <- as.formula(paste("homeostasis ~ se1_c + homelessness_log +", controls_formula_homeo))
f_h8_hml <- as.formula(paste("homeostasis ~ se1_c * entropy_c * ccs_c + homelessness_log +",
controls_formula_homeo))
f_f4_hml <- as.formula(paste("new_firms_percapita ~ se2_c + homelessness_log +", controls_formula_fin))
f_f8_hml <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c + homelessness_log +",
    controls_formula_fin))

m_h4_hml <- lm(f_h4_hml, data = homeostasis_hml)
m_h8_hml <- lm(f_h8_hml, data = homeostasis_hml)
m_f4_hml <- lm(f_f4_hml, data = finance_hml)
m_f8_hml <- lm(f_f8_hml, data = finance_hml)

cat("\n============================================================================\n")
cat("All regions: Homeostasis, Model 4\n")
cat("============================================================================\n")
print(robust_summary(m_h4_hml)["homelessness_log", ])
cat(sprintf("Migration Concentration: estimate=%.4f, p=%.4f | N=%d\n",
            robust_summary(m_h4_hml)["se1_c","Estimate"], robust_summary(m_h4_hml)["se1_c","Pr(>|t|)"],
                nobs(m_h4_hml)))

cat("\nAll regions: Productive Entrepreneurship, Model 8\n")
print(robust_summary(m_f8_hml)["homelessness_log", ])
cat(sprintf("Three-way: estimate=%.4f, p=%.4f | N=%d\n",
            robust_summary(m_f8_hml)["se2_c:entropy_c:ccs_c","Estimate"],
            robust_summary(m_f8_hml)["se2_c:entropy_c:ccs_c","Pr(>|t|)"], nobs(m_f8_hml)))

# ============================================================================
```

```
# PART 2. HIGHEST-HOMELESSNESS REGION EXCLUDED
# ============================================================================
outlier_sa4 <- homelessness_2016 %>% arrange(desc(homelessness_raw)) %>% slice(1) %>% pull(sa4)
cat(sprintf("\n[CHECK] Excluding region sa4=%s (homelessness rate=%.0f)\n",
            outlier_sa4, homelessness_2016$homelessness_raw[homelessness_2016$sa4==outlier_sa4]))

homeostasis_hml_noout <- homeostasis_hml %>% filter(sa4 != outlier_sa4)
finance_hml_noout     <- finance_hml %>% filter(sa4 != outlier_sa4)

m_h4_hml_noout <- lm(f_h4_hml, data = homeostasis_hml_noout)
m_f8_hml_noout <- lm(f_f8_hml, data = finance_hml_noout)

cat("\n============================================================================\n")
cat("Region excluded: Homeostasis, Model 4\n")
cat("============================================================================\n")
print(robust_summary(m_h4_hml_noout)["homelessness_log", ])
cat(sprintf("Migration Concentration: estimate=%.4f, p=%.4f | N=%d\n",
            robust_summary(m_h4_hml_noout)["se1_c","Estimate"], robust_summary(m_h4_hml_noout)
["se1_c","Pr(>|t|)"], nobs(m_h4_hml_noout)))

cat("\nRegion excluded: Productive Entrepreneurship, Model 8\n")
print(robust_summary(m_f8_hml_noout)["homelessness_log", ])
cat(sprintf("Three-way: estimate=%.4f, p=%.4f | N=%d\n",
            robust_summary(m_f8_hml_noout)["se2_c:entropy_c:ccs_c","Estimate"],
            robust_summary(m_f8_hml_noout)["se2_c:entropy_c:ccs_c","Pr(>|t|)"], nobs(m_f8_hml_noout)))

# ============================================================================
# PART 3. VIF AND COMPARISON WITH THE PRIMARY MODEL
# ============================================================================
cat("\n============================================================================\n")
cat("VIF (all regions)\n")
cat("============================================================================\n")
print(vif(m_h4_hml))
cat(sprintf("MEAN VIF: %.3f\n", mean(vif(m_h4_hml))))

orig_3way_f <- coeftest(fin_raw[[8]], vcov = vcovHC(fin_raw[[8]], type="HC1"))["se2_c:entropy_c:ccs_c", ]
cat("\n============================================================================\n")
cat("Three-way interaction (Productive Entrepreneurship)\n")
cat("============================================================================\n")
cat(sprintf("  Primary specification:             estimate=%.4f, p=%.4f\n", orig_3way_f["Estimate"],
    orig_3way_f["Pr(>|t|)"]))
cat(sprintf("  + homelessness, all regions:       estimate=%.4f, p=%.4f\n",
            robust_summary(m_f8_hml)["se2_c:entropy_c:ccs_c","Estimate"], robust_summary(m_f8_hml)
["se2_c:entropy_c:ccs_c","Pr(>|t|)"]))
cat(sprintf("  + homelessness, region excluded:   estimate=%.4f, p=%.4f\n",
            robust_summary(m_f8_hml_noout)["se2_c:entropy_c:ccs_c","Estimate"],
                robust_summary(m_f8_hml_noout)["se2_c:entropy_c:ccs_c","Pr(>|t|)"]))
```

II.C.7 Candidate Controls: Voluntary Work and Homelessness (Screening)

Paper3_Script13_Candidate_Voluntary_Work.R

Table C8: correlations of voluntary work and homelessness with the existing variables.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 13: CANDIDATE CONTROLS - VOLUNTARY WORK AND
# HOMELESSNESS: CORRELATION SCREENING (TABLE C8)
#
# Voluntary work (share of residents doing voluntary work) and homelessness
# (log(1 + rate) because of strong right skew), both from the 2016 Census,
# joined on the numeric SA4 code. Reports their correlations with every
# existing control and moderator and lists the regions with the highest
# homelessness rates.
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session.
# ============================================================================
f_progress_dashboard <- file.path("data", "progressdashboard_2024data (2).csv")

required_objects <- c("homeostasis_analysis", "finance_analysis", "sa4_lookup_clean")
missing_objects <- required_objects[!sapply(required_objects, exists)]
if (length(missing_objects) > 0) {
  stop(sprintf("Missing required object(s): %s. Run Paper3_Script1_Primary_Specification.R first, in the same
R session.",
               paste(missing_objects, collapse = ", ")))
}

pd_raw <- read_csv(f_progress_dashboard, show_col_types = FALSE)

# ---- Voluntary work, 2016 Census ---------------------------------------
voluntary_work_2016 <- pd_raw %>%
  filter(region_type == "sa4_code_2021", full_variable_name == "Voluntary work",
         as.numeric(date) == 2016) %>%
  transmute(sa4 = as.character(region_id), voluntary_work = value)

cat(sprintf("[CHECK] voluntary_work_2016: %d regions\n", nrow(voluntary_work_2016)))
unmatched_vw <- voluntary_work_2016 %>% filter(!sa4 %in% sa4_lookup_clean$sa4_code)
cat(sprintf("[CHECK] Unmatched sa4 codes: %d %s\n", nrow(unmatched_vw),
            if (nrow(unmatched_vw) == 0) "(all matched)" else "- check codes"))

# ---- Homelessness, 2016 Census, log-transformed ------------------------
homelessness_2016 <- pd_raw %>%
  filter(region_type == "sa4_code_2021", full_variable_name == "Homelessness",
         as.numeric(date) == 2016) %>%
  transmute(sa4 = as.character(region_id), homelessness_raw = value,
            homelessness_log = log(1 + value))

cat(sprintf("[CHECK] homelessness_2016: %d regions\n", nrow(homelessness_2016)))
cat("[CHECK] Raw vs log distribution:\n")
cat(sprintf("  RAW: mean=%.1f, sd=%.1f, min=%.1f, max=%.1f\n",
            mean(homelessness_2016$homelessness_raw, na.rm=TRUE), sd(homelessness_2016$homelessness_raw,
na.rm=TRUE),
            min(homelessness_2016$homelessness_raw, na.rm=TRUE), max(homelessness_2016$homelessness_raw,
na.rm=TRUE)))
cat(sprintf("  LOG: mean=%.2f, sd=%.2f, min=%.2f, max=%.2f\n",
            mean(homelessness_2016$homelessness_log, na.rm=TRUE), sd(homelessness_2016$homelessness_log,
na.rm=TRUE),
            min(homelessness_2016$homelessness_log, na.rm=TRUE), max(homelessness_2016$homelessness_log,
na.rm=TRUE)))

# ---- Merge onto the analysis datasets ----------------------------------
homeo_test <- homeostasis_analysis %>%
  left_join(voluntary_work_2016, by = "sa4") %>%
  left_join(homelessness_2016, by = "sa4")
fin_test <- finance_analysis %>%
  left_join(voluntary_work_2016, by = "sa4") %>%
  left_join(homelessness_2016, by = "sa4")

# ============================================================================
# CORRELATIONS WITH THE EXISTING CONTROLS AND MODERATORS
# ============================================================================
existing_controls <- c("density_100", "log_med_income_avg", "unemployment_rate_avg",
                       "age_25_64_avg", "employment_rate_avg", "creative_class_share",
```

```
                        "entropy", "specialisation_index")

cat("\n============================================================================\n")
cat("Voluntary work: correlations with existing variables\n")
cat("============================================================================\n")
for (v in existing_controls) {
  r <- cor(homeo_test$voluntary_work, homeo_test[[v]], use = "pairwise.complete.obs")
  cat(sprintf("  r(voluntary_work, %s) = %.3f\n", v, r))
}

cat("\n============================================================================\n")
cat("Homelessness (log): correlations with existing variables\n")
cat("============================================================================\n")
for (v in existing_controls) {
  r <- cor(homeo_test$homelessness_log, homeo_test[[v]], use = "pairwise.complete.obs")
  cat(sprintf("  r(homelessness_log, %s) = %.3f\n", v, r))
}

cat("\n============================================================================\n")
cat("Voluntary work and homelessness (log)\n")
cat("============================================================================\n")
cat(sprintf("  r(voluntary_work, homelessness_log) = %.3f\n",
            cor(homeo_test$voluntary_work, homeo_test$homelessness_log, use = "pairwise.complete.obs")))

# ---- Regions with the highest homelessness rates --------------------------
cat("\n============================================================================\n")
cat("Five regions with the highest homelessness rates\n")
cat("============================================================================\n")
top5_homeless <- homelessness_2016 %>% arrange(desc(homelessness_raw)) %>% head(5)
print(top5_homeless)
```

II.C.8 Omitted-Variable Sensitivity Check (N = 79)

Paper3_Script14_Omitted_Variable_Check_N79.R

Section 4.4.5: Model 8 of Table 4.6 with net migration rate, homelessness and occupational (ANZSCO) unrelated variety added together.

```
# ============================================================================
# PAPER 3 (Chapter 4) - SCRIPT 14: OMITTED-VARIABLE SENSITIVITY CHECK (N = 79)
#
# Re-estimates Model 8 of Table 4.6 with three additional candidate controls
# added together: net migration rate, homelessness (log, 2021) and
# occupational (ANZSCO) unrelated variety (2014-2020 average). Reported in
# the omitted-variable sensitivity paragraph of Section 4.4.5 (N = 79).
#
# Model: new_firms_percapita ~ se2_c * entropy_c * ccs_c + net_migration_rate
#        + homelessness_log + anzsco_uv + controls of Table 4.6
#
# Run after Paper3_Script1_Primary_Specification.R, in the same R session.
# ============================================================================
library(lmtest); library(sandwich); library(dplyr); library(readr)
library(tidyr); library(janitor); library(stringr)

stopifnot(exists("finance_analysis"), exists("migration_raw"),
          exists("sa4_lookup_clean"), exists("controls_formula_fin"))

data_dir <- "data"

if (!exists("population_avg")) {
  population_avg <- population_clean %>%
    group_by(sa4) %>% summarise(population_avg = mean(population, na.rm = TRUE), .groups = "drop")
}

# ---- 1. Net migration rate ------------------------------------------------
inflow  <- migration_raw %>% group_by(j) %>% summarise(inflow = sum(weight, na.rm = TRUE)) %>% rename(sa4 =
j)
outflow <- migration_raw %>% group_by(i) %>% summarise(outflow = sum(weight, na.rm = TRUE)) %>% rename(sa4 =
i)
net_migration_rate <- sa4_lookup_clean %>% distinct(sa4 = sa4_code) %>%
  left_join(inflow, by = "sa4") %>% left_join(outflow, by = "sa4") %>%
  mutate(inflow = replace_na(inflow, 0), outflow = replace_na(outflow, 0),
         net_flow = inflow - outflow) %>%
  left_join(population_avg, by = "sa4") %>%
  mutate(net_migration_rate = (net_flow / population_avg) * 1000) %>%
  select(sa4, net_migration_rate)

# ---- 2. Homelessness (2021, log) ------------------------------------------
pd_raw <- read_csv(file.path(data_dir, "progressdashboard_2024data (2).csv"),
                   show_col_types = FALSE)
homelessness_2021 <- pd_raw %>%
  filter(region_type == "sa4_code_2021", full_variable_name == "Homelessness",
         as.numeric(date) == 2021) %>%
  transmute(sa4 = as.character(region_id), homelessness_log = log(1 + value))

# ---- 3. Occupational (ANZSCO) unrelated variety, 2014-2020 average ------------
clean_sa4 <- function(x) x %>% str_to_lower() %>% str_trim() %>% str_squish()
anzsco_raw <- read_csv(file.path(data_dir, "ANZSCOUV.csv"), show_col_types = FALSE) %>%
  clean_names() %>%
  transmute(sa4_name = clean_sa4(sa4_region),
            y2014 = x2014_anzsco, y2015 = x2015, y2016 = x2016, y2017 = x2017,
            y2018 = x2018, y2019 = x2019, y2020 = x2020) %>%
  mutate(across(starts_with("y2"), ~ ifelse(.x < 1, NA_real_, .x)))
anzsco_uv_avg <- anzsco_raw %>%
  rowwise() %>%
  mutate(anzsco_uv = mean(c(y2014, y2015, y2016, y2017, y2018, y2019, y2020), na.rm = TRUE)) %>%
  ungroup() %>%
  left_join(sa4_lookup_clean, by = "sa4_name") %>%
  transmute(sa4 = sa4_code, anzsco_uv)

# ---- Merge and estimate ---------------------------------------------------
fin_test <- finance_analysis %>%
  left_join(net_migration_rate, by = "sa4") %>%
  left_join(homelessness_2021,  by = "sa4") %>%
  left_join(anzsco_uv_avg,      by = "sa4")
```

```
f <- as.formula(paste("new_firms_percapita ~ se2_c * entropy_c * ccs_c +",
                      "net_migration_rate + homelessness_log + anzsco_uv +",
                      controls_formula_fin))
m <- lm(f, data = fin_test)
print(formula(m)); print(nobs(m))
print(round(coeftest(m, vcov = vcovHC(m, type = "HC1"))["se2_c:entropy_c:ccs_c", ], 4))
```